

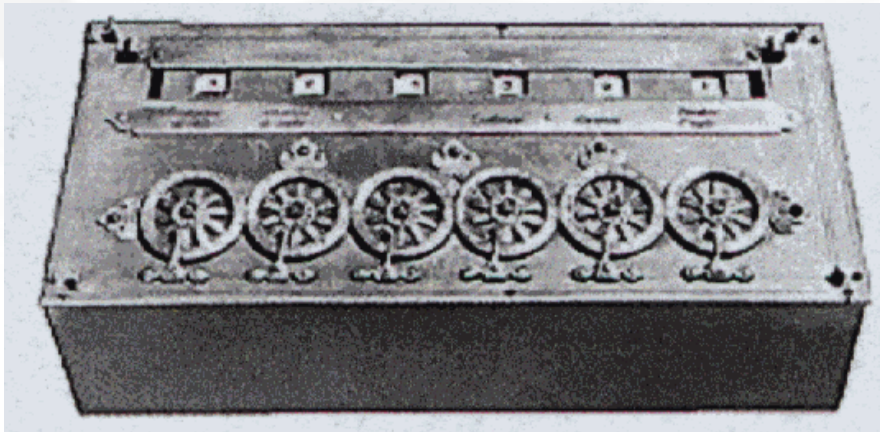


Introdução

(Aula 1)

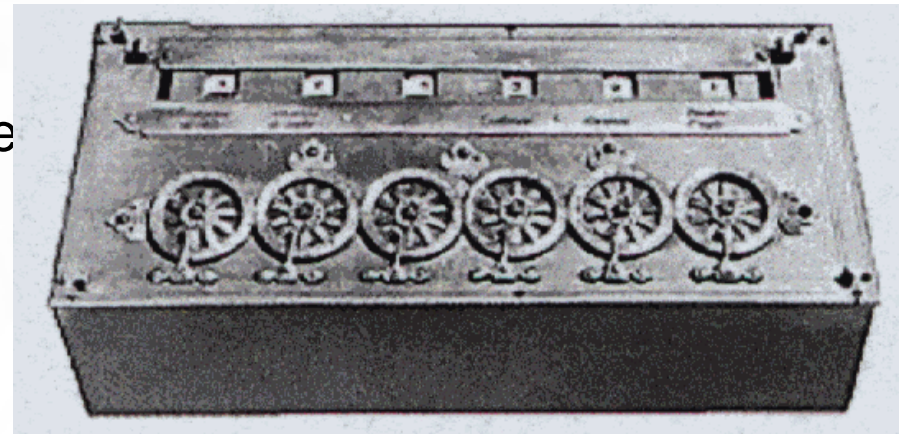
Histórico da Arquitetura de Computadores

Evolução...



Geração Zero – Computadores Mecânicos (1642-1945) ⁽¹⁾

- Blaise Pascal (1623-1662)
 - Construiu em 1642 a primeira máquina de calcular, baseada em engrenagens e alavancas, e que permitia fazer adições e subtrações.
- Leibniz (1646-1716)
 - Construiu outra máquina no mesmo estilo, mas permitia também fazer multiplicações e divisões.



Máquina de Cálculo de Pascal.

Geração Zero – Computadores Mecânicos (1642-1945) (2)

- Charles Babbage (1792-1871)

- **Máquina Diferencial:**

- Tratava tabelas de números para fins de navegação naval
 - Executava um único algoritmo: método das diferenças finitas usando polinômios; a saída era gravada em pratos de cobre e aço.

- **Máquina Analítica:**

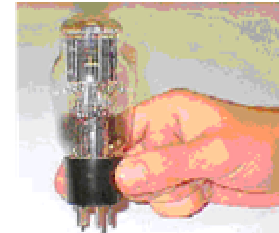
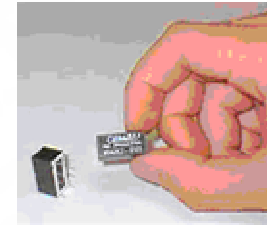
- Máquina de propósito geral (PROGRAMÁVEL!)
 - Lia instruções (de tratamento aritmético e de desvio condicional) através de cartões perfurados e as executava.
 - Quatro componentes
 - Memória, unidade de computação, unidade de entrada (**leitora de cartões perfurados**) e unidade de saída (saída impressa e com perfuração);
 - A unidade de computação lia números da memória, fazia cálculos (adição, subtração, multiplicação e divisão) e podia retornar o resultado para a memória.
 - Primeira pessoa (uma mulher!) a escrever programas: Ada Augusta Lovelace.

- Ambas eram totalmente mecânicas

Geração Zero – Computadores Mecânicos

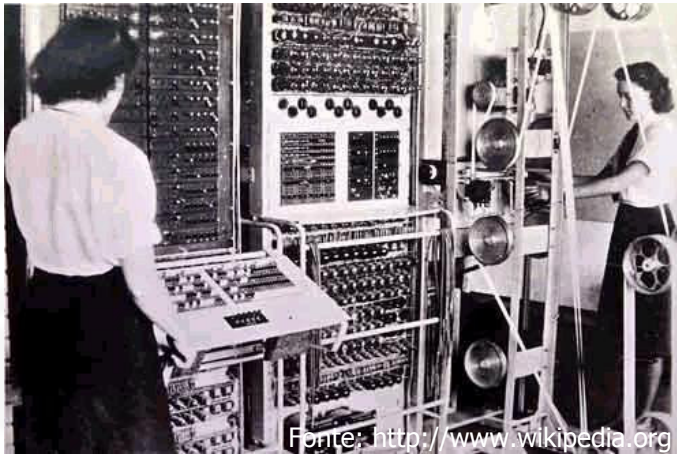
(1642-1945) ⁽³⁾

- Máquinas a relé eletromagnético
- Konrad Zuse (~ 1930)
 - Construiu durante a década de 1930 uma série de máquinas de calcular baseadas em relés.
- John Atanasoff e George Stibbitz
 - Construíram no final da década de 1930 calculadoras que já usavam aritmética binária
 - Memória baseada em capacitores.
- Howard Aiken
 - Construiu em 1944 uma máquina de propósito geral chamada **Mark I**
 - Baseada no trabalho de Babbage
 - Relés eletromagnéticos no lugar de engrenagens



Primeira Geração – Válvulas (1945-1955) ⁽¹⁾

Segunda
Guerra
Mundial

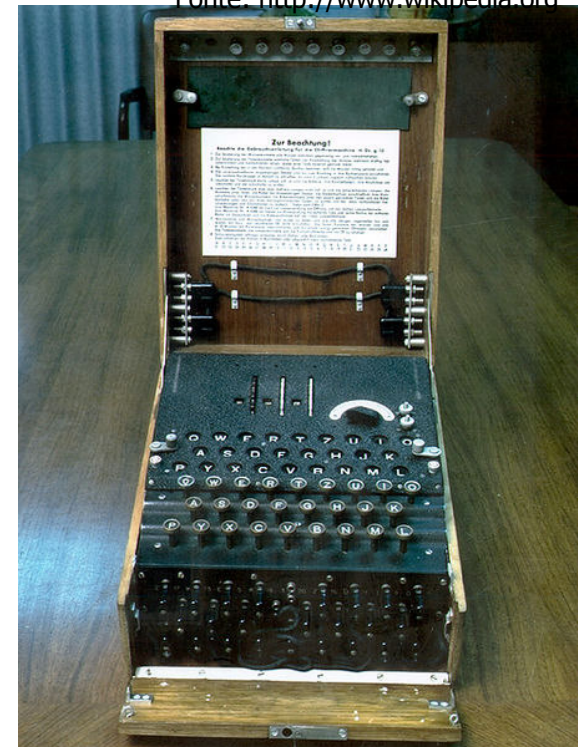


Fonte: <http://www.wikipedia.org>

Fonte: <http://www.wikipedia.org>

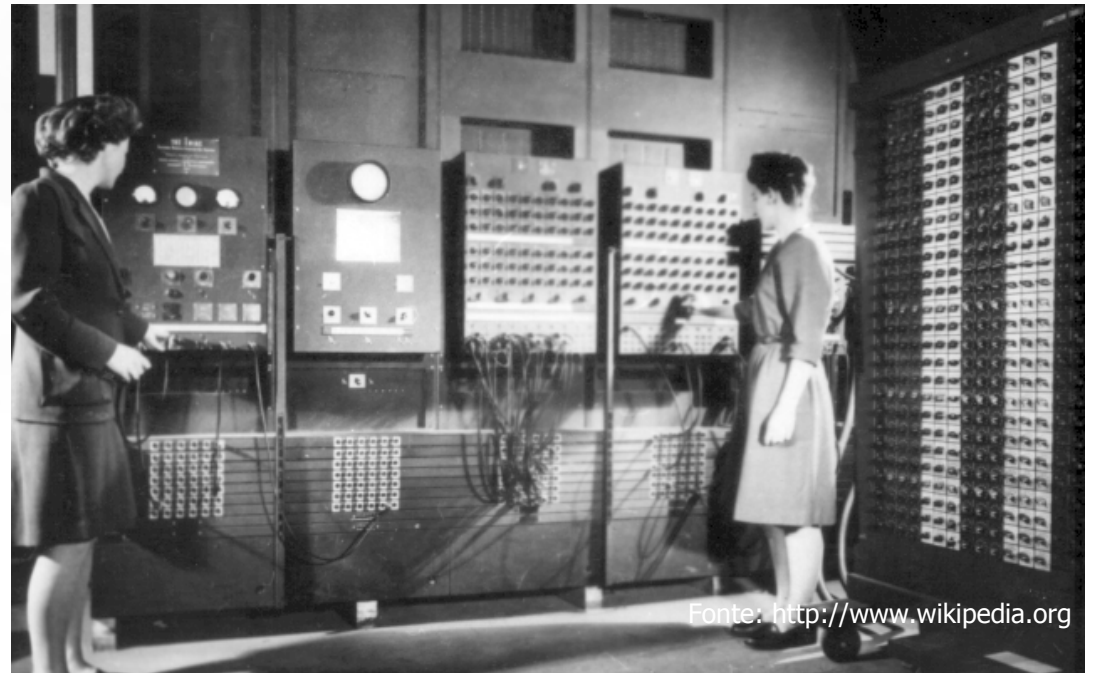
■ COLOSSUS

- 1º computador digital eletrônico
- Construído pelo Governo Britânico em 1943.
- Objetivo: decodificar as mensagens trocadas pelos alemães durante a Segunda Guerra, que eram criptografadas por uma máquina chamada **ENIGMA**.
- Participação de Alan Turing.



Primeira Geração – Válvulas (1945-1955) ⁽²⁾

- **ENIAC** (*Electronic Numerical Integrator and Computer*) (1943)
 - Computador eletrônico construído por John Mauchley e J. Presper Eckert (EUA) em 1946 para fins militares.
 - 18.000 tubos a vácuo; 1.500 relés; 30 toneladas; 140 kilowatts; 20 registradores de números decimais de 10 dígitos
 - Programação através de 6.000 *switches* e de milhares de *jumpers* (cabos de conexão)
 - Participação de John von Neumann.



Fonte: <http://www.wikipedia.org>

Primeira Geração – Válvulas (1945-1955) ⁽³⁾

- Vários computadores surgiram:
 - EDSAC (1949): Universidade de Cambridge; foi o primeiro a armazenar programas em memória
 - JOHNIAC: Rand Corporation
 - ILLIAC: Universidade de Illinois
 - MANIAC: Laboratório de Los Alamos
 - WEIZAC: Instituto Weizmann, Israel



Fonte: <http://www.wikipedia.org>



Válvula termiônica de uso geral utilizada nos primeiros computadores.
Fonte: <http://www.wikipedia.org>

Primeira Geração – Válvulas (1945-1955) (4)

■ John von Neumann

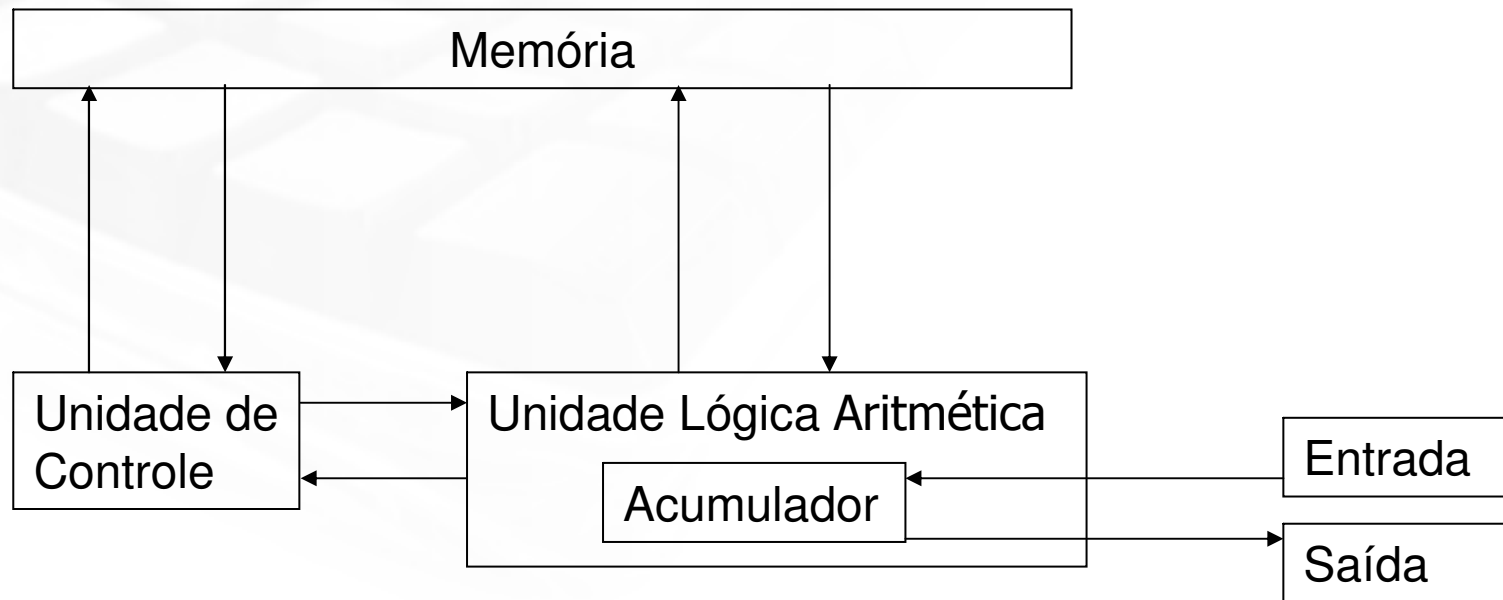
- Construiu em 1952 o computador **IAS** (*Institute for Advanced Study – Princeton, USA*)
- **Programa Armazenado**: programas e dados representados de forma digital em memória (esse mesmo proj. foi usado no EDSAC)
 - Mais flexibilidade e rapidez (ao invés de chaves e cabos)
- Processamento baseado em **aritmética binária**, ao invés de decimal
- Formalizou o projeto lógico de um computador

■ Máquina de Von Neumann

- Base de praticamente todas as máquinas atuais
- Componentes:
 - Memória
 - Unidade de Controle Unidade
 - Unidade Lógica e Aritmética (ULA)
 - Dispositivos de entrada/saída

Primeira Geração – Válvulas (1945-1955) ⁽⁵⁾

Máquina de Von Neumann



Primeira Geração – Válvulas (1945-1955) (6)

- Máquina de Von Neumann

- **Unidade de Controle de Programa**

- Determina a seqüência das instruções a serem executadas
 - Gera os sinais de controle para as outras unidades.

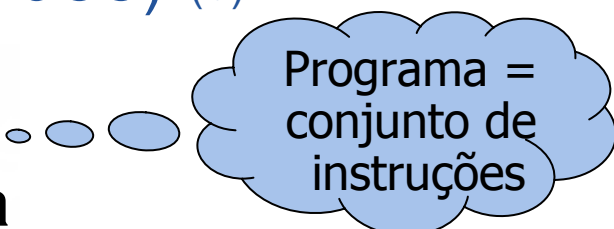
- **Memória**

- **4096** palavras de 40 bits (2 instruções de 20 bits ou um inteiro)
 - Instrução: 8 bits para indicar o tipo, **12 bits** para endereçar a memória

- **Unidade Lógica e Aritmética (ULA)**

- Execução das instruções (operações lógicas e aritméticas)
 - **Acumulador**: registrador especial de 40 bits utilizado para armazenar um operando e/ou um resultado fornecido pela ULA.

- **Dispositivos de Entrada e Saída**



Programa =
conjunto de
instruções

Primeira Geração – Válvulas (1945-1955) (7)

Máquina de Von Neumann

Ciclo de Busca-Decodificação-Execução

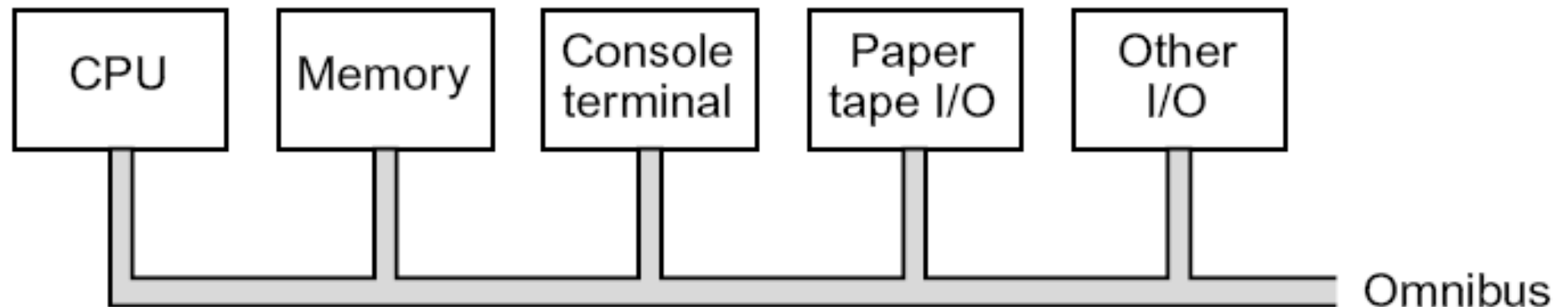
- 1. Busca:** na fase de busca é lida uma instrução da memória
- 2. Decodificação:** nessa fase é determinada qual instrução deve ser executada.
Geralmente realizada por lógica combinacional.
- 3. Execução:** para cada tipo de instrução é realizada sua execução, conforme o necessário.

Segunda Geração – Transistores (1955-1965) ⁽¹⁾

- Invenção do Transistor
 - 1948, Bell Labs, EUA
 - John Bardeen, Walter Brattain e William Shockley
- TX-0 (*Transistorized eXperimental computer 0*)
 - Primeiro computador construído com transistor (no MIT)
- PDP-1 (baratinho...\$120.000,00)
 - 1961, DEC (*Digital Equipment Corporation*)
 - Primeiro computador comercializado
 - Memória: 4K ($K=2^{10}$) palavras de 18 bits
 - Ciclo (período de clock): 5 microsegundos
 - **Clocks**: usados para a sincronização de eventos
 - Primeiro computador com display visual: tela de 512x512 pontos
 - Nascimento da indústria de minicomputadores
- PDP-8
 - Conceito de **Barramento** único:
 - Conjunto de fios paralelos (linhas de comunicação), usado para estabelecer a conexão entre os componentes de um computador.

Segunda Geração – Transistores (1955-1965) ⁽²⁾

- Barramento Único (Características e Problemas)



- Outros tipos de barramento (processador, *cache*, memória, *I/O*)
- O **desempenho** do barramento é medido pela
 - **Largura de banda:** quantidade de bits que podem ser transmitidos ao mesmo tempo
(8 bits, 16 bits, 32 bits, 64 bits, ...)
 - **Velocidade da transmissão:** medida em bps (Bits por segundo)
(10 bps, 160 Kbps, 100 Mbps, 1 Gbps, ...)

Terceira Geração – Circuitos Integrados (1965-1980) ⁽¹⁾

- Invenção do circuito integrado de silício (1958, Robert Noyce)
 - Dezenas de transistores em um único *chip*
 - Possibilitou construir computadores menores, mais rápidos e mais baratos
- System/360 (IBM, 1965)
 - Família de máquinas c/ a mesma linguagem de montagem
 - **Multiprogramação:**
 - Vários programas em memória em execução simultânea (quando um aguardava uma operação de entrada ou saída se completar, outro podia executar).
 - Pseudo paralelismo de execução através do **compartilhamento de tempo (time sharing)**

Terceira Geração – Circuitos Integrados (1965-1980) (2)

- PDP-11 (DEC, 1970)
 - Sucessor de 16 bits do PDP-8
 - Grande sucesso, especialmente nas universidades



Quarta Geração – Circuitos VLSI (1980-?) ⁽¹⁾

- VLSI – *Very Large Scale Integration*
 - Integração de Circuitos em Escala Muito Alta
- Grande compactação dos circuitos integrados
 - Dezenas de milhares, depois centenas de milhares e finalmente milhões de transistores em um chip
 - Desempenho aumentou muito
 - Preços caíram muito
 - Computadores deixaram de ser privilégio de grandes corporações
 - Início da era do **Computador Pessoal**

Quarta Geração – Circuitos VLSI (1980-?) (2)

- Primeiros Computadores Pessoais
 - Desktops
 - Software não era fornecido, o consumidor tinha que escrever seu próprio software
- Organização em 5 níveis:
 - • nível de montadores/compiladores
 - • nível de sistema operacional
 - • nível de máquina convencional
 - • nível de microprogramação
 - • nível de lógica digital

Quarta Geração – Circuitos VLSI (1980-?) ⁽³⁾

- Processadores Intel 8080
- Apple, Apple II
 - Steve Jobs e Steve Wozniak
 - Muito popular para uso doméstico e em escolas
- IBM PC - Personal Computer (IBM, 1981)
 - Intel 8088
 - Projeto de circuitos público
 - Objetivo: permitir que outros fabricassem componentes facilmente acopláveis ao PC
 - Conseqüência: indústria de clones
 - Sistema operacional: MS-DOS
 - Computador mais vendido de toda a história

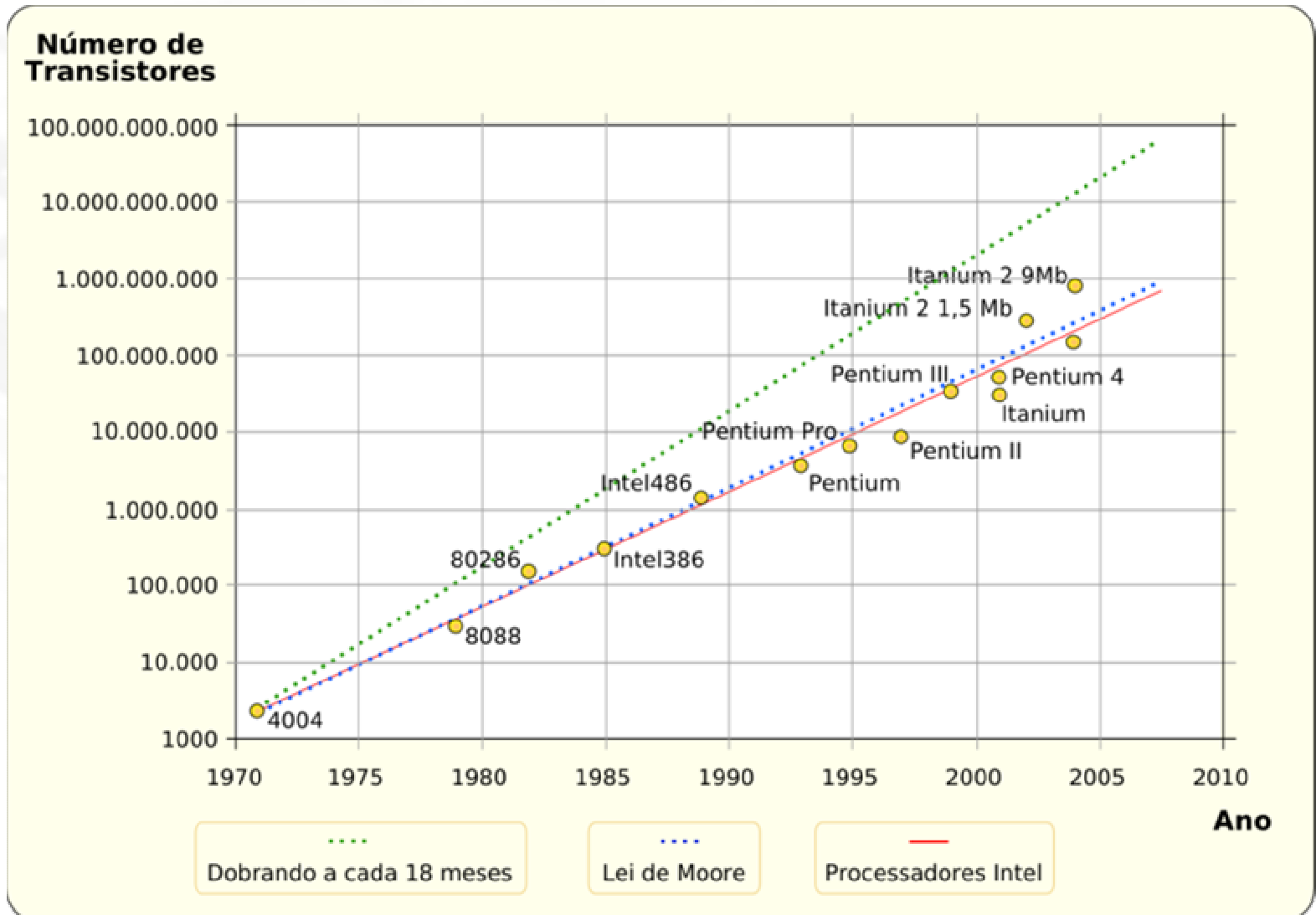
Quarta Geração – Circuitos VLSI (1980-?) (4)

- Surgimento do Windows
- Meados da década de 1980: surge os conceitos das arquiteturas RISC
- Anos 1990: processadores superescalares
 - Várias instruções em paralelo
 - Replicação de unidades funcionais (ex: ALUs)
- Computadores Multi-processados (multi-threading)
 - n processadores
- Processadores de 64 bits



Lei de Moore (1)

- Gordon Moore, 1965, Intel
- Número de transistores em um chip dobra a cada 24 meses (inicialmente 18 meses...).



Lei de Moore ⁽²⁾

- Não é uma lei, mas uma observação empírica
- **Círculo Virtuoso**
 - Avanço tecnológico propicia melhores produtos a preços mais baixos.
 - Preços mais baixos induz ao surgimento de novas aplicações
 - Exemplo: video games

Lei de Nathan (Microsoft)

“Software é como gás: expande até preencher todo o container.”

- Novas aplicações aumentam as possibilidades de mercado e fazem surgir novas empresas.
- Novas empresas leva à competição, criando demanda econômica para o avanço tecnológico.

Família de Processadores Intel ⁽¹⁾

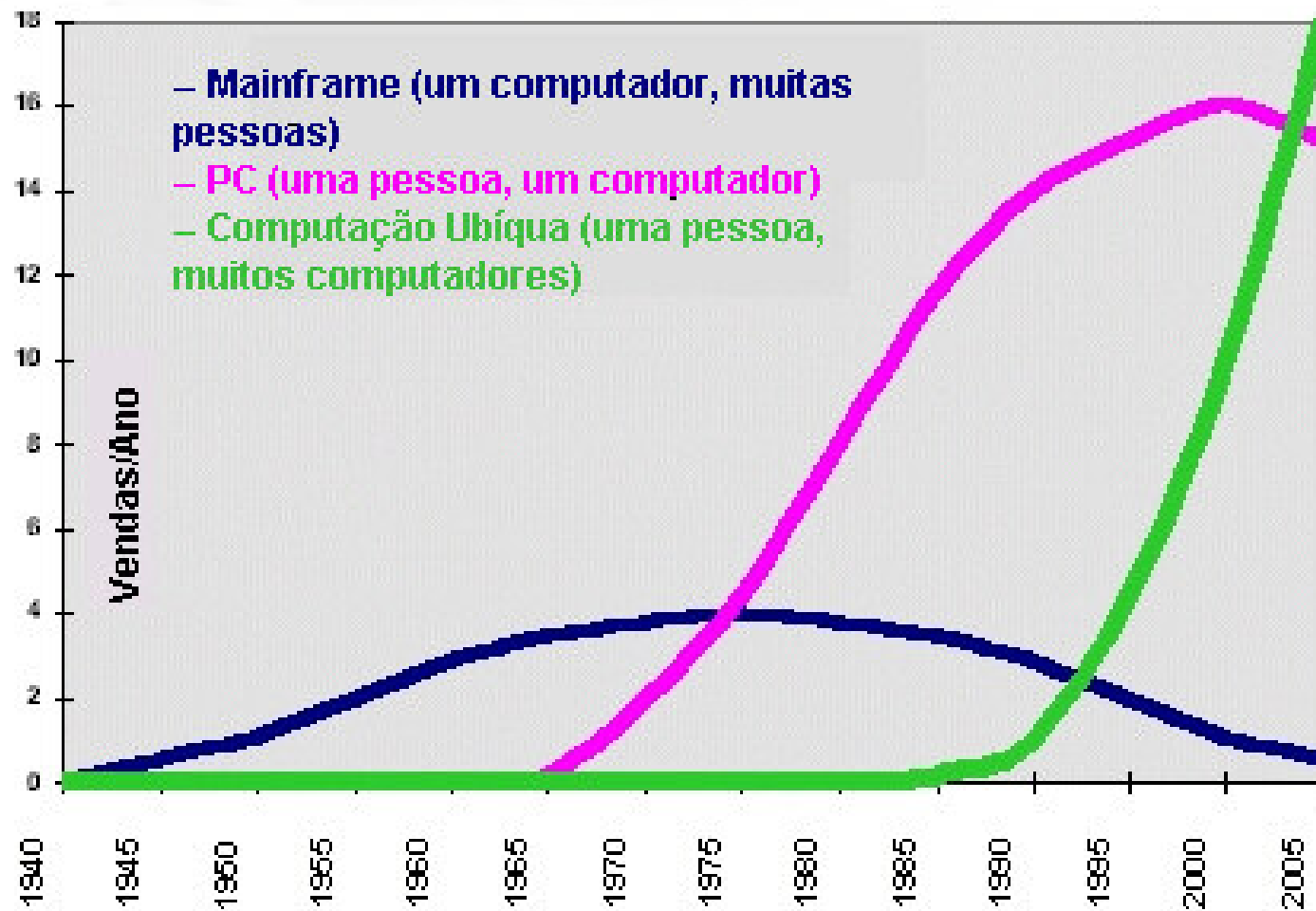
- 1978 - Arquitetura Intel **8086**
 - Barramento/registadores de 16 bits
 - Intel 8088 apareceu logo depois, mas com barramento de 8 bits (+lento/+barato)
- 1980 – Intel 8087
 - Co-processador de ponto flutuante
 - 60 instruções de ponto flutuante
- 1982 – 80286
 - Extensão do 8086
 - Reorganização da memória (espaço de endereço de 24 bits)
- 1985 – 80386
 - Extensão do 80286 para 32 bits
- 1989 – 80486
 - Memória Cache:..
- 1992 – Pentium
 - Dois *pipelines* internos (Superscalar)
- ... – Pentium Pro
 - Mudança na organização interna
 - Dois níveis de memória cache
- 1997 – Pentium II (MMX)
 - Instruções com suporte à multimídia
- 1999 – Pentium III
- 2000 a 2005 – Pentium IV
 - Variações de *clock*
- 2005 – Pentium Extreme Edition
 - 3.73 GHz
 - 64 bits
- 2006 - Intel Core 2
 - Duo (dual-core) e Quad (four-core) processadores
 - 1.60 GHz to 2.93 GHz



Família de Processadores Intel (2)

Chip	Data	MHz	Transistores	Memória	Nota
4004	1971	0,108	2.300	640	Primeiro microprocessador em um único chip
8008	1972	0,108	3.500	16 KB	Primeiro microprocessador de 8 bits
8080	1974	2	6.000	64 KB	Primeira CPU de propósito geral em um chip
8086	1978	5-10	29.000	1 MB	Primeira CPU de 16 bits em um chip
8088	1979	5-8	29.000	1 MB	Usado no IBM PC
80286	1982	8-12	134.000	16 MB	Esquema de proteção de memória
80386	1985	16-33	275.000	4 GB	Primeira CPU de 32 bits
80486	1989	25-100	1,2M	4 GB	Cache interno de 8K, coproces. aritmético interno
Pentium	1993	60-233	3,1M	4 GB	2 pipelines; MMX, Cache de 16KB
Pentium Pro	1995	150-200	5,5M	4 GB	Cache interno em dois níveis
Pentium II	1997	233-400	>7,5M	4 GB	Pentium Pro com MMX
Pentium III	1999	450-1400	>9,5M	4 GB	Serviu de base para o projeto do Pentium M
Pentium IV	1998	1.3-3.8 GHz	>55M	16 EB	Novo projeto de microarquitetura, ponto-flutuante de 64 bits, Hyper-threading, cache 166KB
Pentium M	2002	900-2260	>140M	16 EB	Suporte a frequência variável

Tendências da Computação



CISC (*Complex Instruction Set Computing*)

Conjunto Complexo de Instruções

RISC (*Reduced Instruction Set Computing*)

Conjunto Reduzido de Instruções

■ CISC

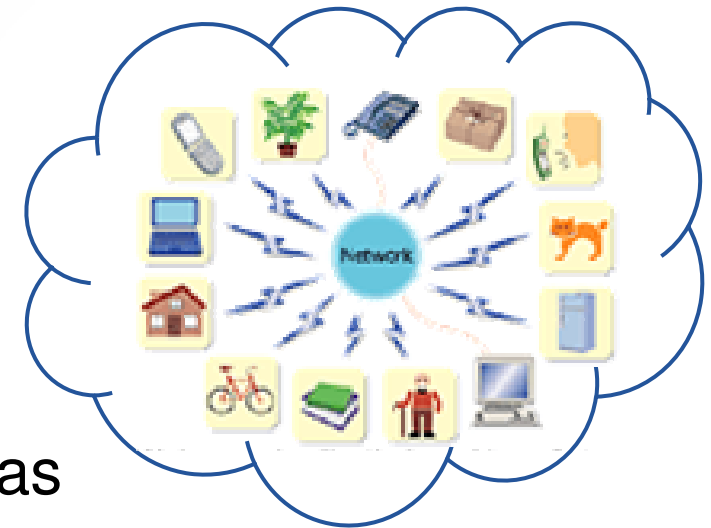
- Reconhece mais de uma centena de instruções
- É mais lento na execução das instruções (quanto > número de instruções > tempo)
- A maioria dos microprocessadores são CISC

■ RISC

- Reconhece um número limitado de instruções que, em contrapartida, são otimizadas para que sejam executadas com mais rapidez
- Redução do conjunto de instruções ao mínimo: as instruções não contempladas são executadas como combinações das existentes
- Desempenho de 50-75% superior a um CISC

Tendências da Computação

- Redes de Computadores
- *Clusters* de Computadores
- Computação Ubíqua
 - Mark Weiser, 1988
 - *“Everytime, Everywhere”*
 - Computação cada vez mais presente no dia-a-dia das pessoas
 - Nova tecnologias, miniaturização de dispositivos, ...



Referências

- Andrew S. Tanenbaum, **Organização Estruturada de Computadores**, Capítulo 1, 5ª edição, Prentice-Hall do Brasil, 2007.
- Lúcia Helena M. Pacheco, **Visão Geral de Organização Estruturada de Computadores e Linguagem de Montagem**. Universidade Federal de Santa Catarina. Centro Tecnológico, Departamento de Informática e de Estatística.
 - <http://www.inf.ufsc.br/~lucia/Arquivos-INE5607/Turma0238B/OrgEstruturada.pdf>
- <http://www.wikipedia.org>
- Página pessoal do Mark Weiser: www.ubiq.com/weiser



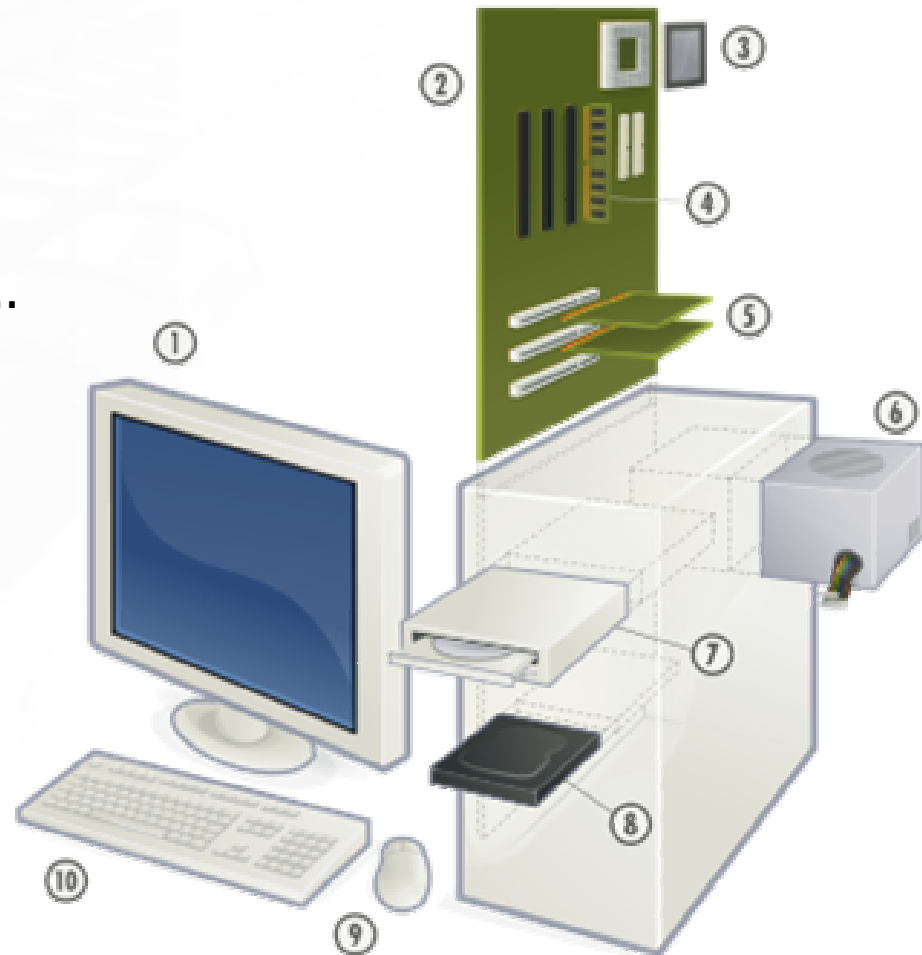
Introdução

(Aula 2)

Organização Estruturada de Computadores

Introdução – Arquitetura de Hardware

- 01- Monitor
- 02- Placa-Mãe
- 03- Processador
- 04- Memória RAM
- 05- Placas de Rede, Som, Vídeo, Fax...
- 06- Fonte de Energia
- 07- Leitor de CDs e/ou DVDs
- 08- Disco Rígido (HD)
- 09- Mouse
- 10- Teclado



Introdução – Conceitos (1)

■ Computador Digital

- É uma **máquina** que pode resolver problemas executando uma série de **instruções**.
- Máquina Programável.

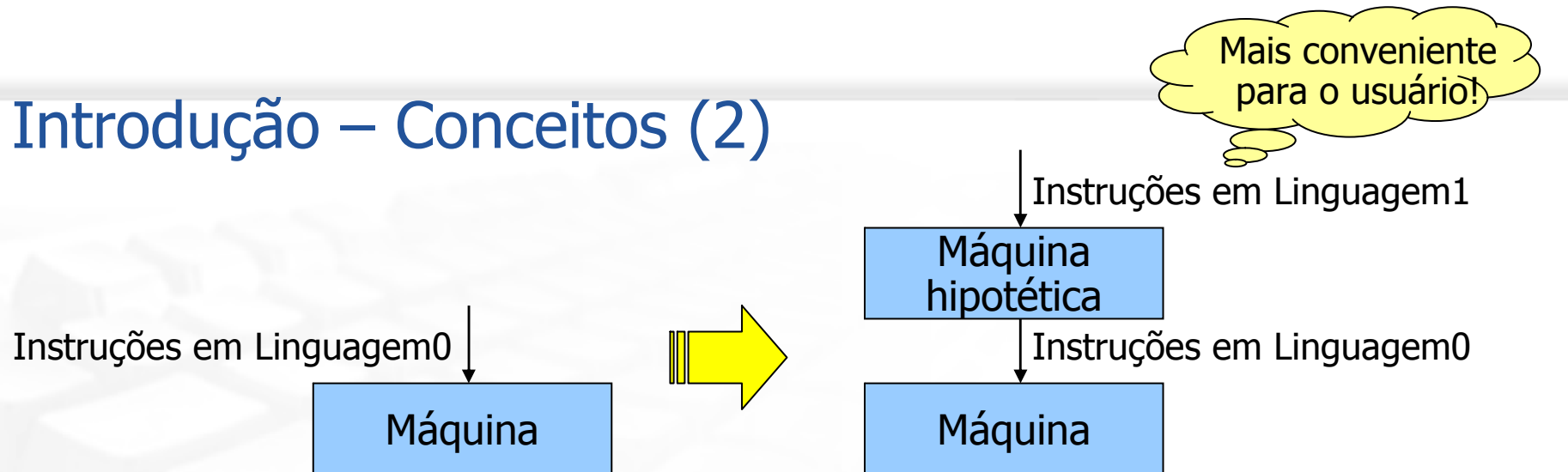
■ Programa

- Uma sequência de **instruções** que descrevem a maneira de se realizar uma determinada tarefa.

■ Linguagem de Máquina

- Conjunto de instruções básicas que os circuitos eletrônicos de um determinado computador pode reconhecer e executar diretamente
- Linguagem primitiva (binária), mas é a linguagem que a máquina realmente entende.
 - Somar 2 números
 - Comparar o valor de um número com zero.
 - Copiar um conjunto de dados de uma parte da memória para outra parte
- **Complicada para uso humano**

Introdução – Conceitos (2)



■ Organização Estruturada de Computadores

- Maneira estruturada de se organizar os computadores em diferentes **Níveis de Abstração**.
- Cada “abstração” é construída com base naquela que encontra-se imediatamente abaixo.
- Projeto de computadores de maneira sistemática e organizada.

Tradução x Interpretação ⁽¹⁾

■ Linguagens de alto nível

- C, C++, Delphi, Pascal, Fortran, Java,...
- Facilitam a comunicação dos seres humanos com a máquina, sendo linguagens mais próximas dos humanos.
- Devem ser convertidas em linguagem de máquina para serem executadas.
- Métodos de conversão: a **tradução** e a **interpretação**.

Tradução x Interpretação (2)

■ Tradução

- Programa de alto nível é primeiro **convertido** por completo para a linguagem de máquina para então ser executado.
- Em linguagens de alto nível, esta tradução é realizada pelo compilador (compilação ~ tradução)
- Ex: C, Pascal.

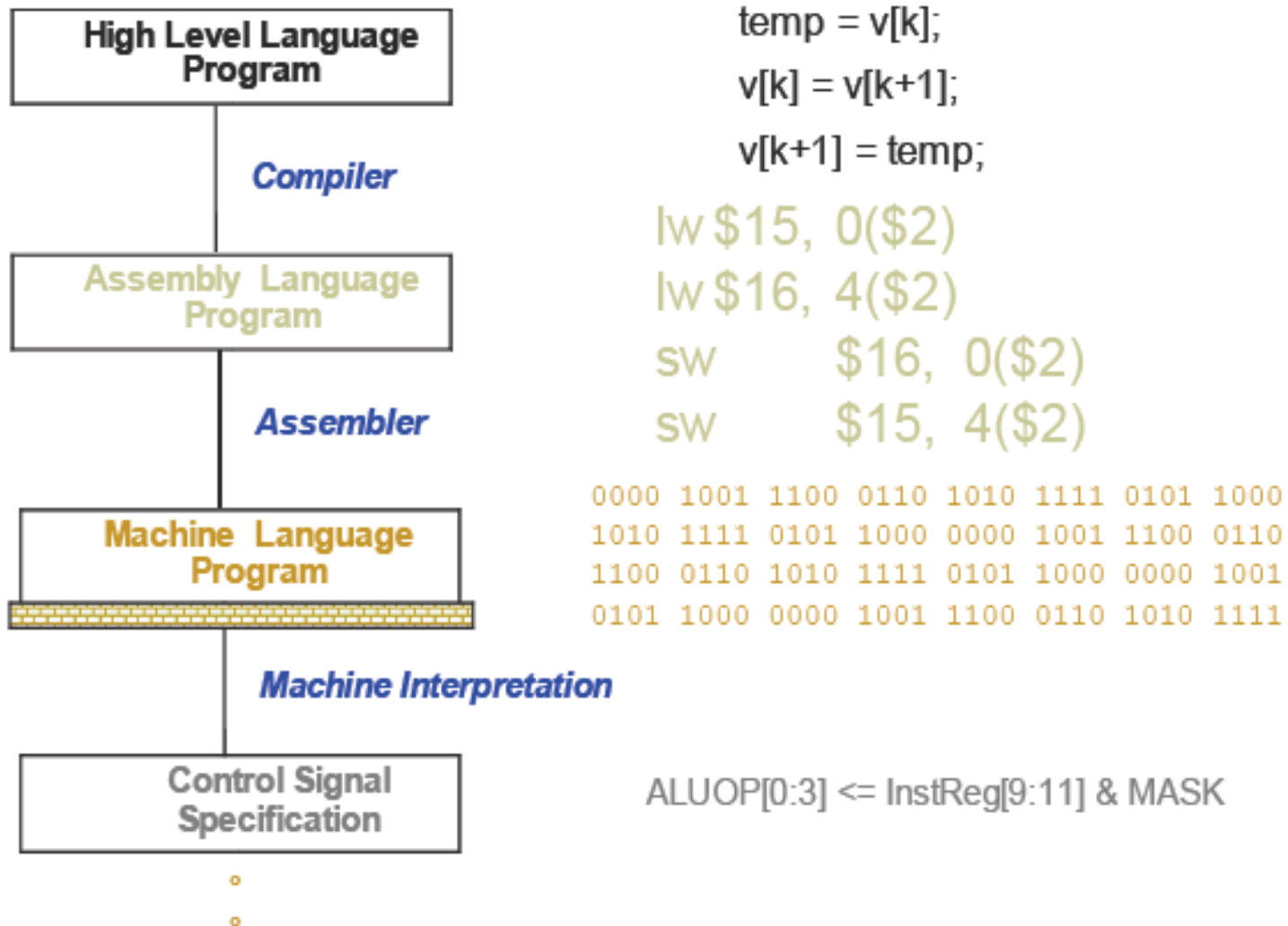
■ Interpretação

- Depois de cada instrução de alto nível ser examinada e decodificada, ela é executada imediatamente.
- Ex: Haskell.

■ Métodos Híbridos

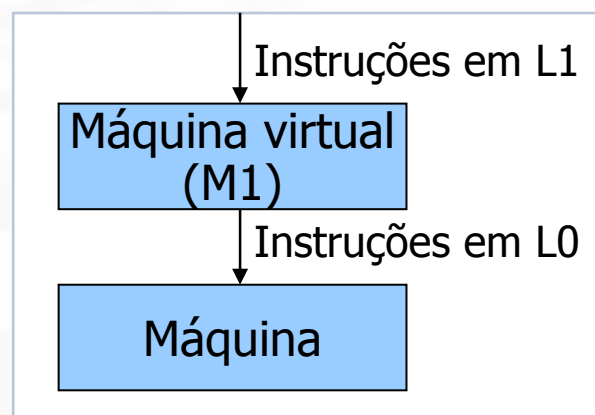
- Ex: Java

Tradução x Interpretação (3)



Máquina Virtual ⁽¹⁾

- Representa uma abstração capaz de reconhecer e executar diretamente as instruções de uma linguagem específica

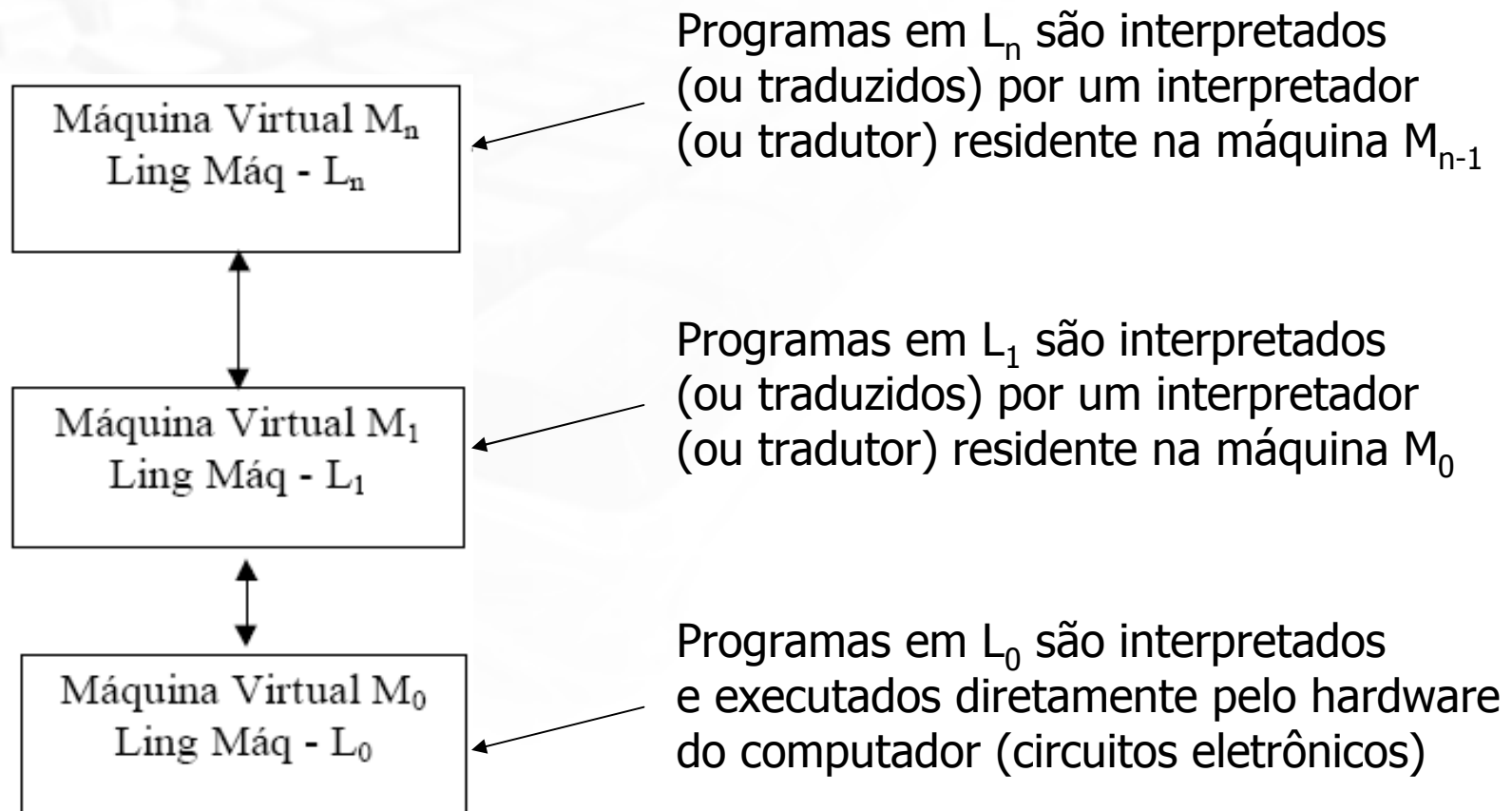


- Seja **L0** uma linguagem de máquina e **M0** a máquina capaz de executá-la
- Seja **L1** uma linguagem em um nível de abstração acima de L0.
M1 é portanto a máquina virtual associada a L1 (abstração)
 - Máquina (M1) hipotética, cuja linguagem de máquina é a linguagem L1.
 - Na prática, M1 representa um programa escrito em L0, sendo executado (interpretado) por M0
 - Os programas escritos em L1 são OU traduzidos OU interpretados por M1

Máquina Virtual

- Para que a tradução ou a interpretação sejam tarefas práticas, L0 e L1 não devem ser muito “diferentes”
- Máquinas Multi-Níveis
 - Diferentes camadas ou níveis de abstração.
 - Nível ou Camada: uma máquina (computador real ou virtual) e sua linguagem correspondente, sobre a qual uma nova camada pode ser acrescentada.
 - A linguagem ou o nível mais baixo é o mais simples, enquanto a linguagem ou o nível mais alto é o mais sofisticado.

Máquina de Vários Níveis ⁽¹⁾



Máquina de Vários Níveis (2)

- Cada máquina virtual tem associada a si uma linguagem, composta de todas as instruções que essa máquina pode executar.

Uma máquina define uma linguagem.
Uma linguagem define uma máquina.

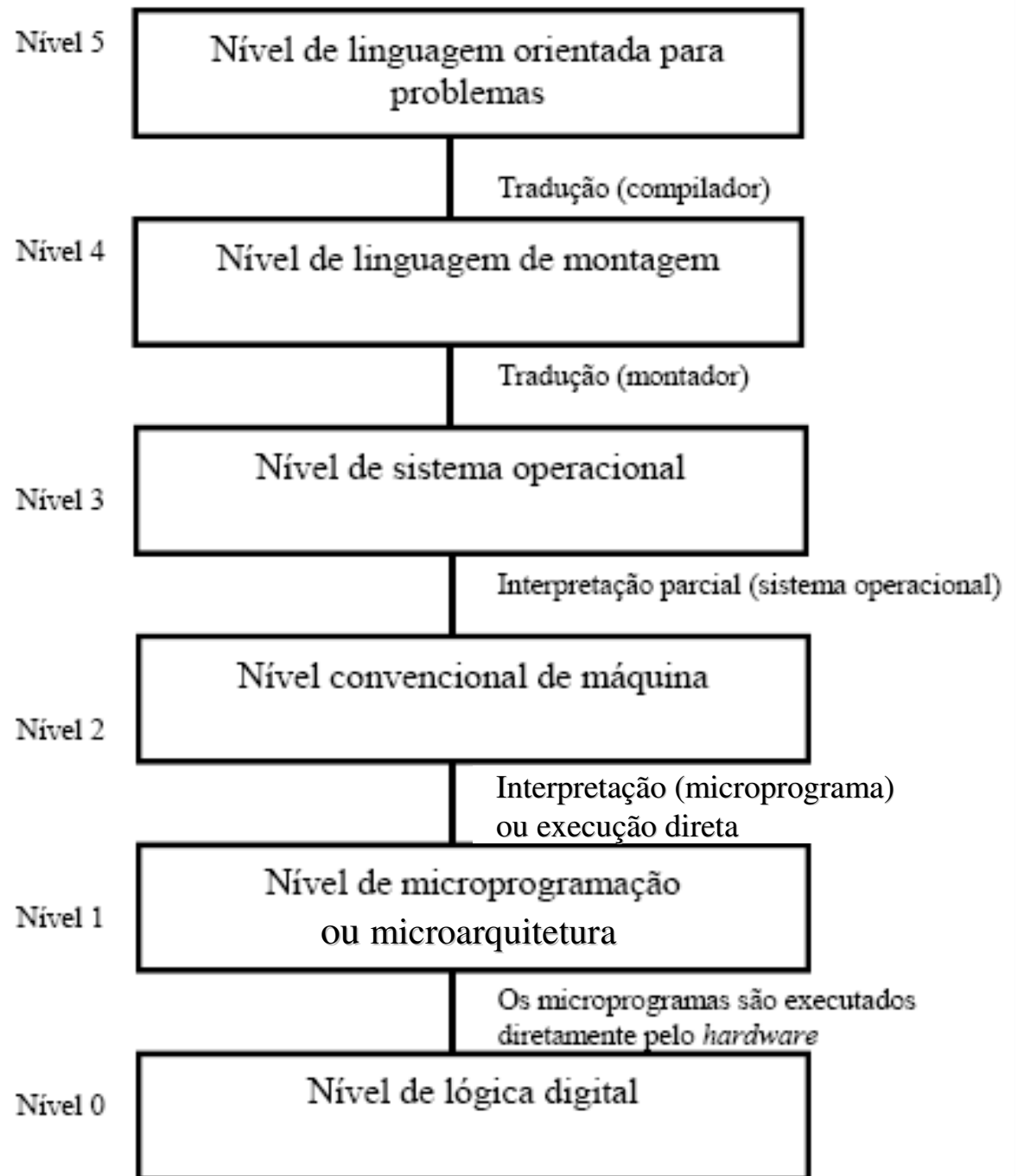
- Um computador com n níveis pode ser visto como n máquinas virtuais distintas.

Máquina de Vários Níveis ⁽³⁾

- Vantagens desta abordagem:
 - Um computador pode ser considerado como composto por "n" máquinas virtuais, cada uma delas com sua própria configuração e linguagem de máquina;
 - Uma pessoa, cujo trabalho seja gerar programas (aplicativos) para a máquina virtual de nível "k" ou outro qualquer, não precisa conhecer ou se preocupar com as particularidades dos níveis inferiores ao seu;
 - A estrutura de níveis permite ver o computador como um conjunto hierárquico de facilidades que possibilitam flexibilidade e independência ao usuário.

Máquina de Vários Níveis Modernas

“A maioria dos computadores modernos possui dois ou mais níveis, sendo que máquinas com seis ou mais níveis são cada vez mais comuns.”



Nível dos Dispositivos (-1???)

- Situado abaixo do nível 0
- Microeletrônica
- Características físicas
- Malha de transistores
- Tecnologias de fabricação de circuitos integrados

Nível 0 ou Nível da Lógica Digital

- É composto pelo hardware da máquina
- O nível 0 executa diretamente as instruções (ou microinstruções) submetidas pelo nível 1.
- **Portas Lógicas**
 - São os objetos de interesse dos projetistas de computadores nesse nível
 - Dispositivos digitais construídos a partir de componentes analógicos (Ex: transistores)
 - Ex: portas AND, OR e NOT
- Combinação de portas lógicas:
 - Funções aritméticas;
 - Memórias (registradores);
 - Processadores.

Nível 1 ou Nível da Microarquitetura ⁽¹⁾

- Nesse nível, inicia-se o conceito de programa como uma seqüência de instruções a serem executadas diretamente pelos circuitos eletrônicos.
- Enxerga-se:
 - Um conjunto de 8 a 32 registradores (memória local)
 - Um circuito chamado **ULA (Unidade Lógica e Aritmética)**
- Os registradores e a ULA são conectados para formar o **Caminho de Dados (*Data Path*)**, estrutura sobre a qual os dados fluem.
- A operação básica do caminho de dados consiste na seleção de um ou de dois registradores para que a ULA opere sobre eles

Nível 1 ou Nível da Microarquitetura (2)

- Em algumas arquiteturas, no nível 1 há um programa denominado **microprograma**
 - Função: interpretar as instruções de nível 2, executando-as em seguida
 - É o microprograma quem controla a operação do caminho de dados
- O controle do *data path* por software
 - Realizado geralmente em máquinas CISC (*Complex Instruction Set Computer*).
- Nas máquinas não microprogramadas, o caminho de dados é quase sempre controlado por hardware
 - Realizado geralmente em máquinas RISC (*Reduced Instruction Set Computer*).
- Poucas são as máquinas que têm mais de 20 instruções no nível da microarquitetura
 - A maior parte destas instruções envolve a movimentação de dados de uma parte da máquina para outra, ou alguns testes simples.

Nível 1 ou Nível da Microarquitetura ⁽³⁾

■ Controle por software

- Microprograma = Interpretador (residente em memória ROM)
 - Busca, decodifica e executa as instruções, uma a uma, usando o caminho de dados para a realização de uma tarefa.
 - Exemplo: Execução de uma instrução de SOMA (ADD)
 - A instrução deve ser buscada na memória, seus operandos devem ser localizados e trazidos para os registradores, a soma deve ser calculada na ULA, e o resultado deve ser encaminhado para o lugar apropriado

■ Controle por hardware

- Os mesmos passos acima são executados, mas sem que haja um programa armazenado para controlar a interpretação das instruções

Nível 1 ou Nível da Microarquitetura ⁽⁴⁾

- Vantagens da microprogramação
 - Facilitar o projeto e a construção dos circuitos digitais
 - Implementação de parte da lógica digital dentro do firmware
 - Flexibilizar e possibilitar o desenvolvimento de instruções mais potentes a nível de máquina convencional.

Nível 2 ou Nível ISA

- **Nível ISA – *Instruction Set Architecture***
 - Ou Nível da Arquitetura do Conjunto de Instruções
- **Nível Convencional de Máquina**
- Define o conjunto das **Instruções** executáveis por uma máquina (processador)
 - Cada máquina ou processador tem sua linguagem própria de nível 2, chamada “**Linguagem de Máquina**” (documentada em manuais específicos de cada fabricante).
- Pode ser vista como a verdadeira interface entre o **software** e o **hardware**
- Discute-se:
 - Tipo de Dados
 - Modelos de Memória e de Endereçamento
 - Formato e Tipos de Instruções
 - ...

Nível 3 ou Nível do Sistema Operacional (SO) ⁽¹⁾

- Esse nível suporta um conjunto de novas instruções, uma organização diferente da memória, a capacidade de rodar dois ou mais programas de forma simultânea, e outros.
- Fornece serviços básicos para os níveis acima
 - Interface (gráfica ou linha de comando) com o usuário
 - Gerenciamento de memória
 - Escalonamento de processos
 - Acionamento de dispositivos de entrada e saída de dados, etc.
- Geralmente desenvolvido de forma **híbrida**
 - Existem instruções do nível 3 idênticas às de nível 2, que não são executadas (interpretadas) pelo sistema operacional

Nível 3 ou Nível do Sistema Operacional (SO) ⁽²⁾

- Níveis abaixo: dirigidos aos **programadores de sistema**
 - Níveis projetados para rodar interpretadores e tradutores (ex. compiladores)
 - Os programadores de sistema são especialistas em projetar e implementar novas máquinas virtuais/interpretadores/tradutores.
 - Predominância de Interpretação.
 - Linguagens freqüentemente numéricas, bom para as máquinas, mas ruim para as pessoas.
- Níveis acima: dirigidos aos **programadores de aplicação**
 - Programadores com problemas a serem solucionados.
 - Predominância de tradução (mas nem sempre).
 - Linguagens contendo palavras e abreviações (significativas para as pessoas).

Nível 4 ou Nível de Linguagem de Montagem

- Uma forma simbólica de representação das linguagens dos níveis mais baixos.
- Provê um método para as pessoas escreverem programas para os níveis 1, 2, e 3 de uma maneira não tão desconfortável.
 - Mnemônicos para as instruções de máquina.
- Os programas escritos em linguagem de montagem são primeiramente traduzidos para a linguagem dos níveis 1, 2 ou 3, e depois interpretados.
- **Montador:** programa que executa a tradução dos programas em linguagem de montagem para uma linguagem do nível 1, 2 ou 3.

Nível 5 ou Nível de Linguagens de Alto Nível

- Linguagens projetadas para serem utilizadas por programadores de aplicação com problemas a serem resolvidos.
- Ex. de linguagens de alto nível: C, C++, Basic, Java...
- Os programas escritos nessas linguagens são geralmente traduzidos para o nível 3 ou nível 4 por tradutores conhecidos como **compiladores**, embora às vezes sejam interpretados (como no caso de Java).

Mais Níveis ???

- Acima do nível 5 encontram-se coleções de programas projetados para criar máquinas especialmente adequadas para certas aplicações (ou domínios), contendo grandes quantidades de informação acerca da aplicação.
- Máquinas virtuais voltadas às aplicações
 - Administração, educação, projeto de computadores, realidade virtual, etc.
- Dependendo do projeto da arquitetura, os níveis podem variar

Arquitetura de Computadores

- Pontos fundamentais:
 - Computadores são projetados como uma série de níveis
 - Cada nível é construído em cima de seus precursores.
- Cada nível representa uma abstração distinta, com diferentes objetos e operações presentes
 - **Arquitetura do Nível:** conjunto de tipos de dados, instruções e características
- Abstrai-se o que é irrelevante, reduz a complexidade e foca no que interessa.

Arquitetura de Computadores:

é o estudo de como projetar as partes de um sistema de computador visíveis aos programadores.

Hardware, Software e Firmware (1)

■ Hardware

- É composto por objetos tangíveis (parte física) - circuitos integrados, placas de circuito impresso, cabos, fontes de alimentação, memórias, impressoras, etc.

■ Software

- É composto de instruções, algoritmos e por suas representações computacionais - os programas.

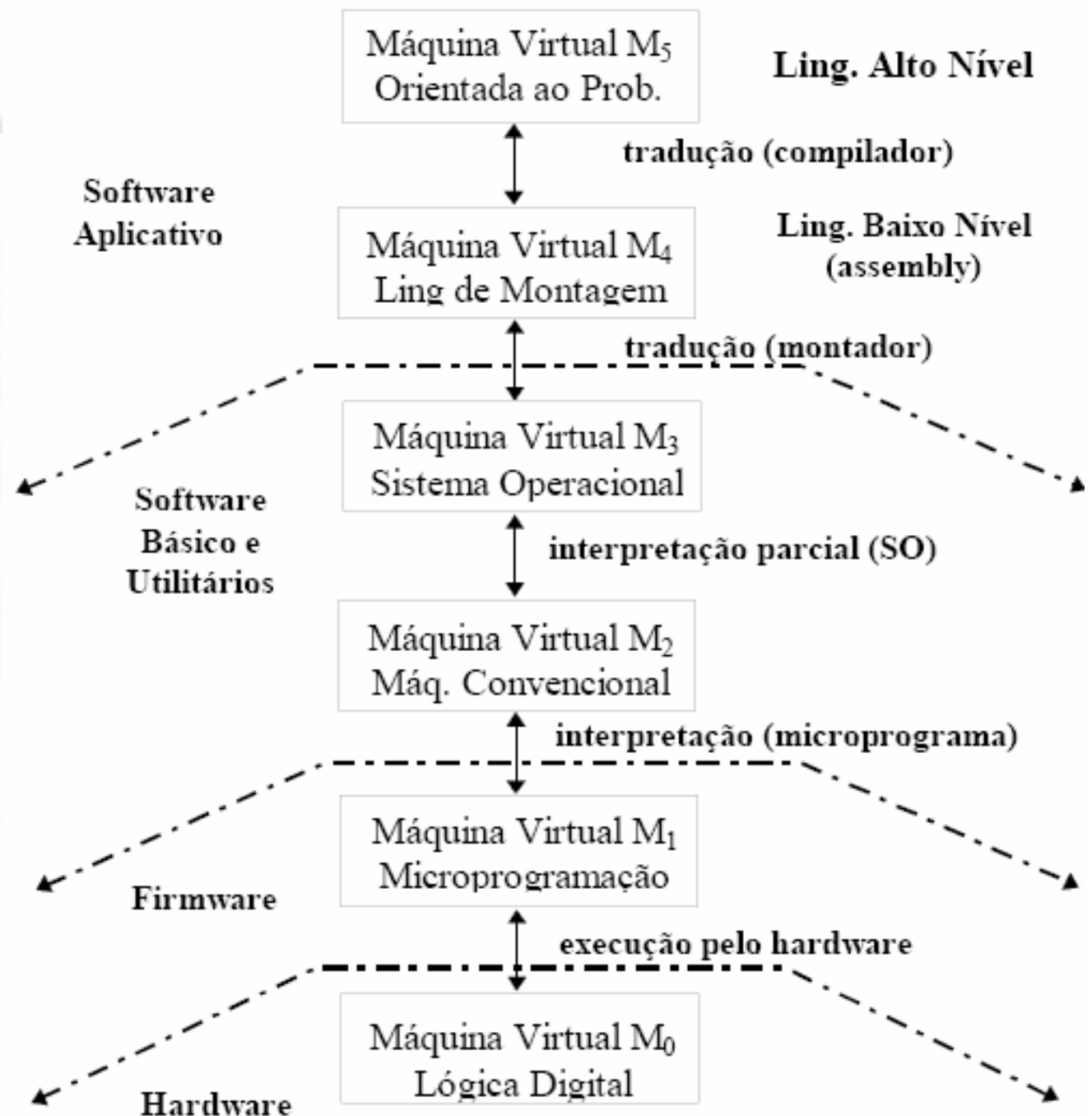
■ Hardware e Software são logicamente equivalentes.

- Qualquer operação efetuada pelo software pode também ser implementada pelo hardware... E qualquer instrução executada pelo hardware, pode também ser simulada pelo software

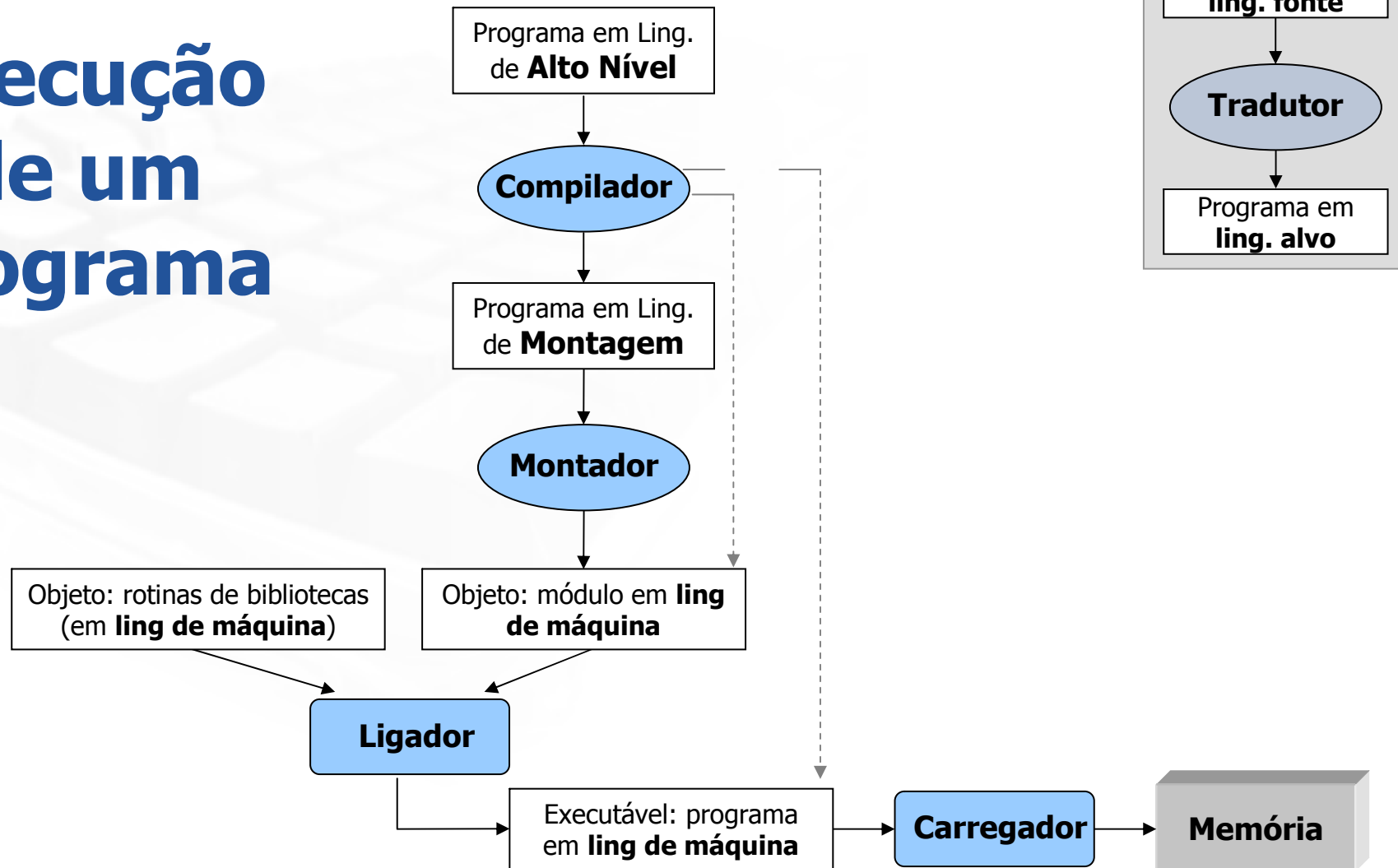
■ Firmware

- Software embarcado (embutido no dispositivo durante a fabricação)
- Controla o hardware diretamente. Ex.: BIOS (*Basic Input/Output System*).
- Em muitos computadores o microprograma está em firmware

Hardware, Software e Firmware (2)



Execução de um Programa



Compiladores, Montadores, Ligadores e Carregadores

(1)

■ **Compiladores**

- São programas que recebem como entrada arquivos texto contendo módulos escritos em linguagem de alto nível e geram como saída programa em linguagem de montagem (ou diretamente arquivos objeto) correspondentes a cada módulo.
- Se todas as bibliotecas ou módulos são apresentados como entrada, geram um programa executável diretamente.

■ **Montadores** (*Assemblers*)

- Montam um programa em linguagem de máquina a partir de sua versão em linguagem de montagem.
- Geram um **arquivo objeto**. Em geral, não pode ser executado diretamente pela máquina, por conter referências a sub-rotinas e dados especificados em outros arquivos (bibliotecas).

Ambos são TRADUTORES!

Compiladores, Montadores, Ligadores e Carregadores

(2)

■ **Ligadores (Linkers)**

- São programas especiais que recebem como entrada arquivos objetos e geram como saída o programa final em linguagem de máquina.
- Gera um programa executável a partir de um ou mais arquivos objeto.

■ **Carregadores (Loaders)**

- Para executar um programa, um loader deve ser utilizado.
- O carregador é, em geral, parte do sistema operacional.

Interpretadores

■ Interpretadores

- Recebem como entrada arquivos texto contendo programas em linguagem assembly, ou linguagem de alto nível, ou arquivos binários com instruções de máquina, e os executam diretamente.
- Interpretadores percorrem os programas, a partir de seu ponto de entrada, executando cada comando.
- Processadores são interpretadores implementados em hardware!

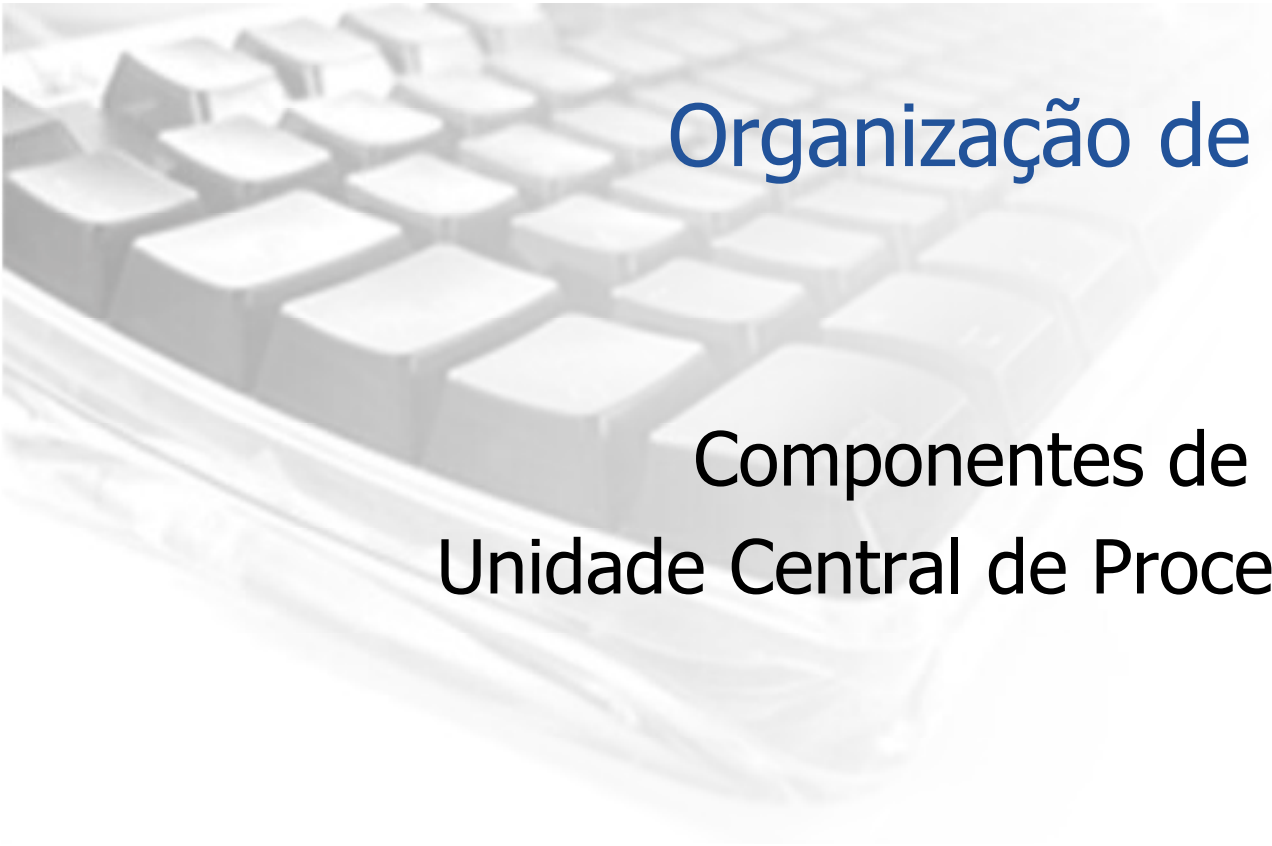
RISC x CISC

- CISC - *Complex Instruction Set Computer*
 - Arquitetura cujo processador é capaz de executar centenas de instruções complexas diferentes, sendo assim extremamente versátil.
 - Exemplos: 386, 486 da Intel.
 - Muitas das instruções guardadas no próprio processador.
- RISC - *Reduced Instruction Set Computer*
 - Uma linha de arquitetura de computadores que favorece um conjunto simples e pequeno de instruções.
 - Exemplos: SPARC, MIPS, PowerPC, DEC Alpha, etc.
 - Considerado mais eficiente e flexível que as CISC
 - As instruções tendem a ser executadas em poucos (ou mesmo um único) ciclos de relógio.
- Tamanho do código X Desempenho
 - Geralmente, o desempenho de um RISC é melhor do que de um CISC;
 - Código gerado por um RISC tende a ser mais longo e complexo.

Referências

- Andrew S. Tanenbaum, ***Organização Estruturada de Computadores***, Capítulo 1, 5ª edição, Prentice-Hall do Brasil, 2001.
- Lúcia Helena M. Pacheco, **Visão Geral de Organização Estruturada de Computadores e Linguagem de Montagem**. Universidade Federal de Santa Catarina. Centro Tecnológico, Departamento de Informática e de Estatística.

<http://www.inf.ufsc.br/~lucia/Arquivos-INE5607/Material%20anterior/OrganizacaoEstruturada170906.pdf>



Organização de Computadores

(Aula 3)

Componentes de um Computador
Unidade Central de Processamento (CPU)

Componentes de um Computador ⁽¹⁾

- Computador Eletrônico Digital

- É um sistema composto por:

- Memória – Principal e Secundária
 - Unidade Central de Processamento (*CPU – Central Processing Unit*) ou Processador
 - Unidades de Entrada/Saída (*I/O – Input/Output Unit*)
 - Barramentos (vias de interligação)

Componentes de um Computador (2)

■ Memória

- Função: Armazenar dados e programas (conjunto de instruções), permitindo o processamento automático dos dados.
- Para serem executados pela CPU, os programas devem estar armazenados na Memória (como veremos, Memória principal).
- Unidade básica é o Bit
- Diferentes maneiras de se organizar a memória
 - Hoje em dia o padrão é o byte
 - Cada byte representa um endereço de memória

Componentes de um Computador ⁽³⁾

Tipos de Memória

■ Memória Principal (MP)

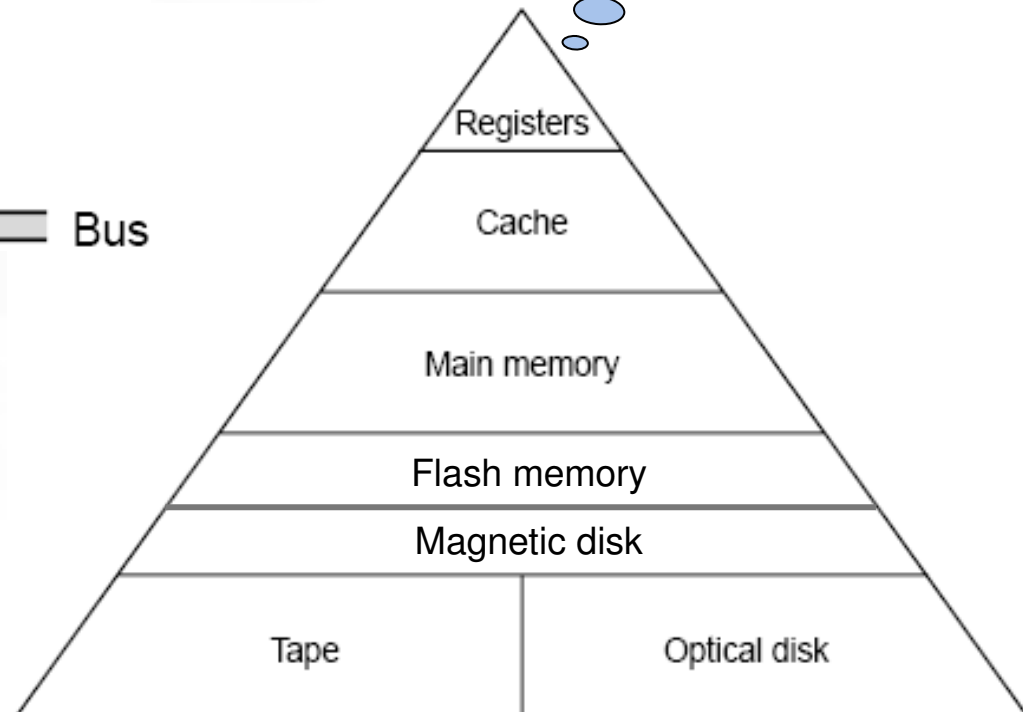
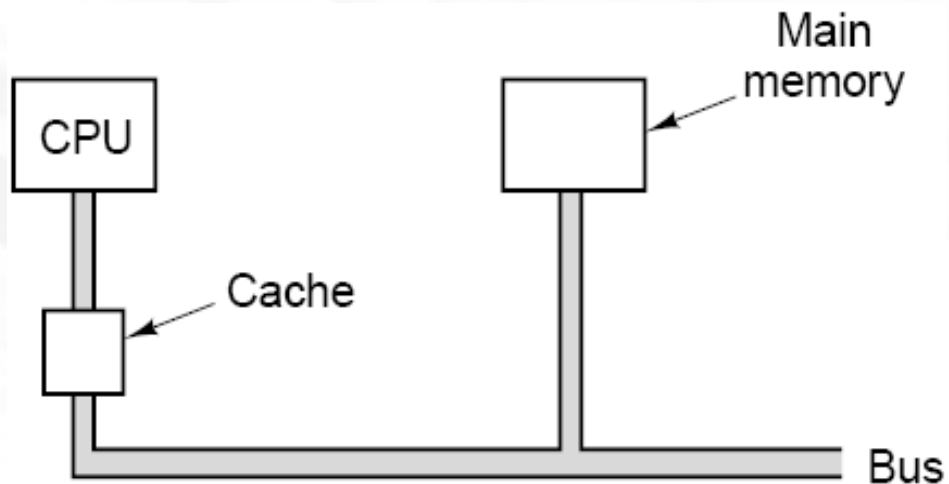
- Limitada (armazena somente alguns programas e dados).
- É a memória de trabalho, nela são carregados todos os programas e dados usados pela CPU no momento.
- Geralmente o seu conteúdo será perdido uma vez que o computador seja desligado.
- Ex.: RAM (*Random Access Memory*)

■ Memória Secundária (MS)

- Grande capacidade de armazenamento
- Mais lentas do que a MP
- Geralmente não-voláteis, permitindo guardar os dados permanentemente.
- Ex.: Discos rígidos (HD - *Hard Disk*), CDs, DVDs, disquetes, Pen drives, etc.

■ E a Memória Cache?

Componentes de um Computador (4)



Componentes de um Computador ⁽⁶⁾

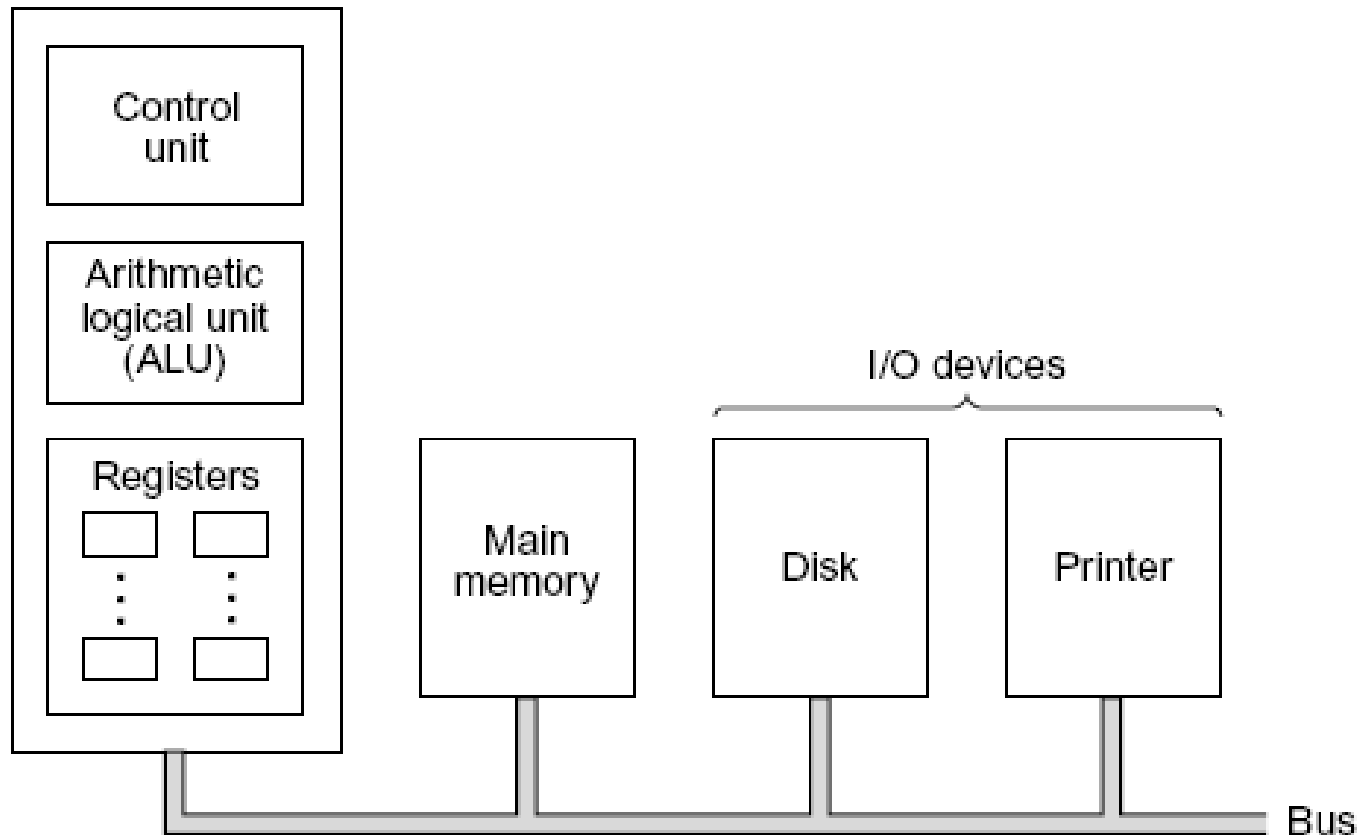
- **Unidade Central de Processamento – Processador**
 - Função: Executar os programas armazenados na MP, buscando cada instrução, interpretando-a e em seguida a executando.
 - **Unidade de Controle (UC):** busca instruções na MP e determina o tipo de cada instrução.
 - **Unidade Lógica e Aritmética (ULA):** realiza um conjunto de operações necessárias à execução das instruções.
 - **Conjunto (Banco) de Registradores:** pequena memória de alta velocidade, usada para armazenar resultados temporários e certas informações de controle.

Componentes de um Computador (7)

- Dispositivos de Entrada/Saída
 - Função: Servem para a comunicação do computador com o meio externo.
 - Muitos são usados para a interação com o usuário
 - Transformam e transferem da representação interna dos dados (sinais eletrônicos usados no computador) para a representação externa usada por pessoas ou vice-versa.
 - Ex.: impressoras, teclados, mouses, monitores, caixas de som, **discos**, etc.

Componentes de um Computador (8)

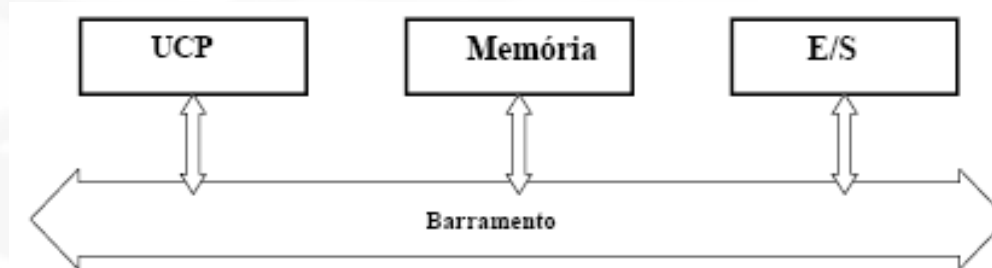
Central processing unit (CPU)



Componentes de um Computador ⁽⁹⁾

■ Barramentos (*Bus*)

- Função: interconectar os demais elementos básicos.



- Nesse modelo, é possível transferência direta de dados entre os dispositivos de E/S e a memória
- Transferência feita por dispositivos especiais
 - Controlador de **DMA (Direct Memory Access)** ou processador E/S
 - Assumem momentaneamente o controle do barramento e "isolam" a comunicação CPU com a memória e dispositivos E/S.

Componentes de um Computador (10)

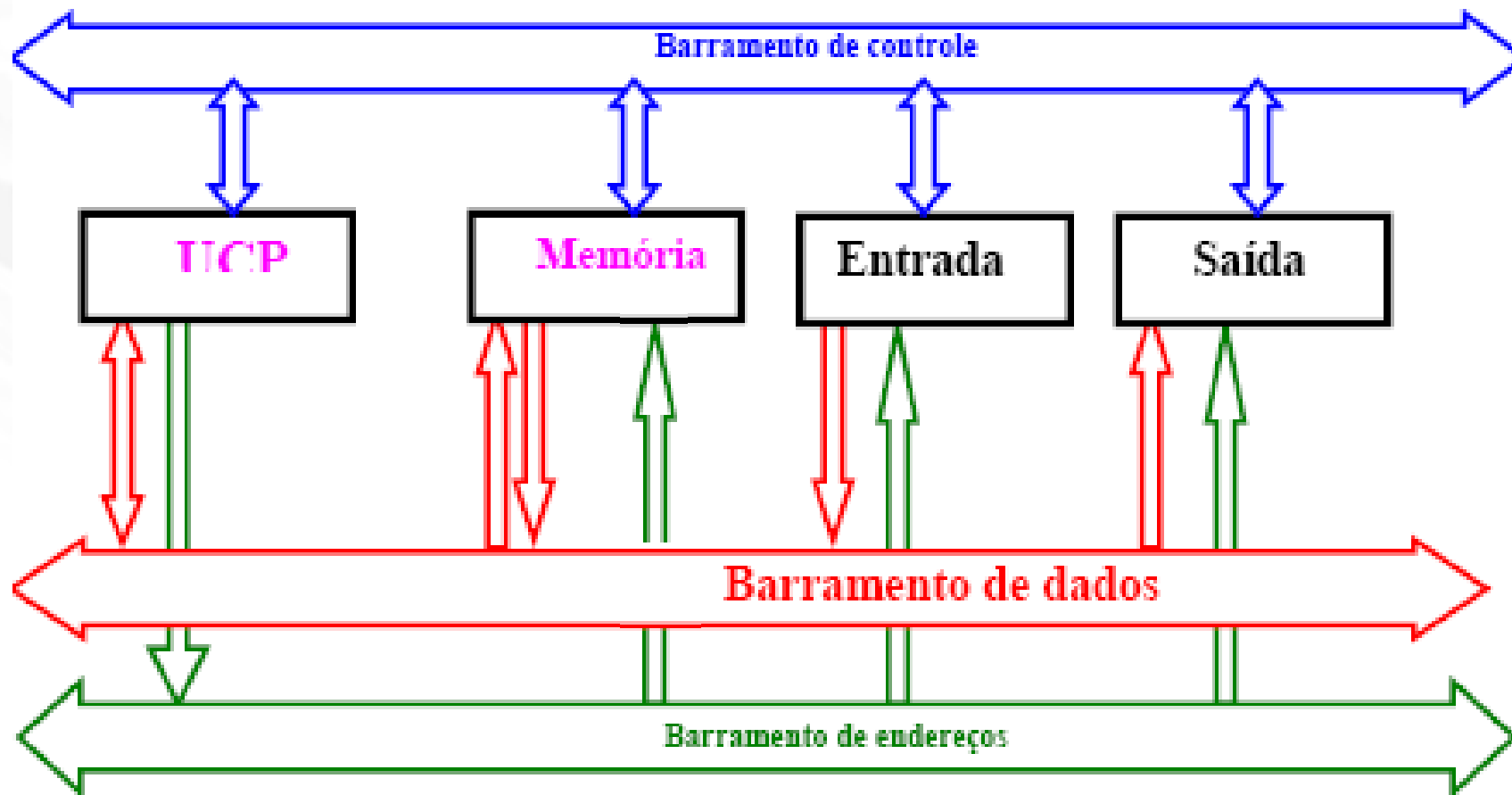
■ Barramentos

- Unidirecionais: conduzir os sinais em uma única direção
- Bidirecionais: conduzir ora em uma direção e ora em outra de acordo com comandos eletrônicos
- Podem ainda ser:
 - Três estados: bidirecionais acrescidos da possibilidade de se desconectarem eletricamente
 - Unidirecional com a possibilidade de desconexão elétrica
 - A possibilidade de desconexão elétrica controlada eletronicamente torna viável a comunicação seletiva entre os dispositivos.
 - Por exemplo, a CPU se comunica com a memória e se desconecta dos dispositivos E/S.

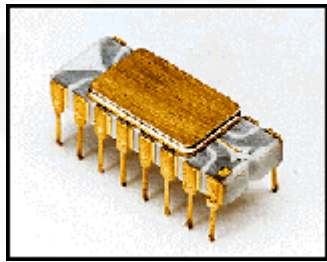
Componentes de um Computador (11)

- Tipos de barramentos:
 - **Barramento de Dados:** responsável pela transferência de dados e instrução entre os dispositivos.
 - É de três estados entre a memória e a CPU e unidirecional com a possibilidade de desconexão elétrica nos dispositivos E/S.
 - **Barramento de Endereços:** conduz o endereço a ser selecionado na memória ou dispositivos E/S.
 - É geralmente unidirecional entre a CPU (que define o endereço) e a memória e os dispositivos E/S.
 - **Barramento de Controle:** transfere os sinais de controle que ativam ou desativam os dispositivos, que selecionam determinado modo de operação ou sincronizam os circuitos.
 - Formam a **Arquitetura de Barramentos**

Componentes de um Computador (12)



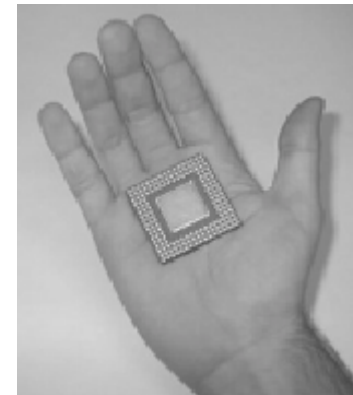
Unidade Central de Processamento - CPU (1)



Intel 4004



Pentium II – Slot 1



→ o processador ainda é o componente básico de qualquer computador

Unidade Central de Processamento - CPU (2)

- Conceitos básicos

- Instruções: comando que define integralmente uma operação a ser executada
 - Programas: instruções ordenadas logicamente
- “... Para que um programa possa ser executável ele precisa ser construído de uma série de instruções de máquina...”

- O **processador** é o “cérebro” do computador

- Função:

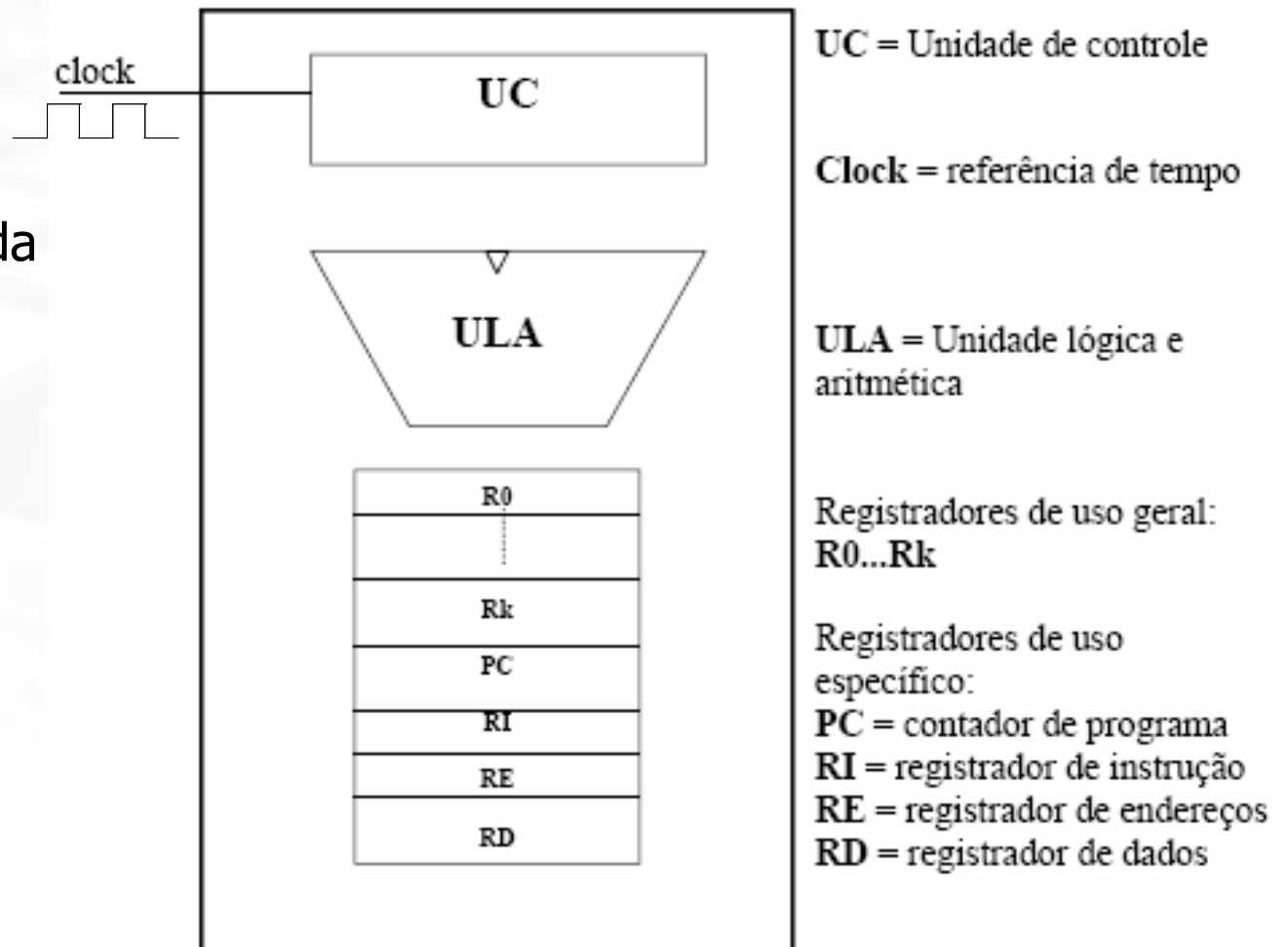
- Executar os **programas** armazenados na memória principal, buscando cada uma das **instruções** do programa, examinando-as, e executando-as uma após a outra.

Unidade Central de Processamento - CPU ⁽³⁾

- Gera sinais de controle para os demais componentes do computador, além de reconhecer e responder sinais externos (**interrupções**)
- A CPU é responsável pela **supervisão e controle** de todo o **funcionamento** da máquina e pela realização das operações lógicas e aritméticas sobre os dados
- **Clock** (ou Relógio) é a referência de tempo necessária a CPU.
 - Circuito eletrônico oscilador que gera uma forma de onda quadrada, essencial para o seqüenciamento das operações eletrônicas realizadas pela CPU.
 - Está relacionado com a frequência (taxa) de operação do processador.

Unidade Central de Processamento - CPU (4)

- A UCP pode ser dividida em :
 - Unidade de controle (UC)
 - Unidade lógica e aritmética (ULA)
 - Registradores



Unidade Central de Processamento - CPU (5)

■ **Unidade de Controle (UC)**

- Função: dirigir e coordenar as atividades das demais unidades do sistema
- É a parte mais complexa da UCP (o centro nervoso)
- Controla todos os componentes e dispositivos de um computador, emitindo sinais elétricos (pulsos de controle)
 - Sincronizados através do clock
- Responsável pela busca das instruções na memória principal, sua decodificação e execução.
 - Controla a ação da ULA e a movimentação de dados entre a UCP e a memória
 - Solicita a busca da instrução que será executada, armazena-a em um **registrador**
 - Dispara ações necessárias para execução da instrução

Unidade Central de Processamento - CPU ⁽⁶⁾

■ **Unidade de Controle (UC)** (cont.)

■ **Contador de Instrução (*Program Counter – PC*):**

- Armazena o **endereço** da (aponta para a) próxima instrução a ser executada.
- Determina a seqüência (fluxo) de execução de um programa, endereçando instrução por instrução

■ **Registrador de Instruções (Instruction Register – *RI*)**

- Armazena (contém) a instrução que está sendo executada.

Unidade Central de Processamento - CPU (7)

■ **Unidade de Lógica e Aritmética (ULA)**

- Possui os circuitos necessários para executar operações lógicas e aritméticas.
 - Exemplos: Somas, subtrações, determinação de sinal, comparações algébricas de números, verificar se um número é maior ou menor que zero, etc
- ... os operandos utilizados pela ULA são armazenados em registradores internos da UCP
- ... o resultado das operações da ULA também é armazenado em um registrador
 - **Acumulador (ACC)**

Unidade Central de Processamento - CPU ⁽⁸⁾

Instrução

- Comece a ler entradas a partir da unidade de fitas 2
- Se X é maior que Y, vá para o passo 46 do programa
- Multiplique A por B e armazene o resultado em C

Local de execução

- Unidade de controle
- Unidade lógica e aritmética (instrução lógica)
- Unidade lógica e aritmética (instrução aritmética)

Unidade Central de Processamento - CPU ⁽⁹⁾

- Conjunto (Banco) de Registradores
 - Memória interna de pequena capacidade de armazenamento, mas de alta velocidade, usada para armazenar resultados temporários e certas informações de controle
 - Local onde os dados ficam armazenados enquanto são processados ou enquanto sujeitos a operações lógicas.
 - Formada por:
 - **Registradores de uso geral:** acessíveis ao usuário no nível convencional de máquina
 - **Registradores de uso específico:** acessíveis diretamente apenas pela UC durante a execução de um programa.
 - O número de registradores e seus tamanhos variam entre arquiteturas de computadores

Unidade Central de Processamento - CPU (10)

- Conjunto (Banco) de Registradores (cont.)
 - Contador de programa (PC)
 - Aponta para as instruções a serem processadas
 - Incrementado ao fim do processamento da instrução
 - Registrador de instrução (RI)
 - Armazena o código binário da instrução propriamente dita
... modificado toda vez que uma nova instrução deve ser executada
 - Registrador de endereço de memória (REM)
 - Mantém o endereço da posição de memória usada como operando
... modificado quando é necessário apontar para um operando em outra posição de memória

Unidade Central de Processamento - CPU ⁽¹¹⁾

■ Instruções

- São comandos atômicos fornecidos ao computador para que ele execute uma tarefa específica (soma, teste, desvio, etc)
- Uma série dessas instruções constituem um programa.
- Cada computador (leia-se processador) tem um repertório particular de instruções, cuja rapidez de execução é um fator determinante na “potência de computação” (performance) da máquina
- Conforme proposto por Von Neumann, as instruções, tal qual os dados, **devem residir na MP**, e dispostas **sequencialmente**.
- A seqüência de execução pode ser alterada por **Instruções de Desvio**

Unidade Central de Processamento - CPU (12)

■ Formato das Instruções

Código da Instrução (<i>OpCode</i>)	Operando - <i>Op</i> (Informação Complementar)
--	---

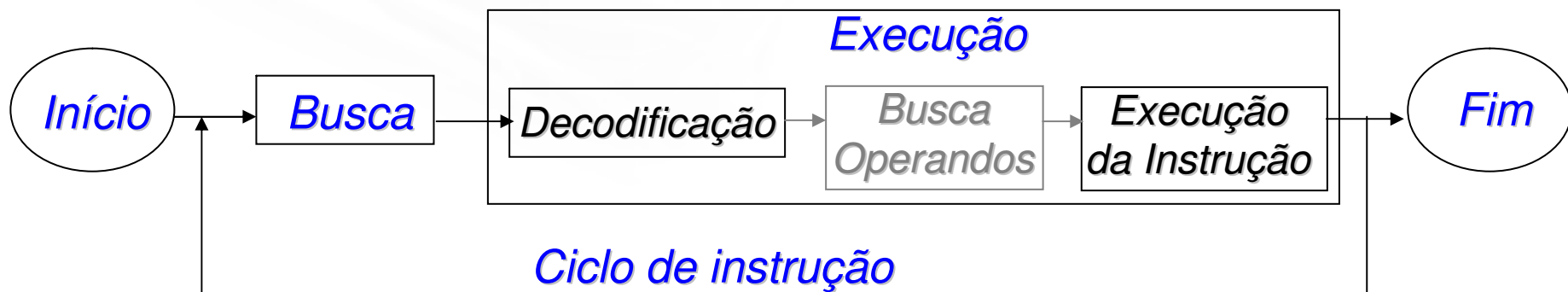
- *OpCode*: identifica a instrução a ser efetuada
- *Op*: identifica sobre o que (quem) a instrução manipulará
- Existem instruções de 1, 2 ou 3 operandos
- O tamanho do *OpCode* indica o número máximo de instruções da máquina.
 - $n \text{ bits} \Rightarrow \text{até } 2^n \text{ instruções}$
- **Ciclo de Instrução**: conjunto de operações realizadas pelo computador para completar a execução de uma instrução

Unidade Central de Processamento - CPU (13)

- **Palavras** são as unidades de dados movidas entre a memória e os registradores.
- **Instruções Registrador-Memória**
 - Permitem que o conteúdo de palavras de memória seja armazenado nos registradores (**load**) ou conteúdos de registradores armazenados na memória (**store**)
 - Referências a uma palavra deve ser feita através de **Endereços**.
- **Instruções Registrador-Registrador**
 - Típico:
 - busca dois operandos em registradores
 - coloca-os nas entradas da ULA
 - realiza alguma operação sobre eles
 - ... e armazena o resultado em um dos registradores
 - Determina um caminho de dados

Unidade Central de Processamento - CPU (14)

- Ciclo de Instrução
 - Um programa em execução é uma série de instruções de máquina que estão armazenadas na memória principal e serão interpretadas e executadas pela UCP
 - Conjunto de ações (operações) realizadas pelo computador (leia-se CPU) para completar a execução de uma instrução
- Diagrama do Ciclo de Instrução
 - Ciclo de Busca (Fetch Cycle) e Ciclo de Execução (Execution Cycle).



Unidade Central de Processamento - CPU (15)

■ **Ciclo de Busca**

1. A UC lê (busca) a instrução na memória e carrega (armazena) a instrução em RI para ser decodificada e executada.
2. Atualização do valor de PC (incremento), fazendo-o apontar para a instrução seguinte

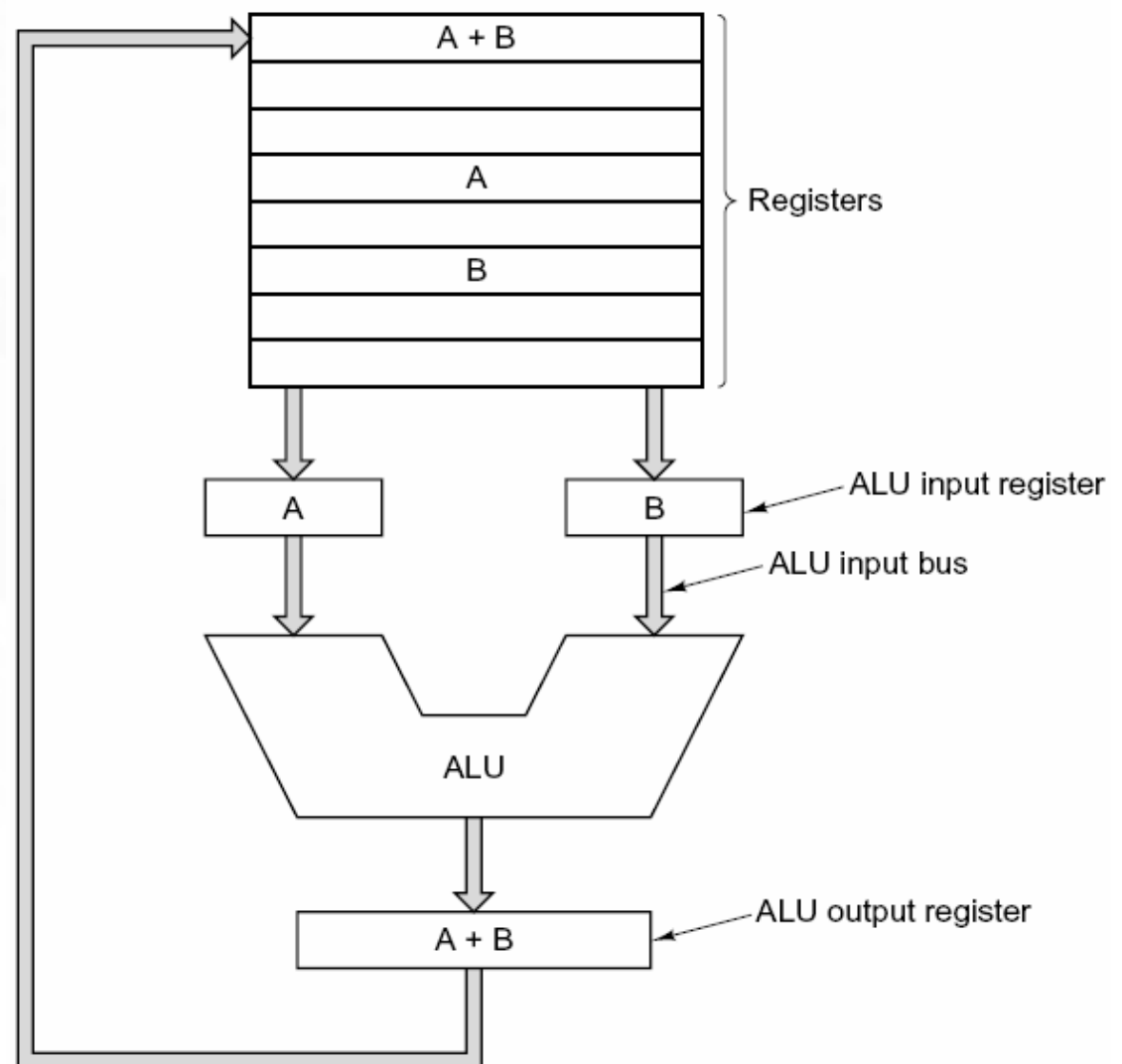
■ **Ciclo de Execução**

3. Determinação do **tipo de instrução** que está armazenada em RI (decodificação)
4. Determinação de **onde (endereço)** uma palavra de dado **ESTÁ** armazenada, caso necessário
5. **Busca da palavra**, se necessário, e armazenamento em um dos registradores do processador
6. **Execução** da instrução
7. **Retorno ao passo 1** para iniciar a execução da instrução seguinte

Unidade Central de Processamento - CPU (14)

■ Caminho de Dados

- Registradores
 - ULA
 - Controlado pela UC
-
- A velocidade do ciclo do caminho de dados (Ciclo de Instrução) determina, em última análise, a velocidade do processador.



Unidade Central de Processamento - CPU (13)

- Programa Interpretador
 - Equivalência em software do Ciclo de Instrução realizado em hardware
 - Emular um processador: “interpretando instruções através de software”
 - Exemplo em C

Questão

- De acordo com o que foi discutido no texto, na sua opinião, quais os fatores que afetam a velocidade de processamento de um computador?
 - Tamanho da memória RAM e cache
 - Largura do barramento
 - Arquitetura interna do processador
 - Tamanho e a quantidade de registradores
 - Busca de paralelismo no nível das instruções!

Paralelismo no Nível das Instruções ⁽¹⁾

- Existem limites tecnológicos para desenvolvimento / desempenho / performance do hardware do chip de processamento
- Solução para aumentar a velocidade do processador:

Uso de Paralelismo

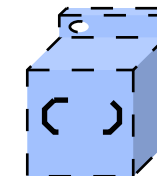
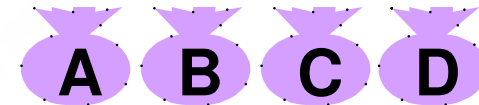
- No **Nível das Instruções**: um único processador deve executar mais instruções por segundo
- No **Nível do Processador**: vários processadores trabalhando juntos na solução do mesmo problema

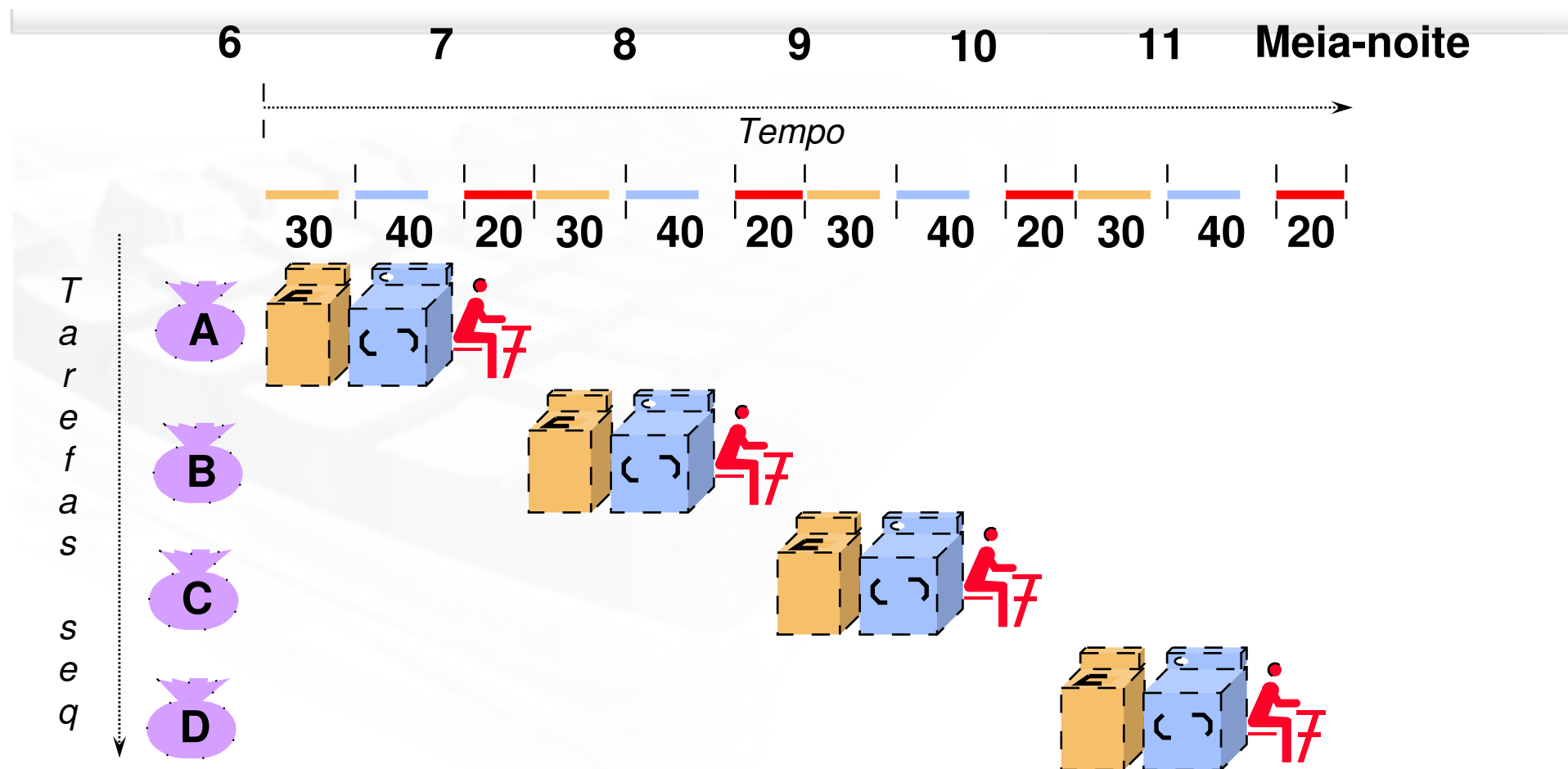
Paralelismo no Nível das Instruções (2)

- Maior gargalo para a velocidade de execução de instruções é o acesso à memória
- Execução em **Pipeline**
 - O processamento em pipeline divide a execução de instruções em várias “partes”
 - Cada “parte” é tratada por um hardware dedicado exclusivamente a ela.

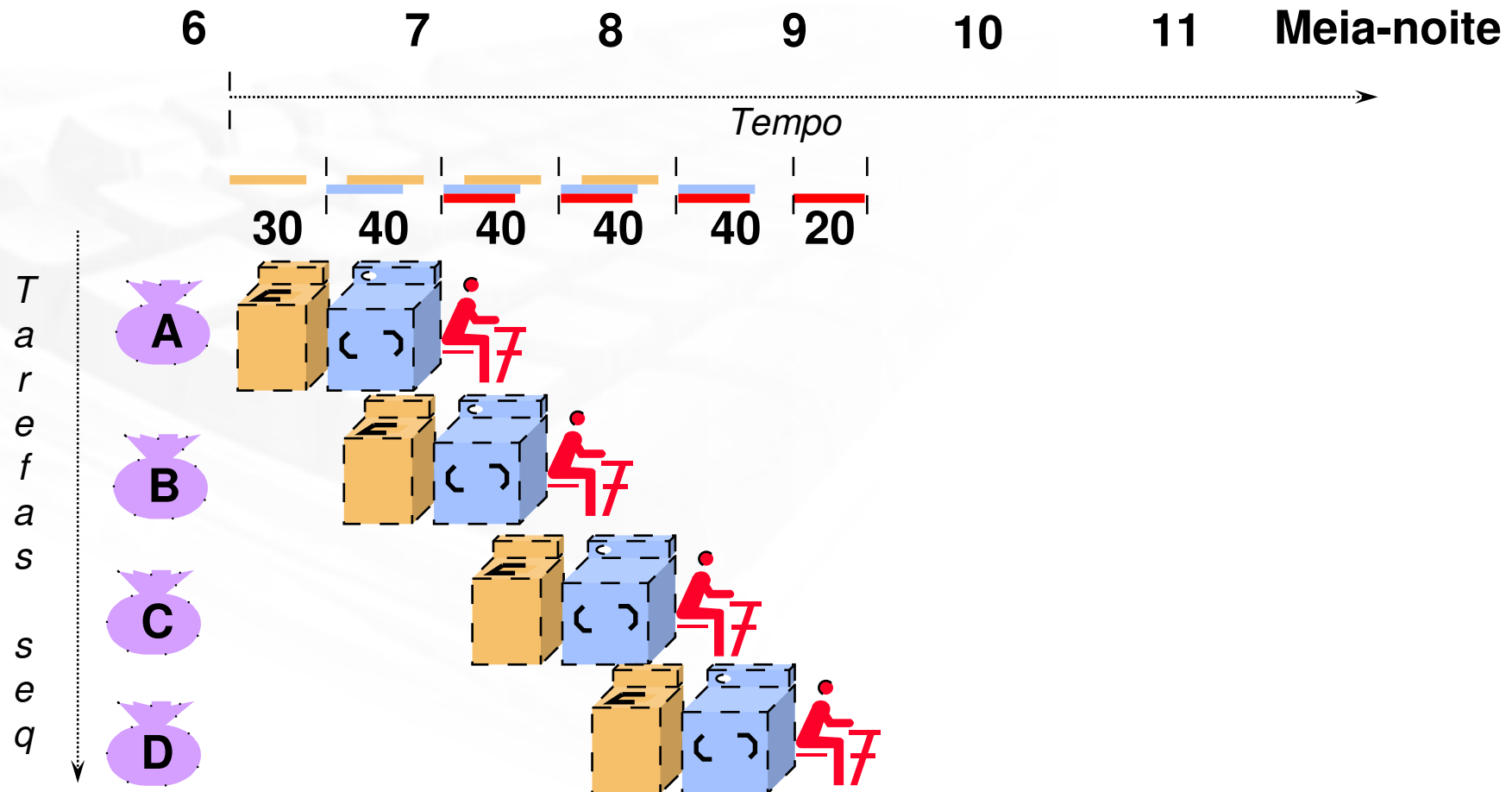
Pipelining em uma Lavanderia !?!

- 4 pessoas A, B, C e D possuem
 - 4 sacolas de roupa para lavar, secar e dobrar
- Lavar leva 30 minutos
- Secar leva 40 minutos
- Passar e dobrar leva 20 minutos





- Lavanderia sequencial leva 6 horas para terminar
- Se eles conhecessem computação, quanto tempo levaria?



- Lavanderia com *pipelining* leva 3hs e 30min !!!

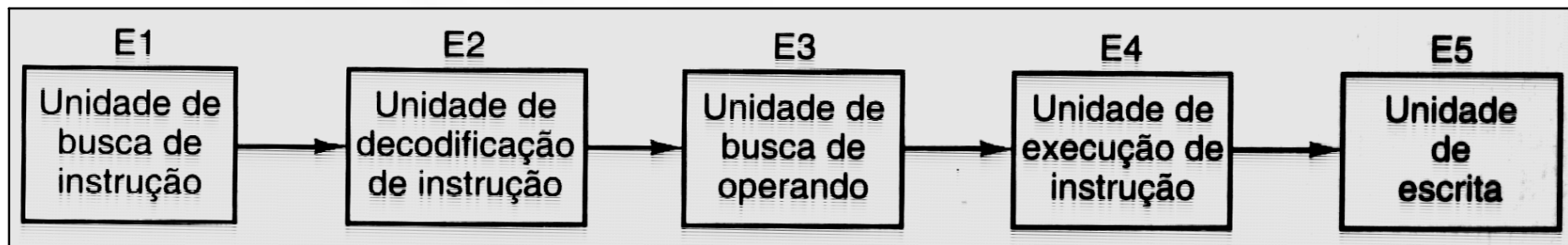
Considerações sobre Pipeline (Texto comp.)

- *Pipelining* **não** melhora a **latência** de uma única instrução, mas melhora o **throughput** do trabalho todo (nº de instruções por unidade de tempo)
 - $\text{Tempo de cada instrução} = \frac{\text{Tempo de execução sem pipeline}}{\text{num. de estágios}}$
- Taxa de inserção de instruções é limitada pelo estágio mais lento
- Existem múltiplas instruções sendo executadas em um dado instante
 - SpeedUp potencial => **número de estágios**
- Tempo para encher o pipeline e tempo de dreno **reduzem** o speedup

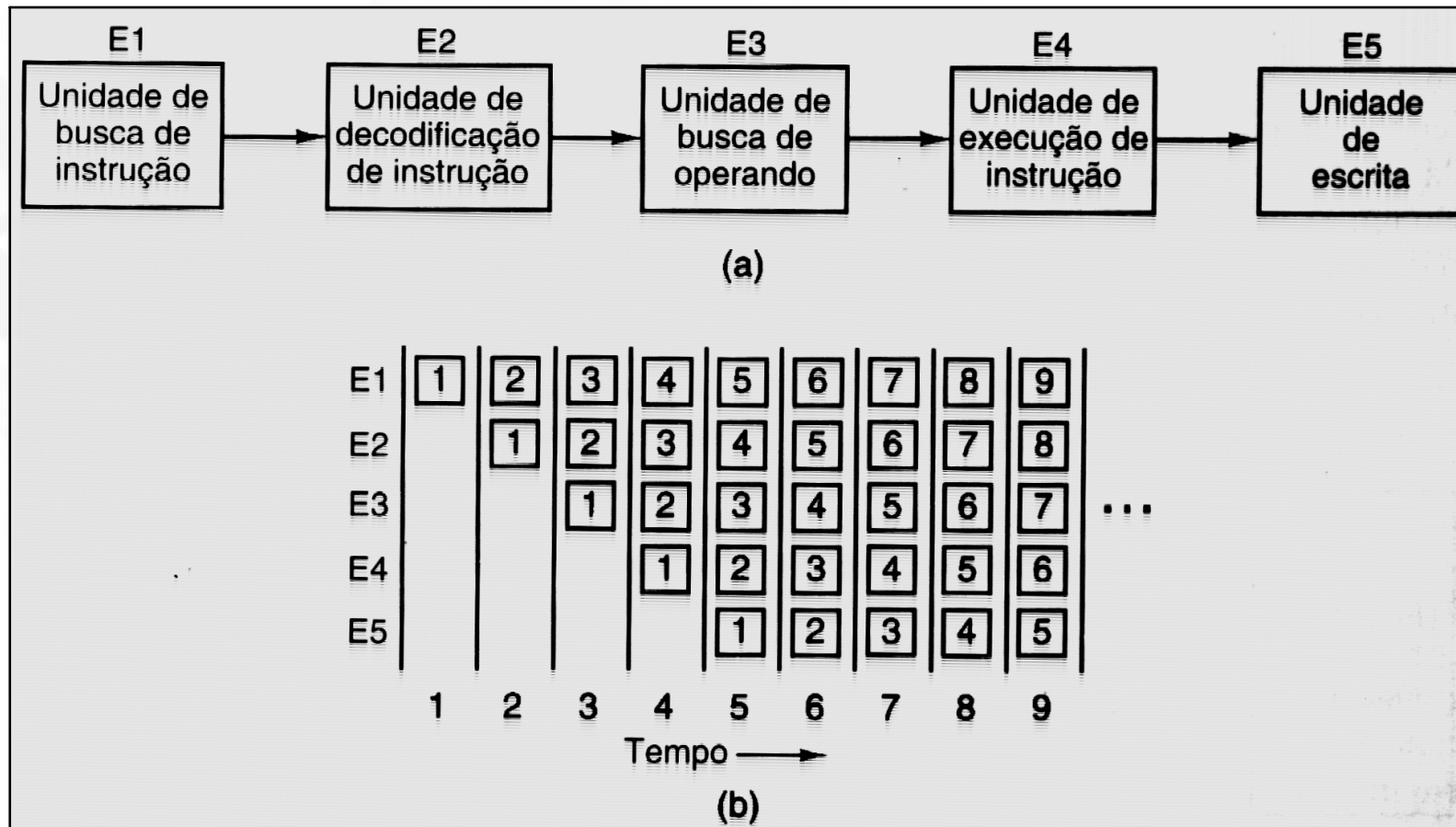
Paralelismo no Nível das Instruções ⁽³⁾

■ Pipeline de 5 estágios

1. Busca a instrução da memória e armazena em uma buffer até chegar a hora de executá-la
2. Decodificação da instrução, determinando tipo e operandos
3. Busca dos operandos na memória ou nos registradores
4. Execução - passagem pelo caminho de dados
5. Resultado do processamento é escrito em uma registrador



Paralelismo no Nível das Instruções (4)



(a) Pipeline de 5 estágios. (b) Estado de cada um dos estágios em função do tempo (estão ilustrados 9 períodos do clock).

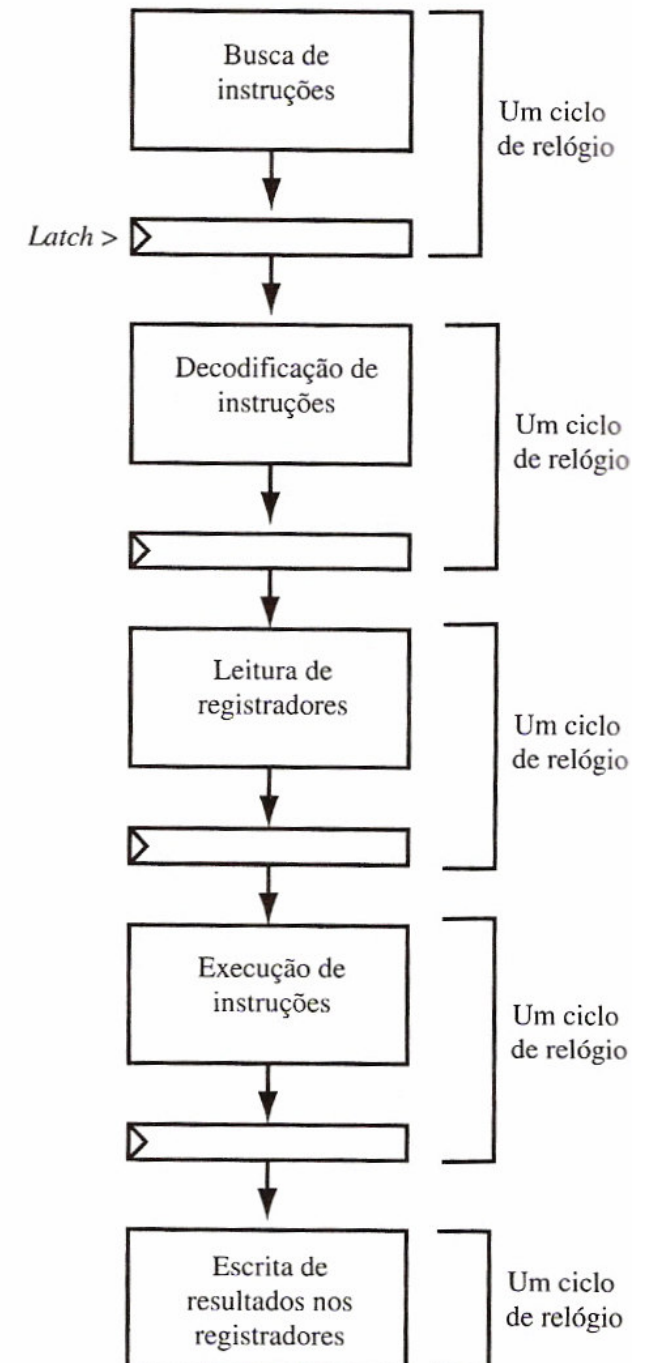
Paralelismo no Nível das Instruções (5)

- A idéia básica do pipeline é a mesma de uma linha de produção em série.
- Vários processamentos estão sendo executados ao mesmo tempo.
- Os estágios de cada processamento são aplicados a várias instruções ao mesmo tempo.
 - Exemplo:
 - **No tempo 1** a instrução 1 está sendo lida
 - **No tempo 2** a instrução 1 está sendo decodificada enquanto que a instrução 2 está sendo lida
 - **No tempo 3** a instrução 1 está buscando dados, a instrução 2 está sendo decodificada e a instrução 3 está sendo lida
 - ... e assim por diante.

Processador sem *pipeline*



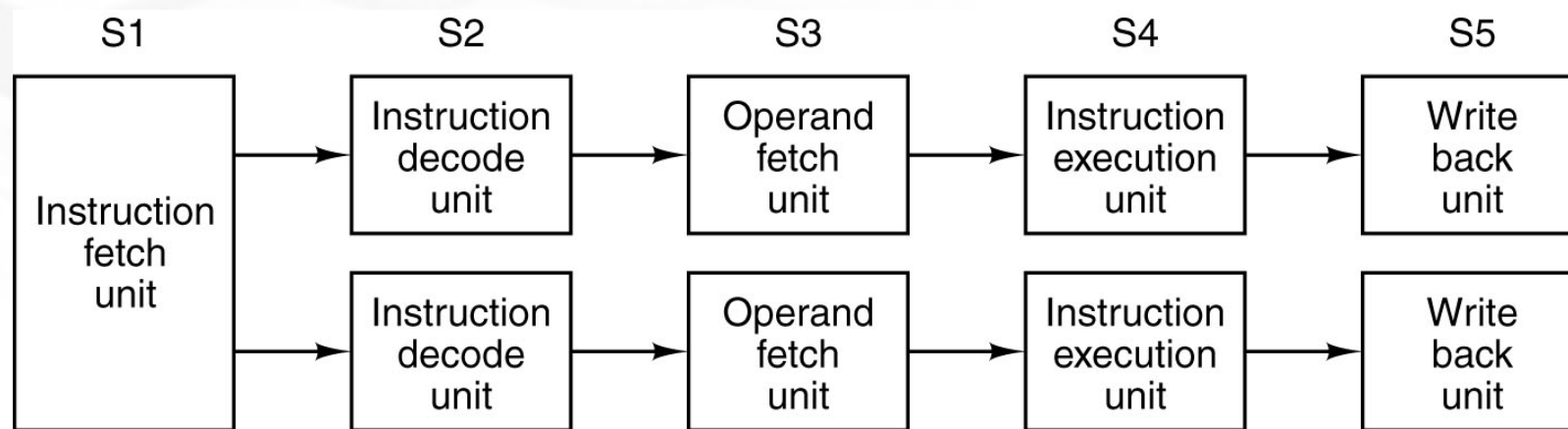
Processador com *pipeline*



Paralelismo no Nível das Instruções (6)

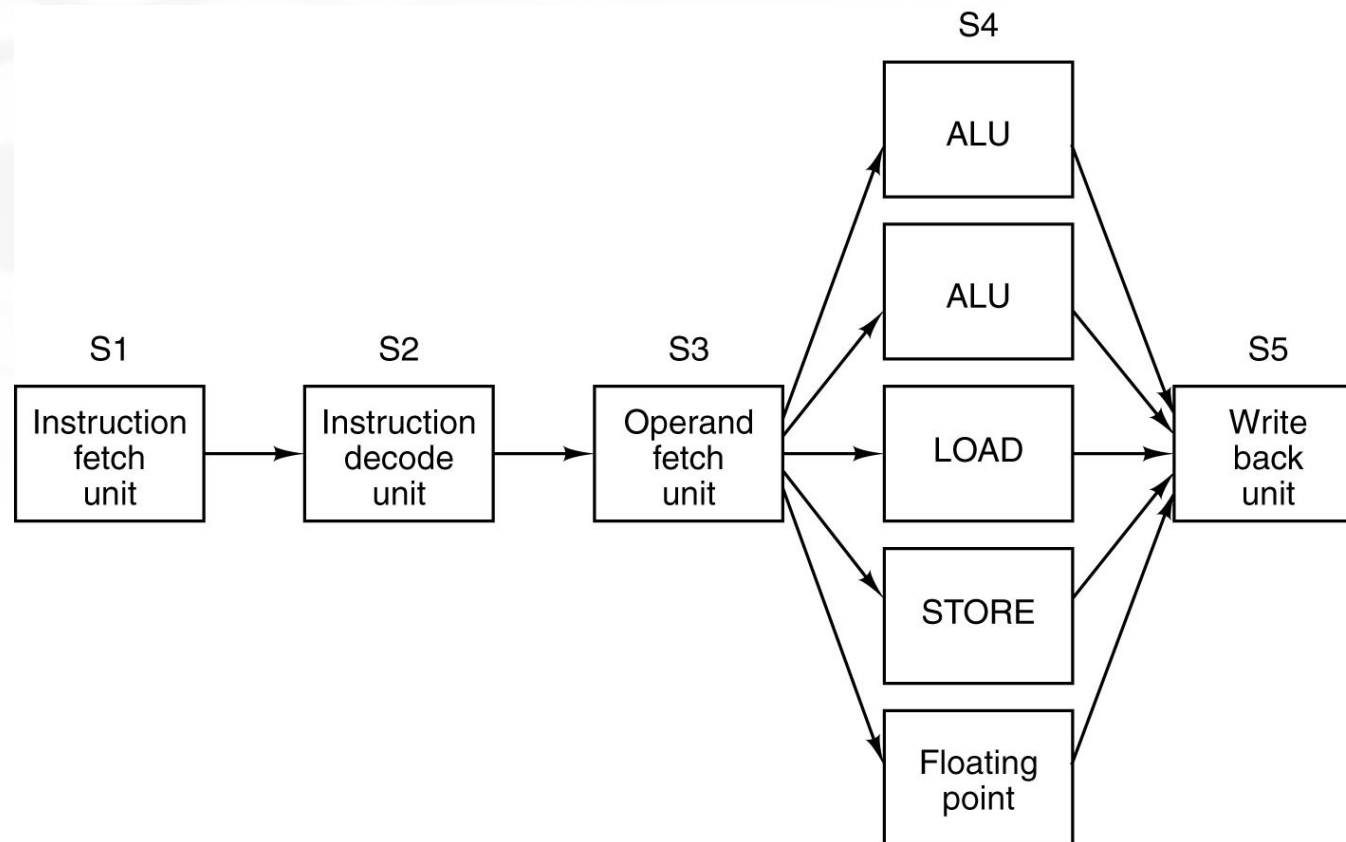
Paralelismo no Nível das Instruções (6)

- Arquiteturas superescalares



Paralelismo no Nível das Instruções (7)

- Arquiteturas superescalares (cont.)

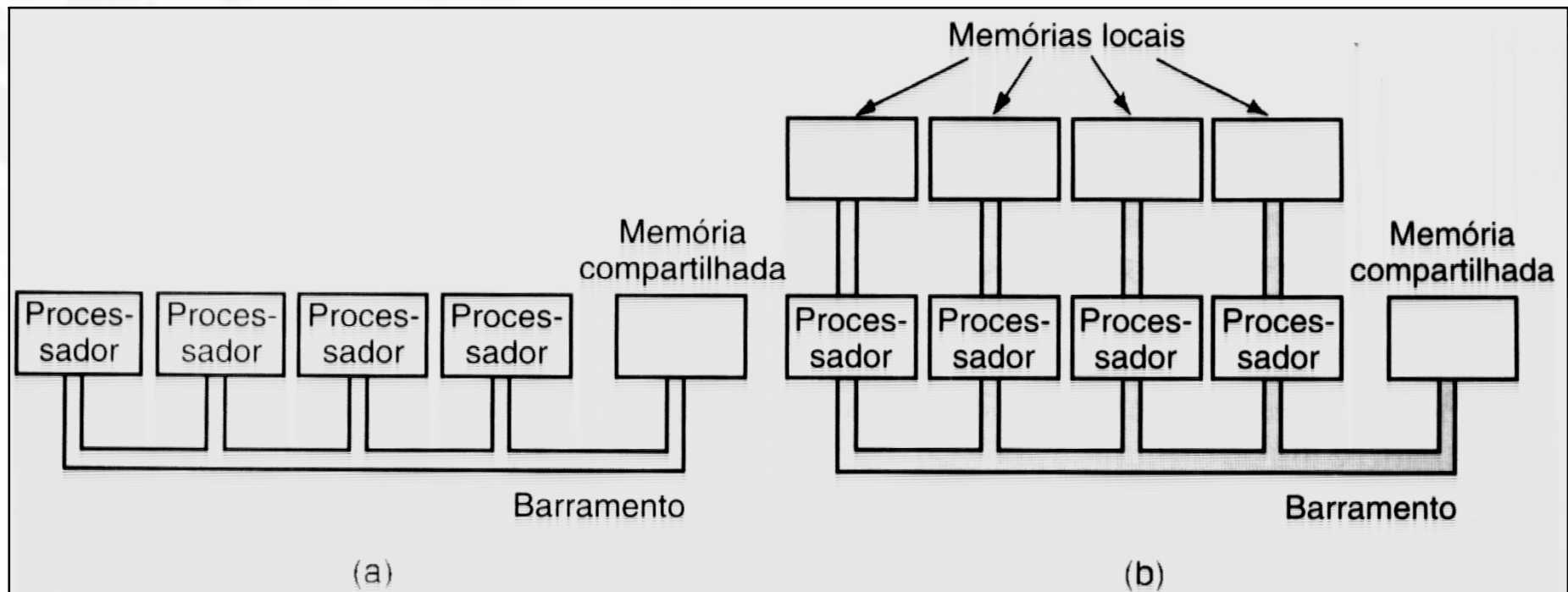


Paralelismo no Nível do Processador ⁽¹⁾

■ Multiprocessadores

- é composto de vários processadores independentes
- compartilham uma mesma memória por um barramento principal
 - OU compartilham uma memória e têm memórias locais
- executam processamentos locais
- liberam tráfego do barramento principal
- é necessário gerenciar conflitos
 - Ex: acesso ao mesmo end. de memória, dispositivo de E/S

Paralelismo no Nível do Processador (2)



- (a) Multiprocessador organizado em torno de um único barramento.
- (b) Multiprocessador com memórias locais.

Paralelismo no Nível do Processador ⁽³⁾

- Multicomputadores

- Sistemas (distribuídos) com um grande número de computadores interconectados
- Não existe memória comum sendo compartilhada
- Comunicação entre computadores é feita através de troca de mensagens a uma velocidade bem alta
 - Protocolo de comunicação
- Computador não precisa estar ligado diretamente com todos os outros (uso de topologias em árvore, anéis, etc..)

- Existem também Sistemas Híbridos

Desempenho de Computadores ⁽¹⁾

- Comparando o Desempenho de Dois Computadores
 - Sejam X e Y dois computadores. X é mais rápido que Y se:
Tempo de execução em X < 1
Tempo de execução em Y
 - O desempenho de um computador X pode ser definido como:
1
Tempo de execução em X
 - A unidade de medida de desempenho computacional mais importante é o **tempo**.

Desempenho de Computadores (2)

■ Medindo o Desempenho

- Em muitas situações, o tempo de execução de um programa não é fácil de se medir
- Sistema Multiprogramado
 - Sistema em que vários usuários ou programas compartilham a mesma CPU
- Tempo de execução de um programa pode ser descrito por:

User CPU time + System CPU time + I/O (Input/Output)

- Tempo de CPU de um programa** {
- *User CPU time*: tempo de CPU gasto pelo usuário
 - *System CPU time*: tempo de CPU gasto pelo sistema
 - *I/O (Input/Output) time*: tempo gasto com entrada/saída

Desempenho de Computadores (3)

Quais programas irão de fato testar o desempenho de um computador?
Como escolher esses programas?

■ Programas para Medir Desempenho

- **Programas Reais:** aqueles usados no dia-a-dia (ex: compiladores)
- **Kernels:** programas contendo apenas um trecho de código representativo (núcleo) de uma parte freqüentemente executada por programas típicos (ex: multiplicação de matrizes)
- **Toy Benchmarks:** programas reais bem simples
- **Synthetic Benchmarks:** são programas artificiais. São criados apenas para gerar medidas de desempenho.

Os programas reais são os mais adequados para se medir desempenho.

Desempenho de Computadores (4)

■ **Benchmark Suites**

- *Benchmark Suite* é um conjunto de programas de avaliação.
- *Standard Performance Evaluation Corporation* (SPEC) tem lançado vários *benchmark suites*: SPEC89, SPEC92, SPEC95 e SPEC2000.
- Esses *benchmark suites* são compostos por programas reais, escolhidos para serem representativos de programas que tipicamente demandam muita CPU e pouco I/O.

Exemplos de
programas inteiros
do SPEC2000:

gzip	Programa para compactar arquivos
gcc	Compilador C
vpr	FPGA Circuit Placement and Routing
craft	Game Playing: Chess
parser	Word Processing
vortex	Sistema de Banco de Dados
perlbnk	PERL Programming Language

Desempenho de Computadores (5)

- Comparando e Sumarizando Desempenho
 - Dúvidas ao comparar/sumarizar desempenho

	Computador A	Computador B	Computador C
Programa P1 (seg.)	1	10	20
Programa P2 (seg.)	1000	100	20
Tempo Total	1001	110	40

- Diferentes definições de “mais rápido”:
 - A é 10 vezes mais rápido que B para o programa P1.
 - B é 10 vezes mais rápido que A para o programa P2.
 - Se tomarmos o tempo total para executar P1 e P2 como medida de desempenho, C é o computador mais rápido; contudo, ele é 20 vezes mais lento que A na execução do programa P1.
- Melhor forma de sumarizar desempenho: **tempo total de execução.**

Desempenho de Computadores (6)

- Comparando o desempenho de um *benchmark suite*:

- **Média Aritmética dos tempos de execução**

$$\text{Média aritmética de } Tempo(i,n) = \frac{1}{n} \sum_{i=1}^n Tempo_i$$

- **Média Harmônica**

$$\text{Média harmônica de } Tempo(i,n) = \frac{n}{\sum_{i=1}^n \frac{1}{Tempo_i}}$$

	Computador A	Computador B	Computador C
Programa P1 (seg.)	1	10	20
Programa P2 (seg.)	1000	100	20
Tempo Total	1001	110	40
Média Aritmética	500,5	55,0	20,0
Média Harmônica	1,998	18,181	20,000

Desempenho de Computadores (7)

- **Princípios Quantitativos para o Projeto de Computadores**

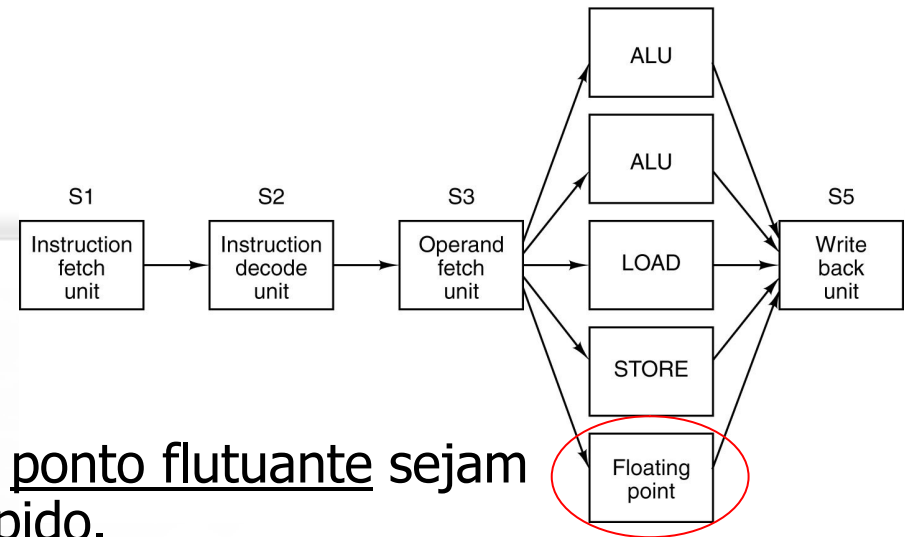
- O principal princípio a ser seguido é: **Torne mais rápidas as tarefas mais freqüentes**

- **Lei de Amdahl**

- *A quantidade de melhoria de desempenho provida por um atributo da arquitetura é limitada pela quantidade de tempo que este atributo é utilizado.*
- Relaciona o **speedup** total de um sistema com o **speedup** de uma porção (*atributo da arquitetura*) do sistema

- $Speedup = \frac{\text{Tempo de execução da tarefa inteira sem usar o novo atributo}}{\text{Tempo de execução da tarefa inteira usando o novo atributo quando possível}}$

Lei de Amdahl - Exemplo



- Suponha que todas as instruções em ponto flutuante sejam melhoradas rodando 5 vezes mais rápido.
- O tempo de execução de alguns benchmark antes da melhoria no ponto flutuante é de 10 segundos.
- Qual será o aumento da velocidade (speedUp) se, antes da melhoria, metade dos 10 segundos eram gastos executando instruções em ponto flutuante?

$$T = T_f + T_o \quad \left\{ \begin{array}{l} T_f: \text{Tempo instruções em ponto flutuante} \\ T_o: \text{outras instruções} \end{array} \right.$$

$$\text{speedUp} = \frac{T_1 \text{ (T sem melhoria)}}{T_2 \text{ (T com melhoria)}} = \frac{T_{f1} + T_{o1}}{T_{f2} + T_{o2}} = \frac{5 + 5}{5/5 + 5} = \frac{10}{5/5 + 5} = 1,67$$

Desempenho de Computadores (9)

■ **Tempo de Execução de um Programa**

- Tempo de CPU + I/O
- PS: por simplicidade chamaremos Tempo de execução o Tempo de CPU (desconsideraremos I/O):

Tempo de CPU de um Programa =

Número de Ciclos de Clock despendidos no Programa $\times$ Tempo de um Ciclo

Tempo de CPU de um Programa =

Número de Ciclos de Clock despendidos no Programa / Frequência de Clock

Desempenho de Computadores ⁽¹⁰⁾

- Exercício:

Um programa roda em 10 segundos em um computador A, que tem um clock de 100MHz.

Um computador B gasta 1.2 vezes mais ciclos de clock para rodar o mesmo programa mas roda este programa em 6 segundos.

Qual é a frequência de clock do computador B?

$$\text{Comp. A: } 10\text{s} = n_{\text{ciclos_A}} / 100\text{MHz}$$

$$n_{\text{ciclos_A}} = 10\text{s} \times 100\text{MHz}$$

$$\text{Comp. B: } 6\text{s} = 1.2 \times n_{\text{ciclos_A}} / \text{freq_B}$$

$$\text{freq_B} = 1.2 \times 10\text{s} \times 100\text{MHz} / 6\text{s}$$

$$\text{freq_B} = 200\text{MHz}$$

Desempenho de Computadores (11)

- ***Clocks per Instruction (CPI)***: é a média do número de ciclos por instrução
- É comum calcularmos o desempenho da CPU em função do CPI
 - O tempo de execução de um programa, T , em uma determinada CPU:

$$T \text{ (segundos)} = N \text{ (instrução)} \times C \text{ (ciclo/instrução)} \times S \text{ (segundos/ciclo)}$$

- N é o número total de instruções executadas
 - C (ou clocks por ciclo de máquina – ***Clocks per Instruction (CPI)***): é a média do número de ciclos por instrução
 - S é o número de segundos por ciclo
-
- Quanto menor o tempo, maior o desempenho.
 - N , C e S são afetados primariamente:
 - pela capacidade de otimização do compilador
 - pela arquitetura do processador e de seu conjunto de instruções
 - pela tecnologia empregada na implementação da máquina.

Desempenho de Computadores (12)

- Exercício: Um projetista de compiladores está tentando decidir sobre qual seqüência de instruções ele deve utilizar para implementar um comando de uma linguagem de alto nível. Ele dispõe das seguintes informações:
 - Existem duas seqüências de instruções possíveis para implementar o comando
 - Seqüência 1: 2 instruções A, uma B e duas C;
 - Seqüência 2: quatro A, uma B e uma C.
 - Na máquina alvo, a instrução A tem o CPI de 1, a instrução B tem CPI de 2 e a C, de 3.
 - **1** - Que seqüência de código executa mais instruções? **2** - Qual seqüência executa mais rápido? **3** - Qual é o CPI médio para cada seqüência?
- Respostas:
 - 1) $\text{seq1} = 2+1+2 = 5$ instruções; **seq2** = $4+1+1 = 6$ instruções
 - 2) $\text{seq1} = 2 \times 1 + 1 \times 2 + 2 \times 3 = 10$ ciclos; **seq2** = $4 \times 1 + 1 \times 2 + 1 \times 3 = 9$ ciclos
 - 3) $\text{CPI}(\text{seq1}) = 10 \text{ ciclos} / 5 \text{ instruções} = 2 \text{ CPI}$; $\text{CPI}(\text{seq2}) = 9 / 6 = 1.5 \text{ CPI}$

Unidades de Medida de Performance ⁽¹⁾

- MIPS – *Million Instructions per Second*

- A unidade MIPS pode ser definida como:

$$\text{MIPS} = \text{Frequência de Clock} / \text{CPI} \times 10^6$$

$$= \text{nº de instruções} / \text{tempo de execução} \times 10^6$$

- O Tempo de Execução de um programa pode ser definido em função da unidade MIPS:

$$\text{Tempo de execução} = \text{No. de Instruções} / \text{MIPS} \times 10^6$$

- MIPS é uma unidade de performance ruim ... Porquê ???

1. Não há como comparar computadores com conjuntos de instruções diferentes. Pois o número de instruções necessário para executar um determinado programa nestes dois computadores vai quase certamente diferir, e se a frequência de clock for a mesma, o computador que demora mais para executar o programa aparentemente terá o mesmo desempenho do que demora menos.
2. A quantidade de MIPS varia entre programas em um mesmo computador, assim, uma máquina não pode ter uma performance única em termos de MIPS.

Unidades de Medida de Performance (2)

- MIPS – *Million Instructions per Second*
 - Exercício: Considere um computador com clock de 100MHz. Sabendo que para um programa específico o compilador 1 gera
 - 5 milhões de instruções do tipo A
 - 1 milhão do tipo B
 - 1 milhão do tipo C,e que o compilador 2 gera
 - 10 milhões do tipo A,
 - 1 milhão do tipo B
 - 1 milhão do tipo Ce que o CPI de cada instrução é igual ao do exercício anterior (a instrução A tem o CPI de 1, a instrução B tem CPI de 2 e a C, de 3),
mostre que a medida de performance em termos de MIPS pode variar inversamente com a performance verdadeira. Suponha que cada instrução gerada é executada apenas uma vez.

Performance = 1 / tempo de execução

MIPS = Freqüência de Clock / **CPI** x10⁶
no de instruções / tempo de execução x10⁶

Unidades de Medida de Performance ⁽³⁾

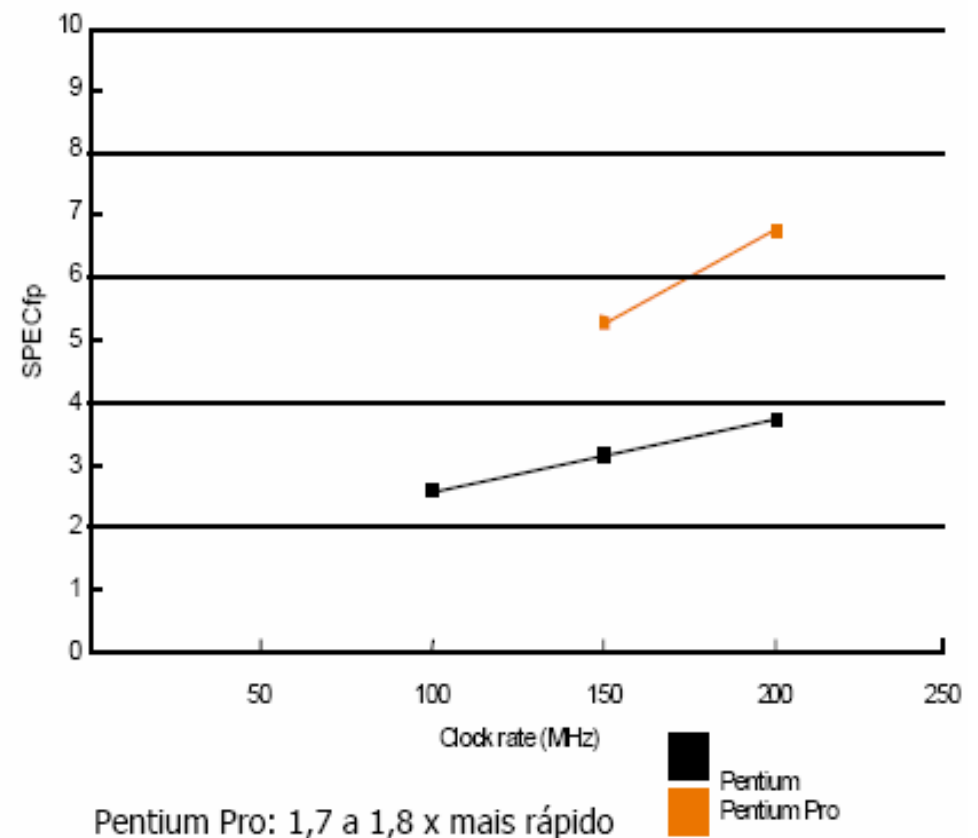
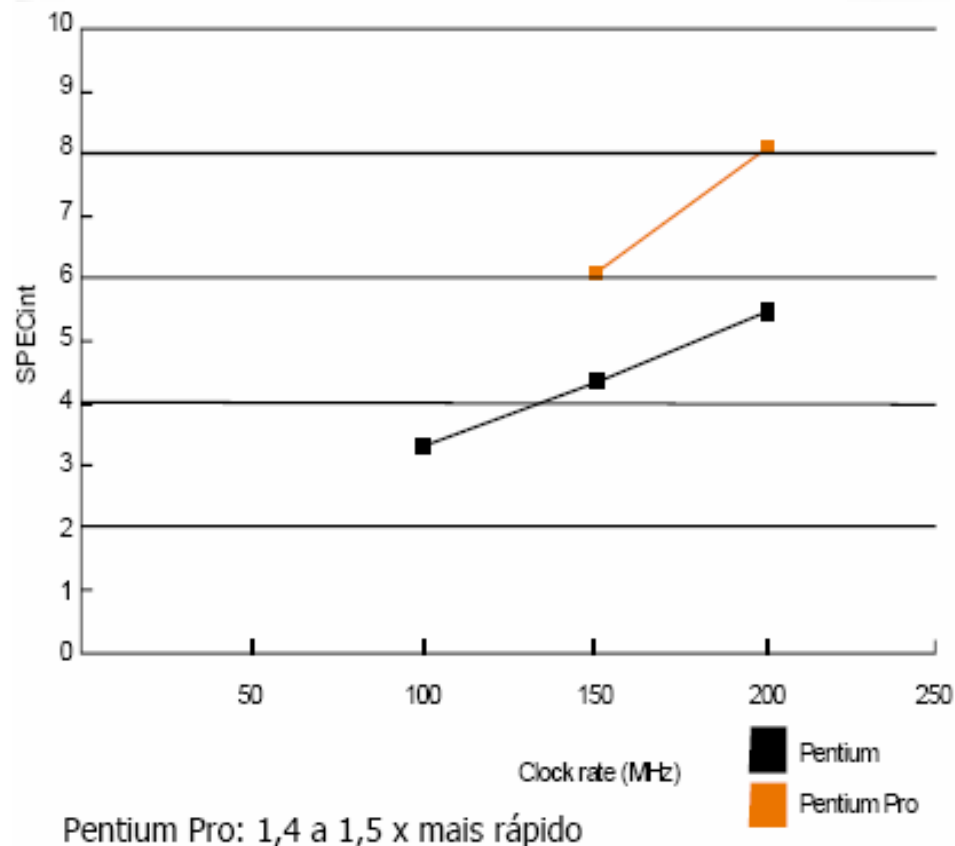
■ *SPEC*

- Medir o tempo de execução usando um conjunto de programas de uso generalizado, com entradas conhecidas, e sob circunstâncias conhecidas e especificadas é a melhor maneira de se medir o desempenho de um computador.
- As unidades de medidas (*score*) **SPECint** e **SPECfp** são consideradas, atualmente, boas unidades de medida de desempenho.

Unidades de Medida de Performance (4)

- SPEC'95 (8 programas inteiros, 10 de ponto flutuante)

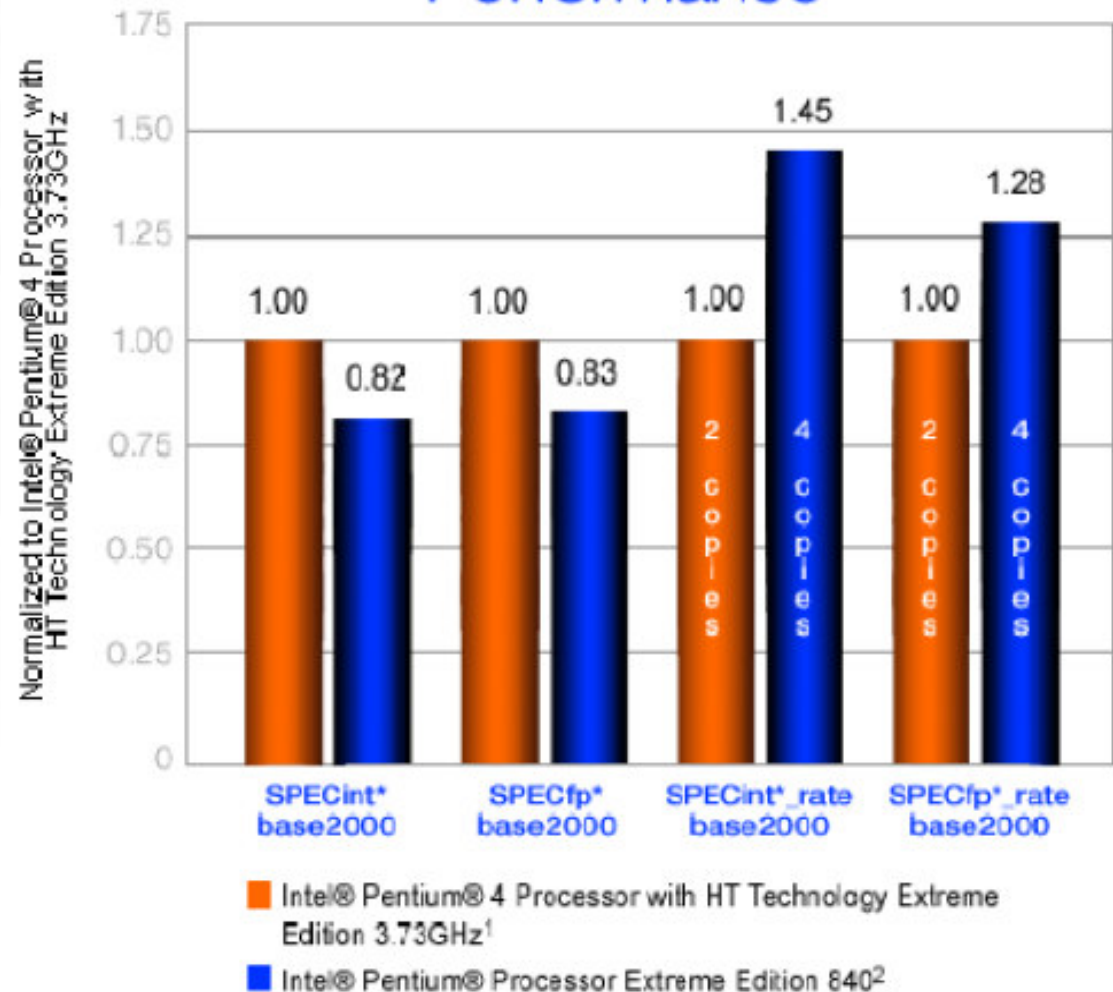
Pentium e Pentium Pro



Unidades de Medida de Performance ⁽⁵⁾

- Spec2000
 - Pentium 4 3.73 GHz
 - Pentium dual-core 840 3.20 GHz

Integer & Scientific Computing Performance



Referências

- Andrew S. Tanenbaum, ***Organização Estruturada de Computadores***, 5ª edição, Prentice-Hall do Brasil, 2007.
- John L. Hennessy and David A. Patterson, **Arquitetura de Computadores: Uma Abordagem Quantitativa**. 3ª edição. Editora Campus, 2003.
- Lúcia Helena M. Pacheco, **Visão Geral de Organização Estruturada de Computadores e Linguagem de Montagem**. Universidade Federal de Santa Catarina. Centro Tecnológico, Departamento de Informática e de Estatística.
- <http://www.wikipedia.org>

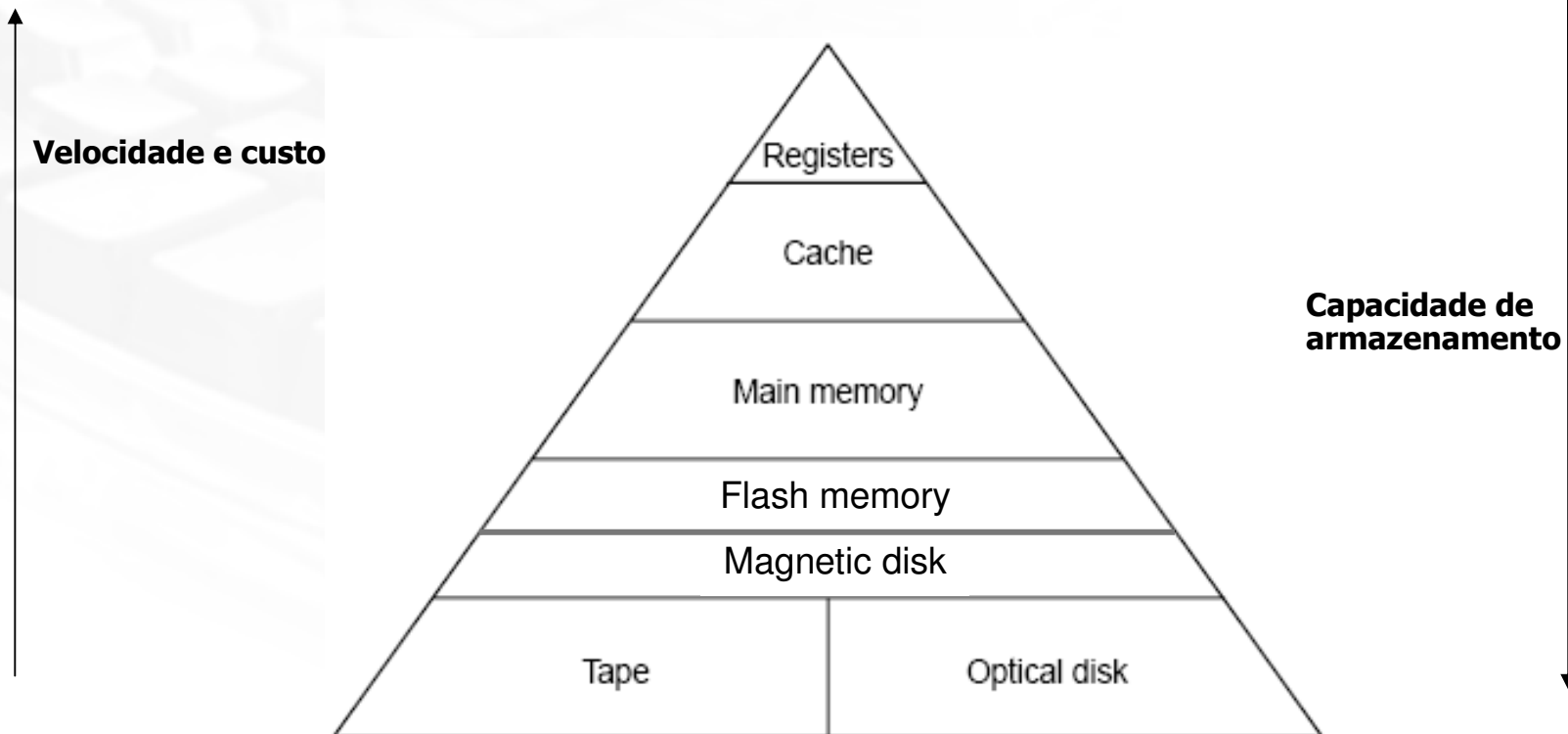


Organização de Computadores

(Aula 4)

Memória

Hierarquia de Memória (1)



Hierarquia de Memória (2)

- A memória *cache* é geralmente controlada por hardware
- A memória principal (RAM) e a secundária é que o usuário tem acesso.
- O sistema operacional através de um mecanismo de **Memória Virtual (Segmentação e/ou Paginação)** cria a “ilusão” ao usuário que a memória total é do tamanho da memória principal + memória secundária.
- A técnica de memória virtual realiza transferência de blocos de informação entre a memória primária e secundária automaticamente sem a intervenção do usuário comum.

Registradores

- São dispositivos (elementos computacionais) capazes de receber dados, mantê-los armazenados por um curto período de tempo e transferi-los para outro dispositivo.
 - São, portanto, elementos de armazenamento temporário.
- Os registradores fazem parte da CPU.
- São extremamente rápidos e armazenam grupos reduzidos de bits.

Memória Principal ⁽¹⁾

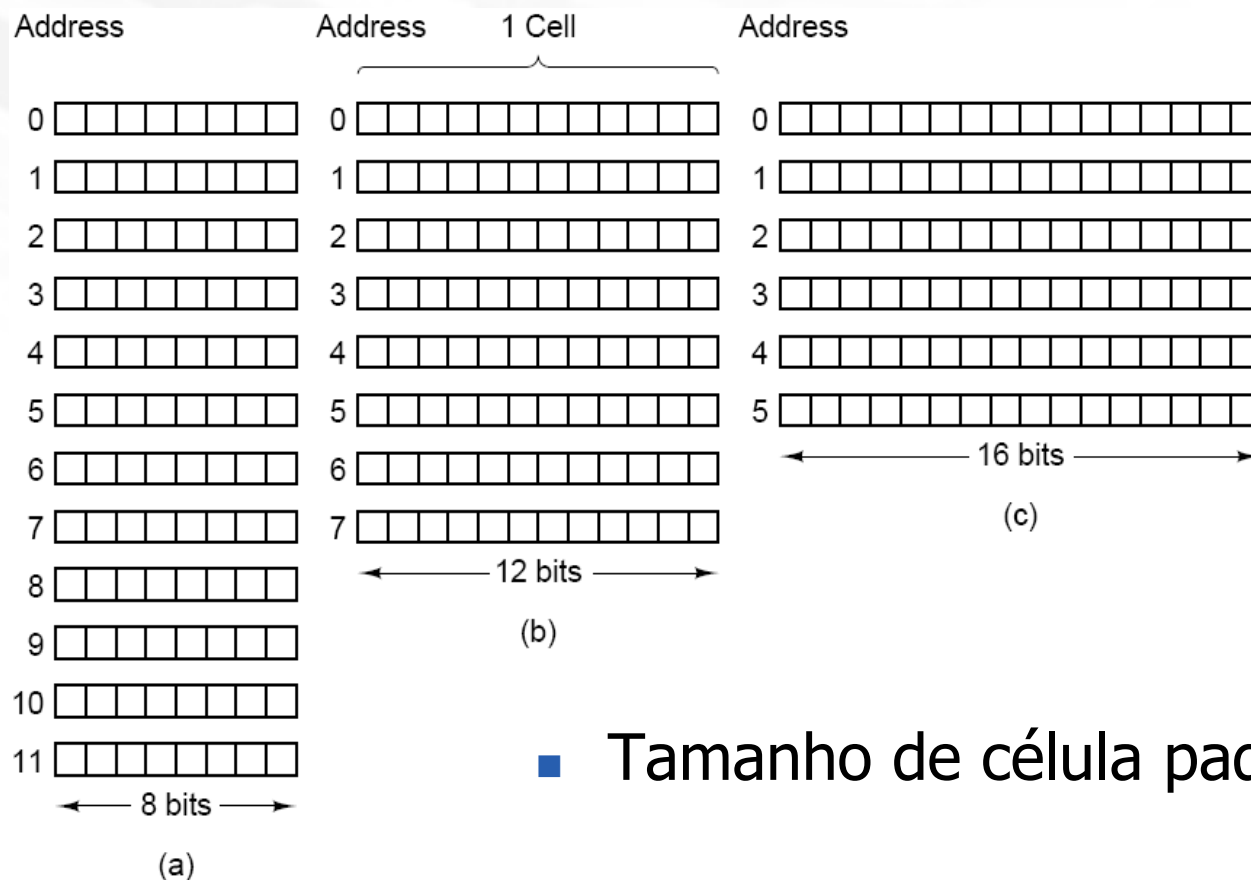
- A memória é a parte do computador em que os programas e os dados são armazenados.
- A memória principal (MP) armazena programas em execução e os dados utilizados por eles.
- Sem uma memória na qual processadores possam ler ou escrever informações, o conceito de computador digital com programa armazenado não pode ser implementado.
 - A CPU processa instruções que são obtidas na MP e os resultados são retornados à MP.

Memória Principal (2)

- A unidade básica de memória é o **bit** (*binary digit*)
 - Abstração de valores 0 ou 1
 - Fisicamente é mais fácil distinguir entre dois valores distintos do que de mais valores. Tensão, corrente, ...
- A memória é formada por um conjunto de **células** (ou **posições**), cada uma das quais podendo guardar uma informação.
 - Todas as células de uma dada memória têm o mesmo número de bits
- Os números que identificam (referenciam) a posição da célula na memória são chamados de **Endereços**.
 - **A célula é a menor unidade endereçável da memória**
- Endereços são indexadores pelos quais os programas podem referenciar dados na memória.

Memória Principal (3)

- A memória é formada por um conjunto de **células**
 - Todas as células de uma memória têm o mesmo nº de bits
- Cada célula tem um **endereço**

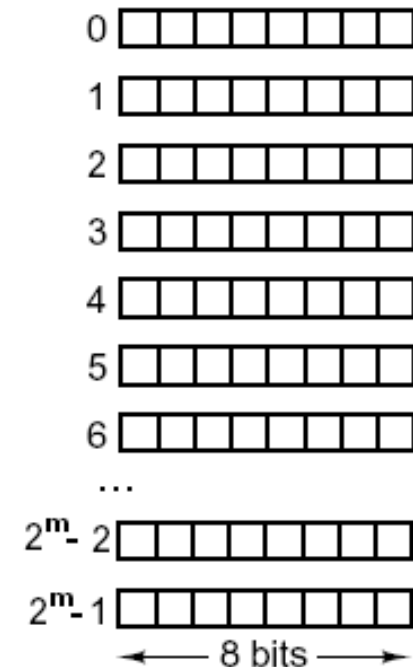


Como
organizar uma
memória de
96 bits?

- Tamanho de célula padrão: 8 bits ... 1 **byte**

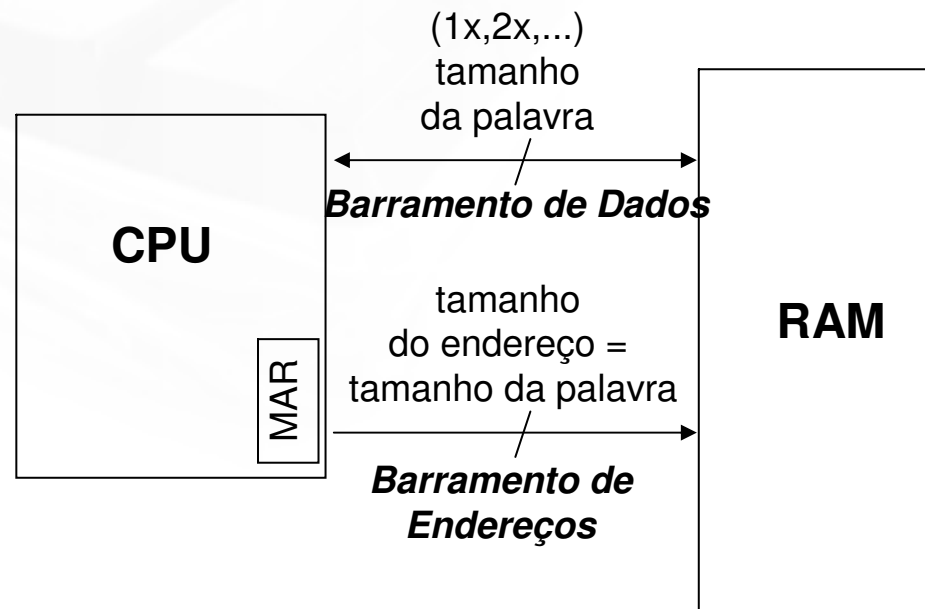
Memória Principal (4)

- Se a memória tiver n células, elas terão endereços de 0 a $(n - 1)$.
- Se uma célula tiver k bits, ela poderá armazenar qualquer uma das 2^k combinações possíveis para os bits.
- Se um determinado endereço tem m bits, o número máximo de células endereçáveis é de 2^m .



Memória Principal (5)

- Os computadores modernos agrupam as células (ou bytes) em Palavras (*word*)
 - Ex: uma palavra de 32 bits tem 4 bytes (ou 4 células)
- Nesses computadores, a Palavra é a parte mínima de dados que podem ser transferidos de/para a memória (MP)
- A informação na palavra pode ser um dado ou uma instrução



Memória Principal (4)

- **“Processadores de 32 bits” têm palavras de 32 bits**
 - **Os registradores são de 32 bits**
 - nº de bits do barramento de endereços em geral (mas não obrigatoriamente) é igual ao nº de bits dos registradores (ex: *Memory Address Register* - MAR)
 - **As instruções são (em geral) de 32 bits**
 - **Cada instrução deve tratar palavras de 32 bits**
 - **movimentar, somar subtrair... dados armazenados em registradores de 32 bits**

Memória Principal (5)

- Os bytes de uma palavra podem ser ordenados na memória da esquerda para a direita OU da direita para a esquerda
- **Big Endian:** Os bytes mais significativos primeiro (Mac)
- **Little Endian:** Os bytes menos significativos primeiro (Intel)

00000000 00000000 00000100 00000001

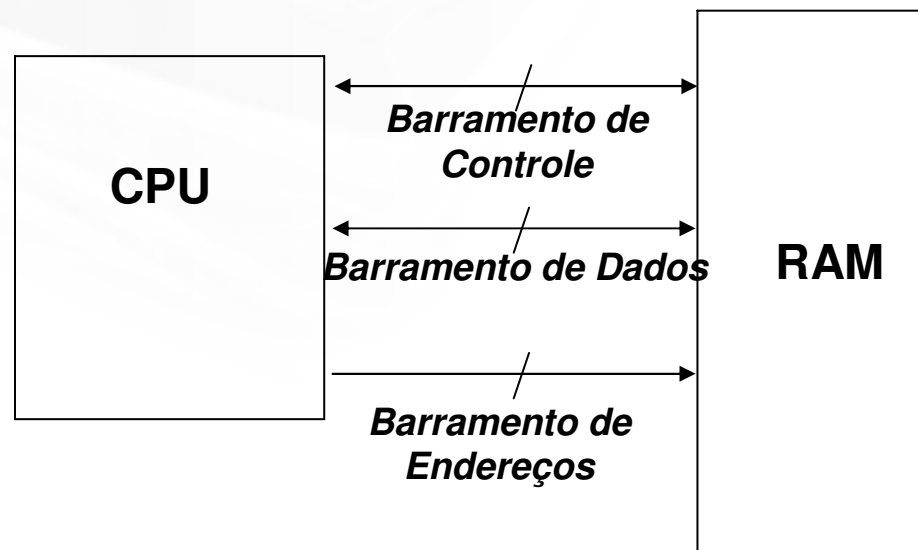
Address	Big-Endian representation of 1025	Little-Endian representation of 1025
00	00000000	00000001
01	00000000	00000100
02	00000100	00000000
03	00000001	00000000

Memória Principal ⁽⁶⁾

- O endereço identifica a palavra na memória, e é usado tanto para armazenar (*store*) como para carregar (*load*).
- **Em geral, registradores são usados para endereçar a memória**
tamanho do barramento de endereços = tamanho da palavra
- No projeto de cada computador é definida a organização da memória (tamanho do barramento de endereços, tamanho da palavra e da célula de memória, etc.)
 - Exemplos de memória:
 - 64K (2^{16}) X 8 bits (65 536 posições de 8 bits = 64 kilobytes)
 - 4 Giga (2^{32}) X 8 bits (4 294 967 296 posições de 8 bits)
 - 1 Mega (2^{20}) X 16 bits (1 048 576 posições de 16 bits)
 - Qual o máximo de memória endereçável de um computador de 64bits?
 - célula = 1 byte

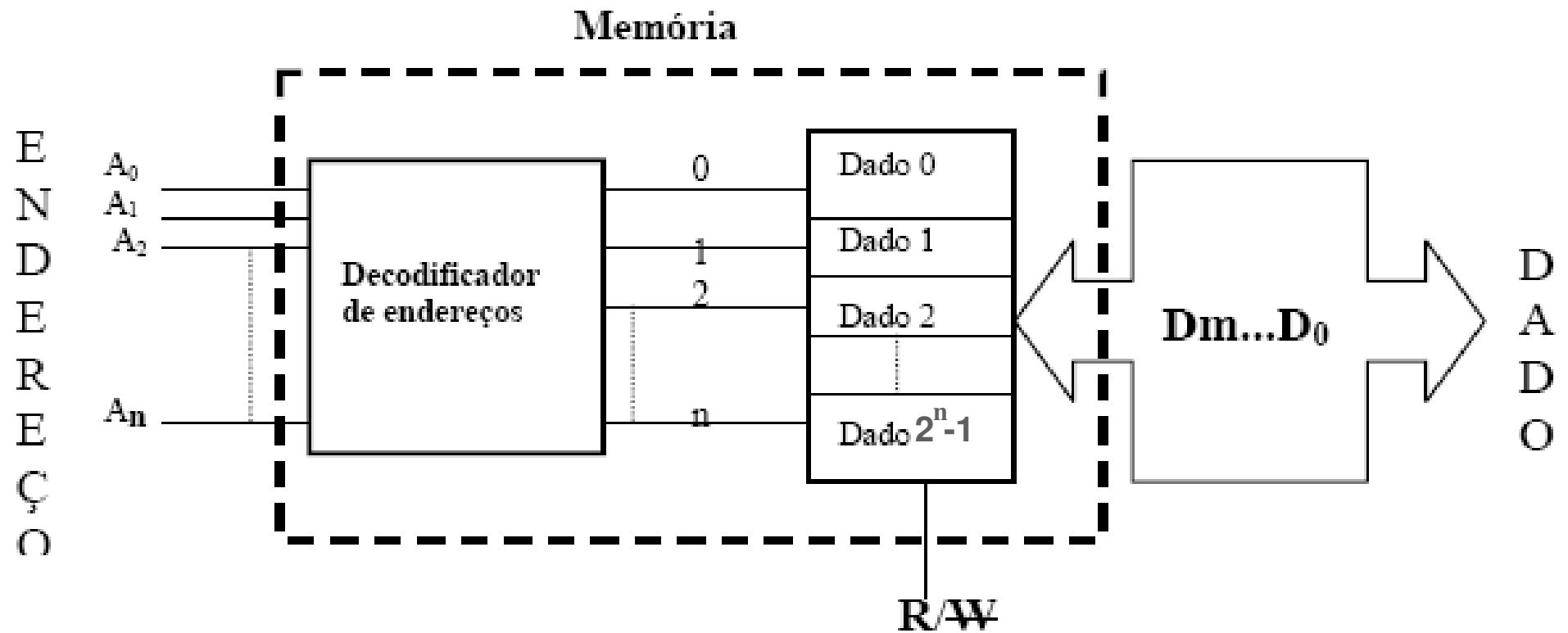
Memória Principal (7)

- **Leitura:** Quando um valor é recuperado da memória, o conteúdo da palavra não é alterado. Apenas uma cópia será enviada pela memória.
- **Escrita:** A gravação de um novo conteúdo em uma palavra se dá com a destruição do conteúdo anterior.



Memória Principal ⁽⁸⁾

- Diagrama esquemático da organização da memória
 - Para simplificar: **1 palavra = 1 célula**



Memória Principal ⁽⁹⁾

- Uma memória com n linhas de endereços e m linhas de dados.
- As linhas de endereços (n bits) permitem endereçar 2^n diferentes células de memória.
- O tamanho do dado, contido em cada célula de memória é de m bits.
- Cada código de endereço de n bits seleciona, através do decodificador de endereços, apenas uma única célula na memória.
- O sentido do fluxo de dados dependerá do **signal de controle R/W**.
 - Se $R/W = 1$, a operação é de READ, ou seja, leitura na memória e o dado sairá da memória
 - Se $R/W = 0$, a operação será de escrita (WRITE) e o dado entrará na memória.

Tipos de Memória (1)

- **Memória Volátil:** É aquela cuja informação original é perdida se a energia for desligada.
 - **Memória Não-volátil:** É aquela que retém o padrão de bits original mesmo que a energia seja desligada.
-
- **Memória Endereçada Seqüencialmente:** Para se obter a informação de um endereço, é necessário percorrer os endereços anteriores.
 - **Ex.: Fita Magnética**
 - **Memória de Acesso Randômico (RAM – *Random Access Memory*):**
 - A célula pode ser acessada sem ter que percorrer os endereços anteriores.
 - O tempo de acesso é praticamente o mesmo para todas as células
 - Volátil

Tipos de Memória (2)

■ RAMs Estáticas (SRAM)

- Baseadas em *flip-flops*
- Conteúdo persiste enquanto circuito alimentado
- Mais rápidas (geralmente usadas como memória cache)

■ RAMs Dinâmicas (DRAM)

- Baseadas em capacitores
- Carga deve ser restaurada periodicamente, pois suas informações vão desaparecer após um certo intervalo de tempo
- Menores, consomem menos potência, mais baratas
- SDRAM (Synchronous Dynamic Random Access Memory)
 - **Módulo SDR (*Single Data Rate*)**: transfere um dado por pulso de clock
 - **Módulo DDR (*Double Data Rate*)** : tráfego é de dois dados por pulso de clock (transfere dados tanto na subida quanto na descida do sinal de clock)
 - **DDR-II**: melhorias no padrão

Tipos de Memória (3)

■ **ROM (*Read-Only Memory*)**

- Simples: decodificador, linhas de saída e portas lógicas
- Aplicações de alto volume

■ **PROM (*Programmable ROM*)**

- Conteúdo escrito com um “queimador” de PROMs. Podem ser escritas com dispositivos especiais mas não podem mais ser apagadas.

■ **EPROM (*Erasable PROM*)**

- Podem ser apagadas e reutilizadas pelo uso de radiação ultravioleta.
- Antigos chips de BIOS (Basic Input/Output System) de PC

■ **EEPROM (*Electrically Erasable PROM*)**

- Conteúdo pode ser modificado eletricamente
- Pode ser lida um número ilimitado de vezes, mas só pode ser apagada e programada um número limitado de vezes (entre 100.000 e 1 milhão).
- Pelo menos 64 vezes menores que uma EPROM
- Armazenam a BIOS em PC atuais

Tipos de Memória (4)



■ Memória Flash

- Como a EEPROM, mas que permite que múltiplos endereços (blocos) sejam apagados ou escritos numa só operação.
- Players MP3, celulares, câmeras digitais, ... "HD" p/ Laptops!?!

■ Encapsulamento de Memória Principal

■ SIMM – *Single Inline Memory Module*

- Uma linha (em apenas um lado) de conectores da unidade de memória para a placa de circuito impresso
- Adequadas p/ barramento de dados de 32 bits



SIMM (72 pinos)

■ DIMM – *Dual Inline Memory Module*

- Uma linha de conectores da unidade de memória para a placa de circuito impresso, em ambos os lados da placa (pente) de memória
- Muito usadas c/ Pentiums (64 bits no barramento de dados)



DIMM DDR 1GB 400 MHz (184-pinos)

Códigos com Correção/Detecção de Erros ⁽¹⁾

- Os dados armazenados na memória dos computadores podem ocasionalmente serem alterados (modificados)
 - Oscilações de tensão de alimentação, etc.
- Para a prevenção desses erros, algumas memórias armazenam informações extras, usando **códigos** e **mecanismos** que permitam a detecção e/ou correção de erros.

- **Palavra de Código:**

uma unidade de n bits = m bits de dados

+

r de redundância

{ Informação referente
à detecção/ correção
de erro

Códigos com Correção/Detecção de Erros (2)

EXEMPLO

- **Bit de Paridade:** bit configurado em 0 ou 1 para assegurar que o número total de bits 1 no campo de dados é par (ou ímpar).
 - Paridade refere-se ao número de bits '1' no número binário.
 - Ex:
1 10010010
bit de paridade dado

0 00011000
bit de paridade dado

Códigos com Correção/Detecção de Erros ⁽³⁾

- **Distância de Hamming:** número de bits que diferem em duas palavras de um código qualquer.

10001001

10110001 XOR

00111000

- Distância de Hamming do código = menor distância de Hamming entre duas palavras
- Se um código possui distância de Hamming de **d**, ele é capaz de
 - detectar **d-1** erros
 - corrigir **mod [(d-1)/2]** erros

Códigos com Correção/Detecção de Erros (4)

Tabela de palavras
de código válidas
(bit de paridade)

0	000
1	001
1	010
0	011
1	100
0	101
0	110
1	111

Distância de Hamming do código = 2



Código capaz de detectar **1** erro
Capaz de corrigir **0** erros

Tabela de palavras
de código válidas
(código hipotético)

00	000
01	011
10	110
11	101

Distância de Hamming do código = 3



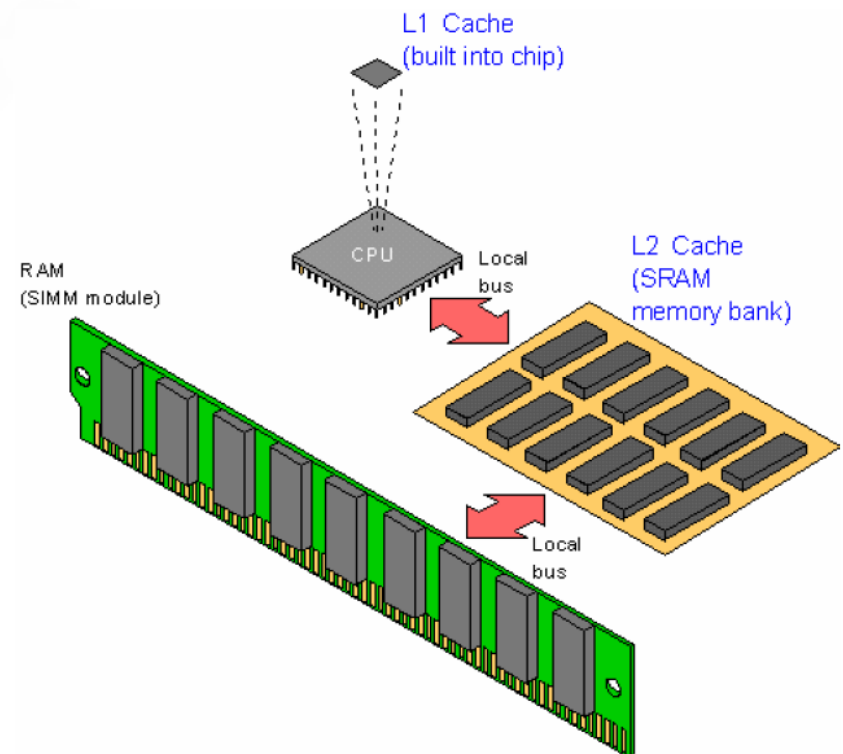
Código capaz de detectar **2** erros
Capaz de corrigir **1** erro

Memória *Cache* ⁽¹⁾

- Princípio básico
 - Na execução de um programa de computador, muitas das referências são a um pequeno conjunto de posições de memória.
 - Muitos comandos presentes em *loops*.
 - Manipulação de matrizes
- **Cache** é um dispositivo interno a um sistema que serve de intermediário entre uma CPU e o dispositivo principal de armazenamento (MP).
- A idéia principal é que o acesso a MP pode ser demorado e vale a pena armazenar as informações mais procuradas em meio mais rápido.
- **Memória *Cache*:** memória pequena (capacidade de armazenamento) e rápida
 - Contém os dados e/ou instruções mais recentemente referenciados pelo processador.

Memória *Cache* (2)

- Quando um processador precisar de uma palavra de memória, ele primeiro busca essa palavra na *cache*.
- Somente no caso de ela não estar armazenada na *cache* é que a busca se dará na memória principal.
- Se uma parte substancial dos acessos for satisfeita pela *cache*, o **tempo médio** de acesso a uma palavra em memória será pequeno, próximo ao tempo de acesso à *cache*.
- Em alguns computadores podem existir diversos níveis de *cache*
 - Ex: nível 1 é implementado dentro do chip ; nível 2 implementado na placa-mãe



Memória *Cache* ⁽³⁾

■ Princípio da Localidade

■ Localidade *Temporal*

- Uma posição de memória referenciada recentemente tem boas chances de ser referenciada novamente em um futuro próximo
- Iterações e recursividade

■ Localidade *Espacial*

- Uma posição de memória vizinha de uma posição referenciada recentemente tem boas chances de também ser referenciada
- Dados tendem a ser armazenados em posições contíguas

■ Como explorar o princípio de localidade?

■ Localidade Temporal

- Mantenha palavras mais recentemente acessados na cache

■ Localidade Espacial

- A leitura na memória, geralmente, é feita com mais de uma palavra ao mesmo tempo (blocos ou linhas de cache)
- Mova **blocos** de palavras contíguas para a cache

Memória *Cache* (4)

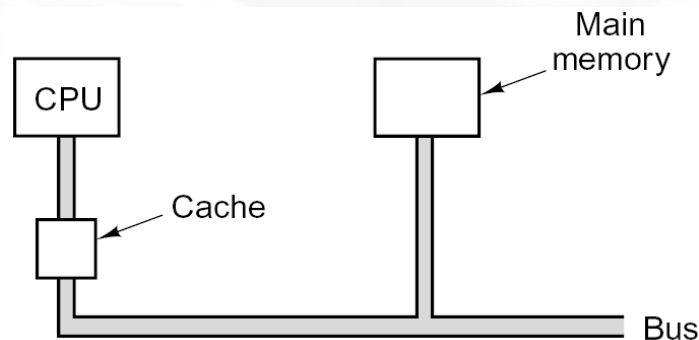
■ Princípio da Localidade (cont.)

■ Bloco ou Linhas de Cache

- Menor unidade de informação transferida entre os níveis da hierarquia de memória
- Um bloco é transferido num único acesso entre a memória principal e a cache, através de um largo barramento de dados

Exemplo

Se uma cache tiver uma linha com 64 bytes, uma referência ao endereço 260 que não está na cache (falha) vai trazer da memória principal p/ a cache a linha composta pelos bytes 256 a 319:



Posicionamento lógico da memória *cache*

Memória Principal

0	—	63
64	—	127
128	—	191
192	—	255
256	—	319

Bloco {

← 64 bytes →

Memória *Cache* ⁽⁵⁾

- Se determinada palavra for lida ou escrita k vezes em um curto intervalo de tempo, o processador precisará
 - referenciar uma **única** vez a memória principal (“lenta”)
 - e **$k - 1$** vezes a memória rápida.

c : tempo de acesso à *cache*

m : tempo de acesso à memória principal

h : **Taxa de acertos** – fração de todas as referências que puderam ser satisfeitas pela *cache*.

$$h = (k - 1)/k$$

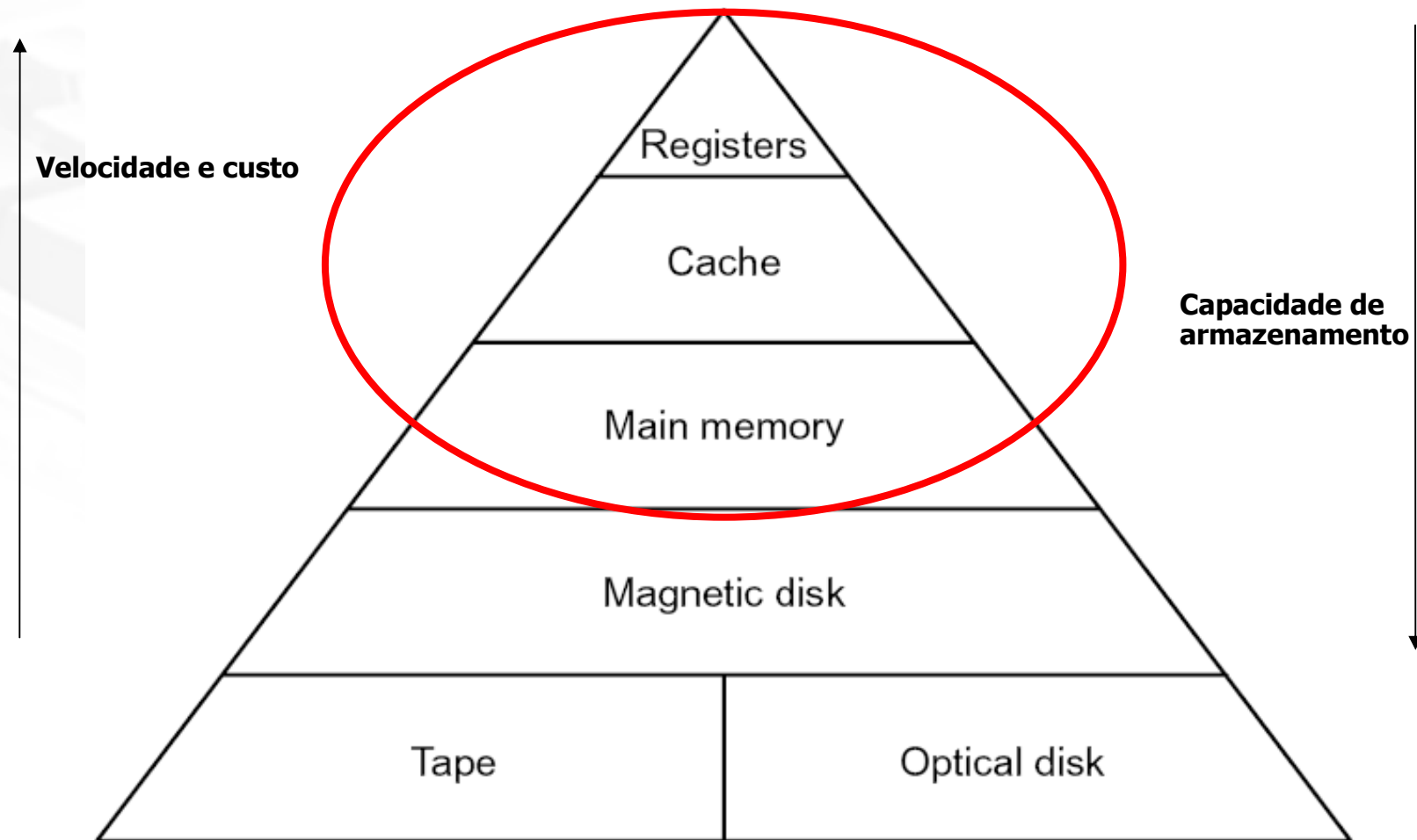
$$\text{Taxa de Falhas} = 1 - h$$

- Tempo médio de acesso = $c + (1 - h)m$

Memória *Cache* ⁽⁶⁾

- **Acerto (*hit*)**
 - Posição acessada está na *cache*
- **Erro (*miss*)**
 - Posição acessada ausente da *cache*
 - Buscada da memória principal
- ***Cache Unificada*** (instruções e dados juntos) x ***Cache Dividida*** (uma *cache* para cada)
- Questões a discutir:
 - Qual o melhor tamanho da *cache*?
 - Qual o tamanho da **linha da *cache***?
 - Em geral, uma linha maior aproveita melhor a localidade espacial
 - MAS linha maior significa maior tempo para preencher a linha

Hierarquia de Memória



Memória Secundária ⁽¹⁾

- Memórias que não podem ser endereçadas diretamente, isto é, a informação precisa ser carregada em memória primária antes de poder ser tratada pelo processador.
- Necessário pois o conteúdo da MP é apagado quando o computador é desligado. Desta forma, tem-se um meio de executar novamente programas e carregar arquivos contendo os dados da próxima vez em que o computador for ligado.
- São geralmente **não-voláteis**, permitindo guardar os dados permanentemente.
 - Discos Rígidos (HDs)
 - CDs
 - DVDs
 - Disquetes, etc.

Memória Secundária (2)

- **Disco Rígido (HD - *Hard Disk*)**

- É a parte do computador onde são armazenadas informações que "não se apagam" (**Aquivos**)
- Caracterizado como memória física, não-volátil.
- O disco rígido é um sistema lacrado contendo **discos de metal recobertos por material magnético** onde os dados são gravados através de **cabeças**, e revestido externamente por uma proteção metálica que é presa ao gabinete do computador por parafusos.
- Também é chamado **Winchester**. (Rifles Winchester 30-30)
- **Nos Sistemas Operacionais mais recentes, o disco rígido é também utilizado para expandir a memória principal, através de mecanismos de Memória Virtual.**



Memória Secundária ⁽⁶⁾

■ Setor de *Boot* – Disco Rígido

- Nele é registrado qual **sistema operacional** está instalado no computador, com qual **sistema de arquivos** o disco foi formatado e quais arquivos devem ser lidos para inicializar o computador.
- Também é conhecido como **Trilha MBR** ou **Trilha 0**.
- **MBR (Master Boot Record)** - Registro de Inicialização Mestre
 - Contém a tabela de partição do disco que dará boot.
 - O MBR é lido pela **BIOS (*Basic Input Output System*)**, que interpreta a tabela de partição e em seguida carrega um programa chamado ***bootstrap***, que é o responsável pelo carregamento do Sistema Operacional.

Memória Secundária (7)

■ Controlador de Disco

- **Controlador**(a) é um dispositivo de hardware que realiza a interface entre o exterior de um dispositivo e o seu funcionamento interno.
- Processador envia o endereço físico de dados para um HD e o controlador traduz esse endereço e aciona os dispositivos mecânicos específicos do disco para que os dados possam ser lidos e enviados.

■ Padrões

- **IDE** - *Integrated Drive Electronics*
- **(E)IDE** - *(Extended) Integrated Drive Electronics*
- **ATAPI** - *Advanced Technology Attachment Packet Interface*
- **SCSI** (***S**mall **C**omputer **S**ystem **I**nterface*).
 - Permite conexão de uma larga gama de periféricos (HDs, CD-ROMs, impressoras, scanners, etc.)

Referências

- Andrew S. Tanenbaum, Organização Estruturada de Computadores, 5ª edição, Prentice-Hall do Brasil, 2007. Capítulo 2
- Lúcia Helena M. Pacheco, **Visão Geral de Organização Estruturada de Computadores e Linguagem de Montagem**. Universidade Federal de Santa Catarina. Centro Tecnológico, Departamento de Informática e de Estatística.
- Artigo sobre correção e detecção de erros
 - <http://www.dcc.ufla.br/infocomp/artigos/v1.1/cce.pdf>
- <http://www.wikipedia.org>

ANEXO 1

Tabela ASCII

Caracteres normais

Binário	Decimal	Hex	Gráfico	Binário	Decimal	Hex	Gráfico	Binário	Decimal	Hex	Gráfico
0010 0000	32	20	(vazio) (□)	0100 0000	64	40	@	0110 0000	96	60	·
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	\$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q
0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r

■ ■ ■



Organização de Computadores

(Aula 5)

Entrada/Saída

Entrada e Saída (*Input/Output – I/O*)

- Computador:
 - Processador,
 - Memória (principal e **secundária**)
 - Equipamentos (dispositivos, elementos, unidades, ...) de **Entrada/Saída (E/S ou I/O)**
- Dispositivos de E/S: Periféricos do Computador
- Função geral:
 - Servem para a comunicação do computador com o meio externo.
- **Entrada:** entrada (inserção) de dados por meio de algum código (ex: programa).
 - Ex: teclado, mouses, scanners, leitoras óticas, leitoras de cartões magnéticos, câmeras de vídeo, microfones, sensores, transdutores, etc
- **Saída:** retorno de dados, como resultado de alguma operação de algum programa.
 - Impressoras, monitores de vídeo, plotters, atuadores, chaves, etc ...

Diferentes Tipos de Dispositivos ⁽¹⁾

- Diferentes funções
 - Comunicação do usuário com o computador
 - Comunicação do computador com o meio ambiente (dispositivos externos a serem monitorados ou controlados)
 - Armazenamento (gravação) de dados.
- Taxa de transferência de dados varia muito para cada dispositivo de I/O
 - Muito lento: ex. teclado, mouse
 - Muito rápido: ex. gráficos escritos no monitor, discos, interfaces de redes

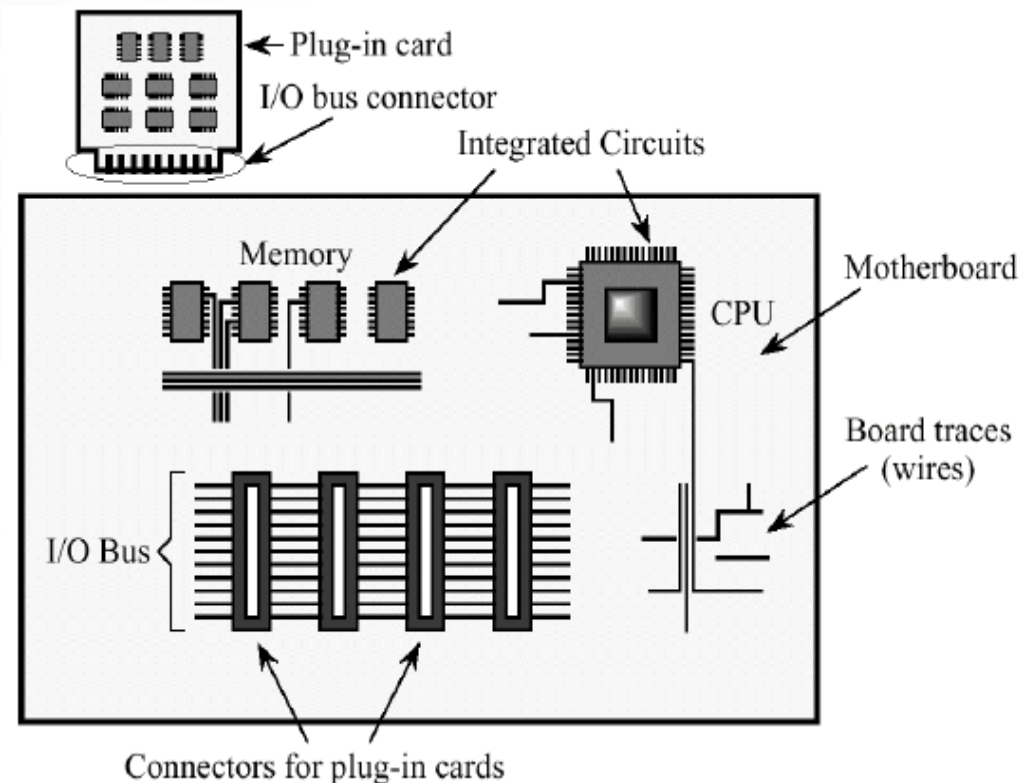
Diferentes Tipos de Dispositivos (2)

- Taxa de transferência de dados varia muito para cada dispositivo de I/O
 - Muito lento: ex. teclado, mouse
 - Muito rápido: ex. gráficos escritos no monitor, discos, interfaces de redes

Device	Behavior	Partner	Data rate (KB/sec)
Keyboard	Input	Human	0.01
Mouse	Input	Human	0.02
Voice input	Input	Human	0.02
Scanner	Input	Human	200.00
Voice output	Output	Human	0.60
Line printer	Output	Human	1.00
Laser printer	Output	Human	100.00
Graphics display	Output	Human	30,000.00
Network-terminal	Input or output	Machine	0.05
Network-LAN	Input or output	Machine	200.00
Floppy disk	Storage	Machine	50.00
Optical disk	Storage	Machine	500.00
Magnetic tape	Storage	Machine	2000.00
Magnetic disk	Storage	Machine	2000.00

Entrada e Saída (*Input/Output – I/O*) (2)

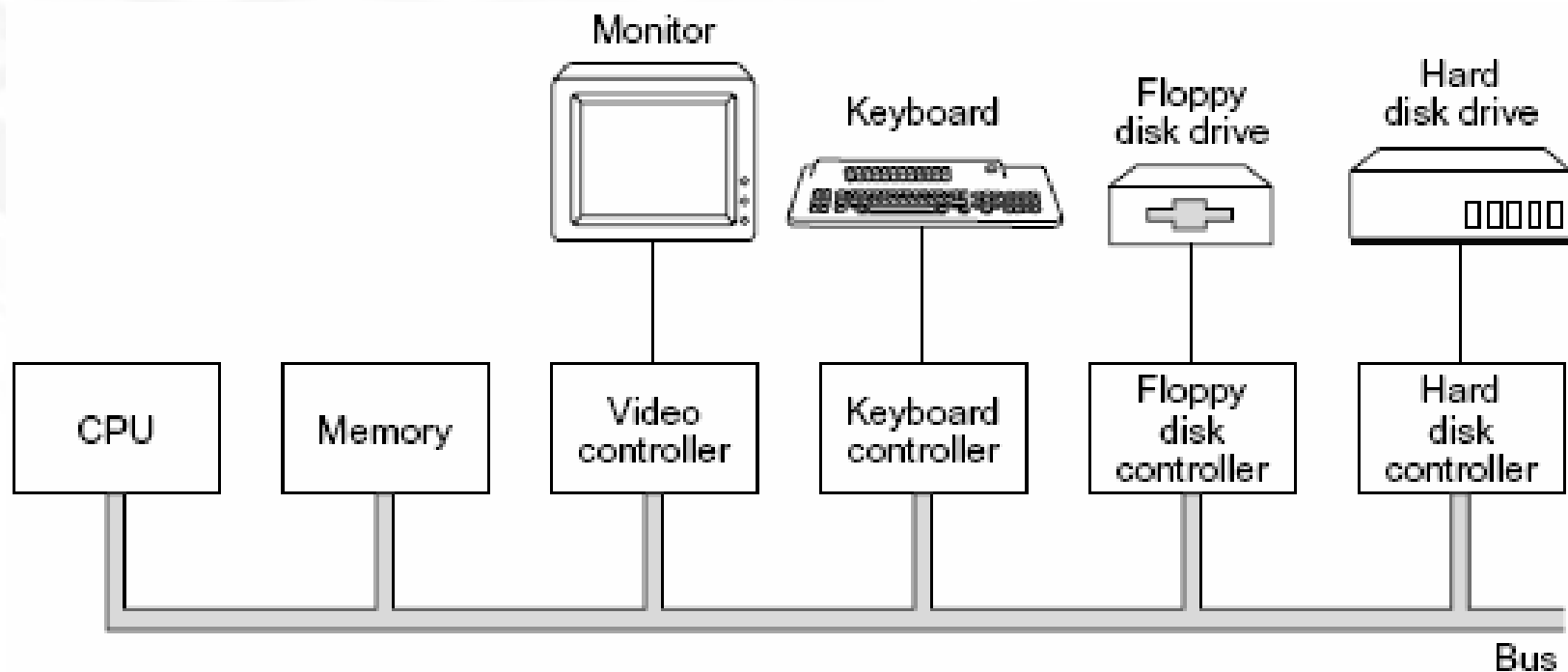
- Geralmente a estrutura física de um PC é composta de uma caixa de metal com uma placa de circuito impresso chamada **placa-mãe**



Entrada e Saída (*Input/Output – I/O*) ⁽³⁾

- Na placa-mãe está
 - a CPU
 - alguns slots para inserção de pentes de memória (ex. módulos DIMM)
 - e vários outros chips
- Ela também tem um **barramento** percorrendo todo o seu comprimento, assim como **soquetes** nos quais serão ligados conectores das **placas controladoras** dos dispositivos de entrada e saída.

Entrada e Saída (*Input/Output – I/O*) (4)

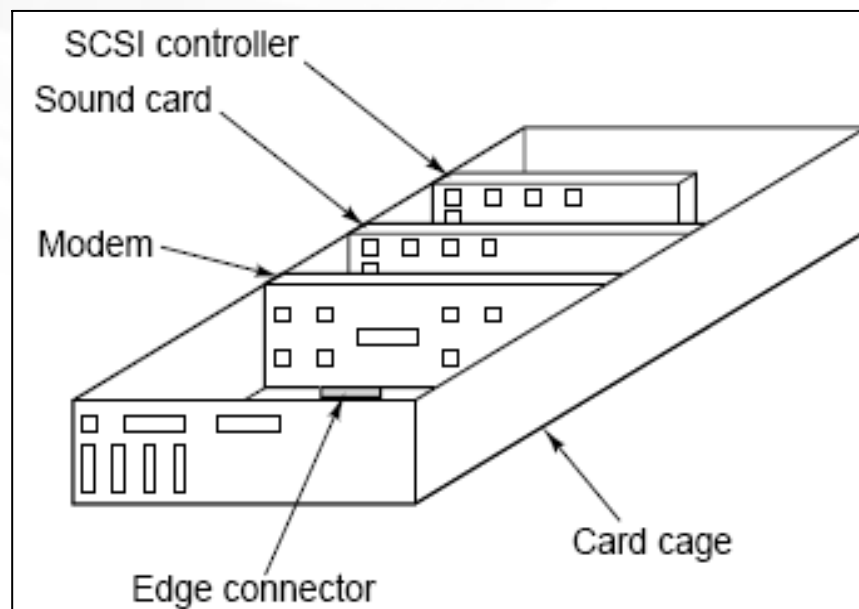


Controladoras de E/S ⁽¹⁾

- Vários dispositivos de I/O distintos podem fazer parte ou estarem conectados a um processador.
- A CPU não se comunica diretamente com cada dispositivo de E/S e sim com "interfaces"
 - Interface de E/S =
Adaptador de Periférico,
Controladora de E/S, Processador de Periférico,
Canal de E/S , *etc.*
 - Compatibilizar as diferentes características.
- Circuitos controladores de I/O são usados para permitir a comunicação entre o processador e os dispositivos.

Controladoras de E/S (2)

- Controladora: seu trabalho consiste em controlar seu dispositivo de E/S e em tratar o acesso do dispositivo ao barramento.
- Placa ligada a um slot livre, ou inserida diretamente na placa-mãe



Controladoras de E/S ⁽³⁾

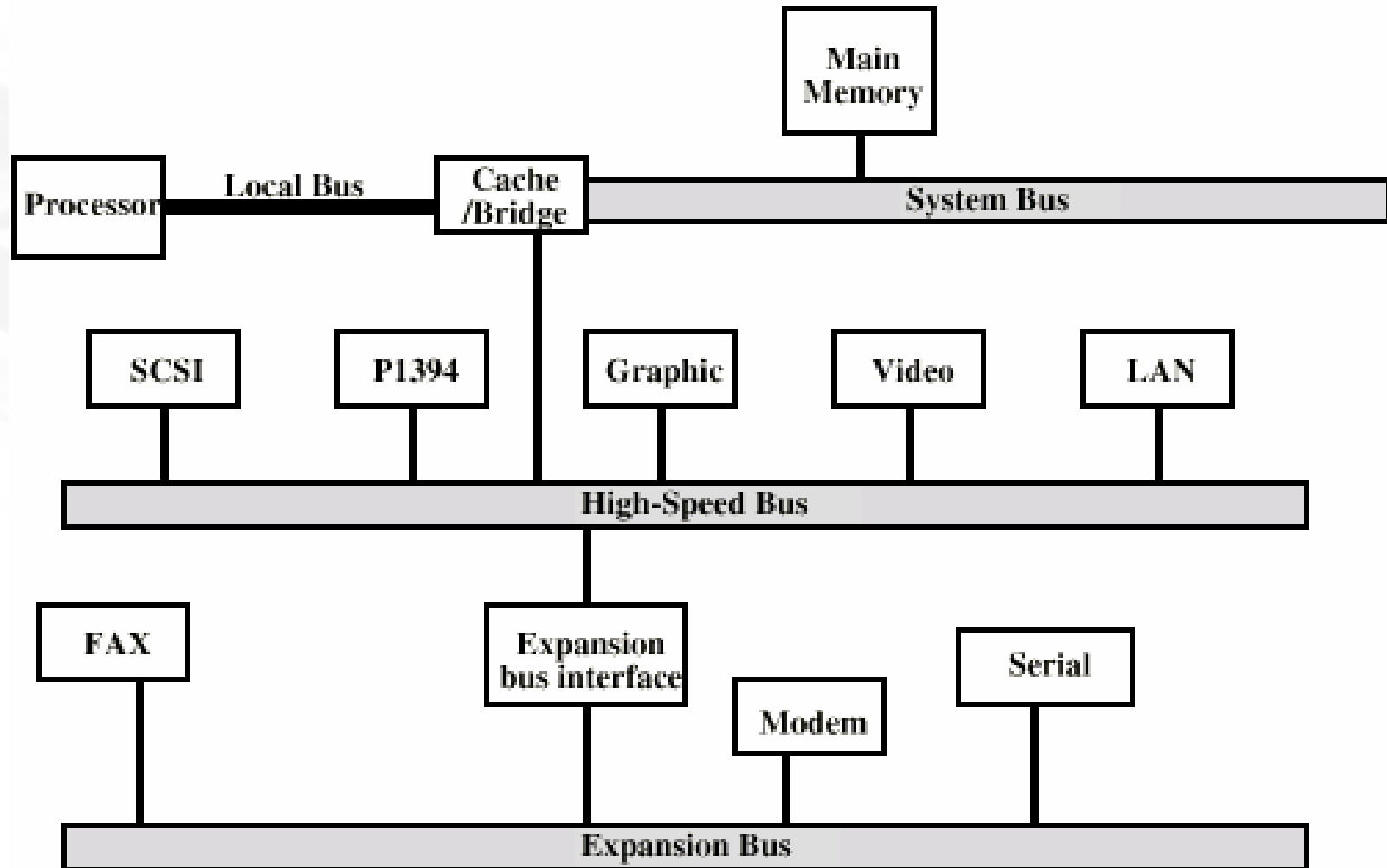
■ Processo geral de comunicação

1. Inicialmente, a CPU interroga o dispositivo, enviando o endereço do dispositivo e um sinal dizendo se quer mandar ou receber dados através da controladora
2. A controladora, reconhecendo seu endereço, responde quando está pronta para receber (ou enviar) os dados.
3. A CPU então transfere (ou recebe) os dados através da controladora
4. A controladora responde confirmando que ...
 - ... recebeu (ou transferiu) os dados (*acknowledge* ou *ACK*)
 - ... ou que não recebeu os dados, neste caso solicitando retransmissão (*not-acknowledge* ou *NAK*).

Barramentos de E/S ⁽¹⁾

- Além dos barramentos de sistema (CPU-Memória), temos os barramentos de E/S
- Barramentos do tipo entrada/saída
 - São mais longos
 - Podem ter muitos dispositivos conectados a ele
 - Muitas vezes necessitam atender uma ampla faixa de bandas passante
 - Não necessariamente têm uma interface direta com a memória
 - Podem usar um dos outros para se comunicar com a memória
- Pode haver mais de 1 barramento
 - Ex: Um de alta velocidade, um de baixa velocidade
 - Diferentes padrões de E/S

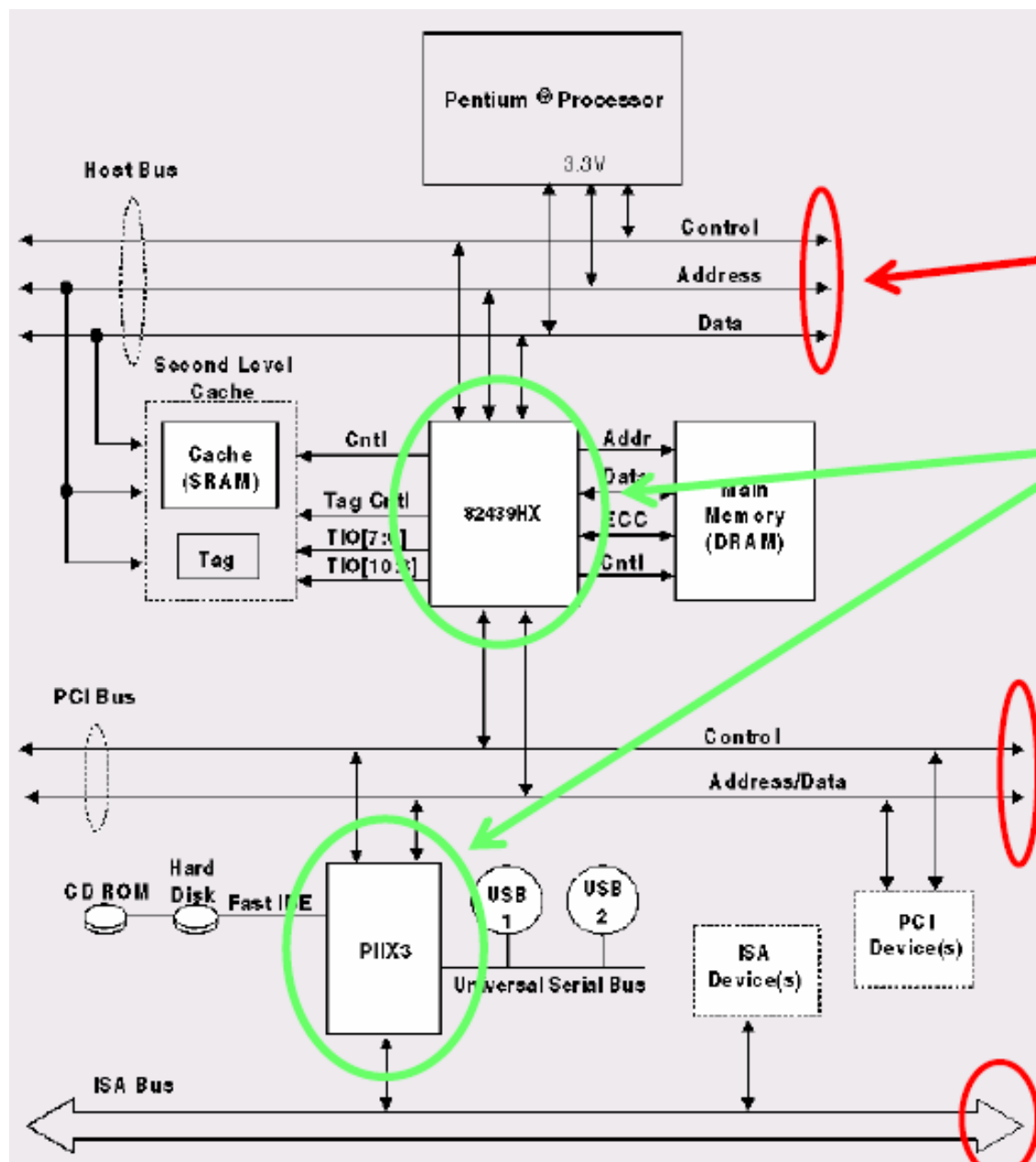
Barramentos de E/S (2)



Barramentos de E/S ⁽³⁾

■ Barramentos do tipo “backplane”

- Nome devido ao local de construção (no “backplane”, estrutura onde as placas são fixadas no chassis da máquina)
- Projetado de modo que processadores, memórias e dispositivos de entrada/saída possam ser interconectados em um único barramento
- Tanto pode ser usado para transportar toda a informação dentro do computador (caso típico dos primeiros PCs)...
... como pode ainda funcionar de interface entre o barramento processador-memória e os barramentos de I/O.



**barramento
processador-memória**

bridge ou hub

**barramento
PCI (barramento
backplane)**

**barramento
I/O**

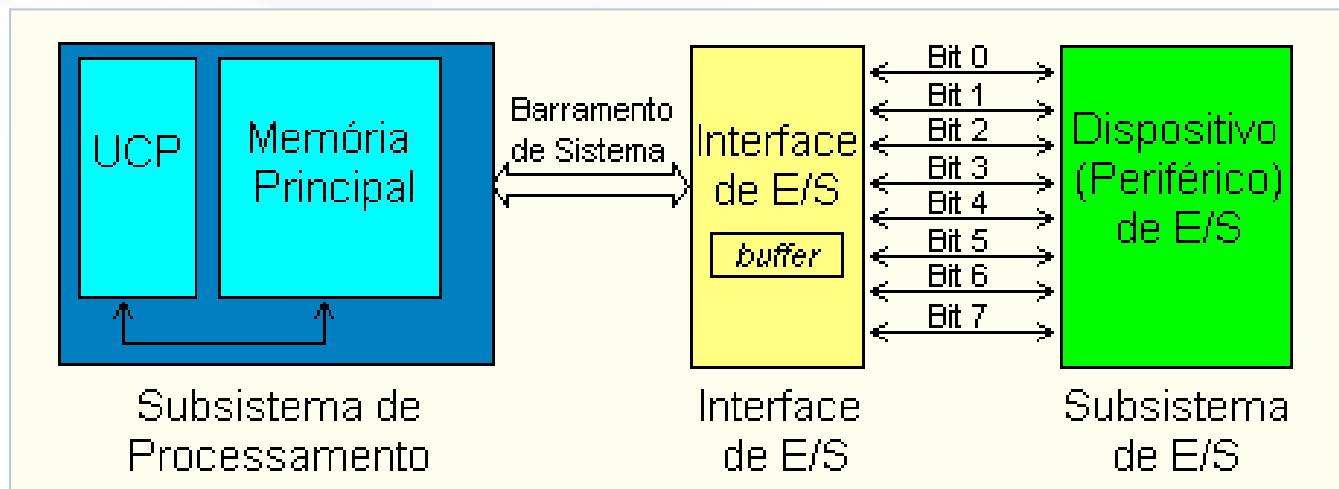
Formas de Comunicação

- Atividades de E/S são assíncronas
 - *Handshaking* pode ser necessário
 - Quando o dispositivo está pronto para recepção/envio de dados
- Qualidade dos dados pode ser incerta
 - Mecanismos de detecção/correção
- Transferências podem ser interrompidas
 - Ex. impressora sem papel
- *Device drivers* devem tratar estes problemas

Formas de Comunicação - Comunicação em Paralelo

(1)

- Grupos de bits transferidos simultaneamente (em geral, byte a byte) através de diversas linhas condutoras dos sinais.
 - taxa de transferência de dados ("*throughput*") é alta
- Por que não usamos sempre comunicação paralela?

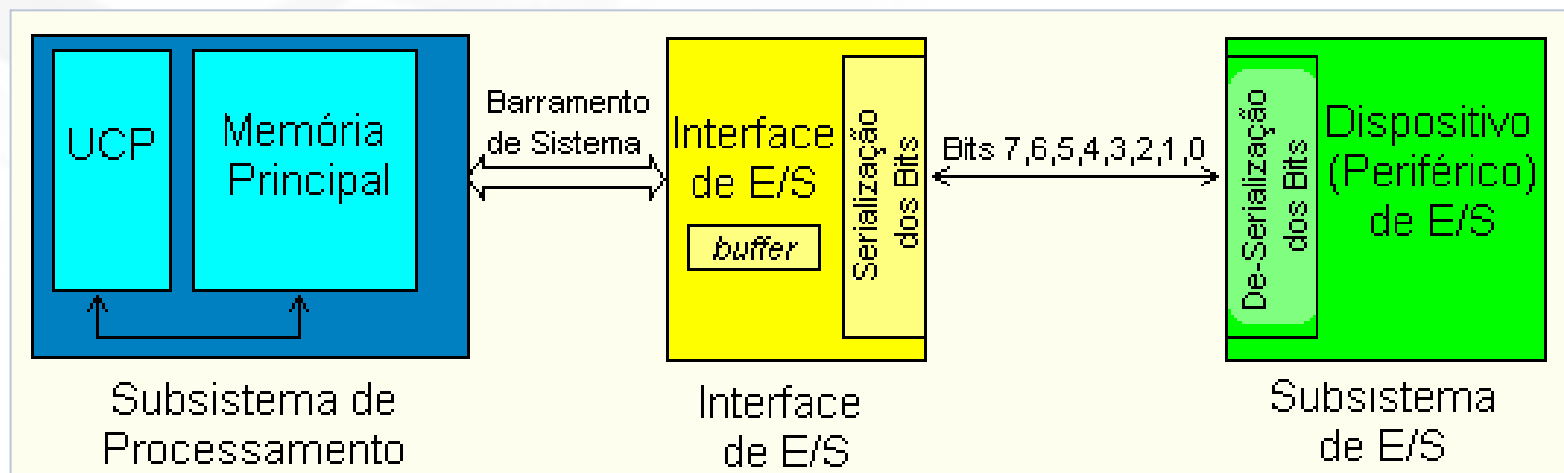


Formas de Comunicação - Comunicação em Paralelo ⁽²⁾

- Processo de transferência em paralelo envolve um controle sofisticado e complexo
 - Mais caro!
- Exemplo de problema
 - Propagação dos sinais no meio físico: se fazer de modo que os sinais (os bits) correspondentes a cada *byte* cheguem simultaneamente à extremidade oposta do cabo
 - Condutores que compõem o cabo usualmente terão pequenas diferenças físicas => a velocidade de propagação dos sinais poderá ser ligeiramente diferente nos diversos fios
 - Um determinado bit x em cada *byte* se propague mais rápido e chegue à extremidade do cabo antes que os outros $n-1$ bits do *byte*.
 - Este fenômeno é chamado *skew*, e as consequências são catastróficas: os bits x chegariam fora de ordem (os *bytes* chegariam embaralhados) e a informação ficaria irrecoverável.
- A comunicação em paralelo se limita a aplicações que demandem altas taxas de transferência
 - unidades de disco, ou que demandem altas taxas de transferência, como CD-ROM, DVD, ou mesmo impressoras

Formas de Comunicação - Comunicação Serial

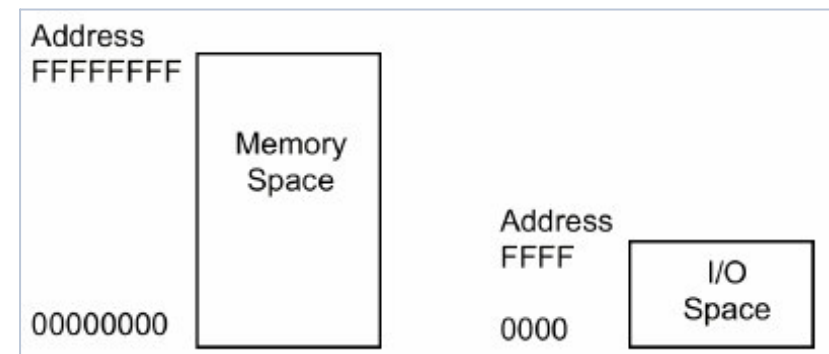
- Os bits são transferidos um a um, através de um único par condutor
- O controle é comparativamente muito mais simples que no modo paralelo
 - Implementação mais barata.
- Muito usada para periféricos mais lentos
 - Teclado, *mouse*, etc.



A transmissão serial tem recebido aperfeiçoamentos importantes (seja de protocolo, de *interface* e de meio de transmissão) que vem permitindo o aumento da velocidade de transmissão -> *USB - Universal Serial Bus*

Endereçamento de Entrada e Saída

- Controladores possuem registradores para comunicação com a CPU
 - E/S é realizada escrevendo-se comandos nos registradores da controladora de E/S
- Endereçamento de entrada e saída
 - Espaços de endereçamento separados (Arq. Pentium)
 - Linha do barramento M/IO
 - Opcodes separados
 - $IN\ reg_d, [end_{16}], OUT\ [end_{16}], reg_d$
 - E/S mapeada em memória
 - Portas E/S operadas como posições de memória
 - Determinados endereços de memória correspondem fisicamente a dispositivos de E/S



Metodologias de Comunicação ⁽¹⁾

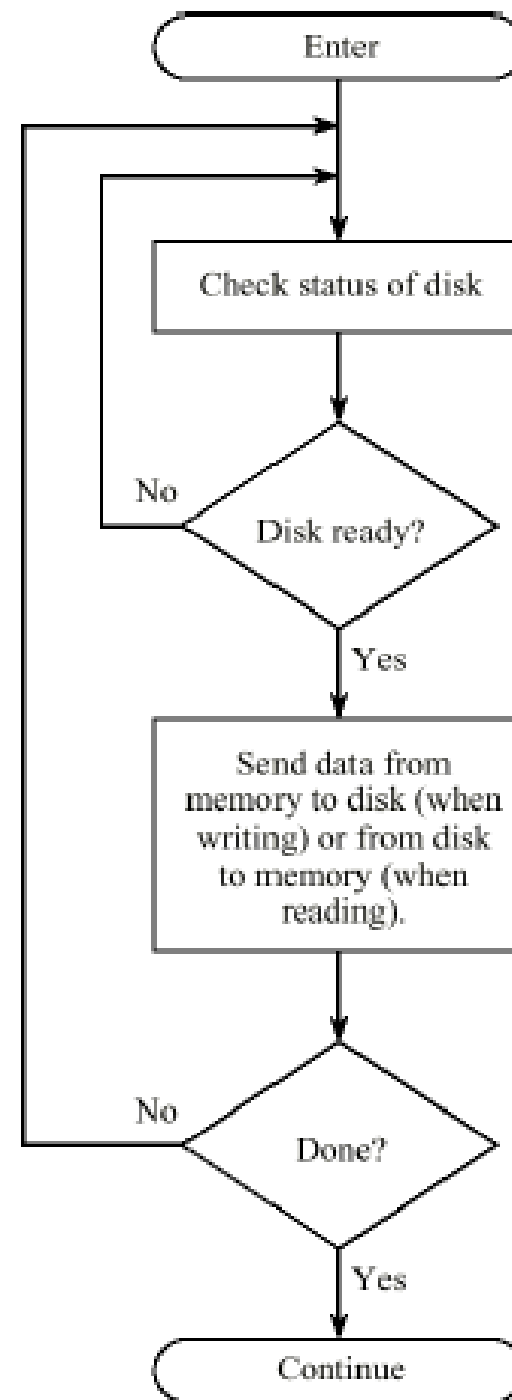
- Entrada e saída programada (ou polling)
- Entrada e saída por interrupção
- Acesso direto à memória (DMA)

Metodologias de Comunicação (2)

- **Entrada e saída programada (ou polling)**
 - Forma mais simples de E/S
 - Frequentemente utilizada em sistemas de pequeno porte ou dedicados
 - A CPU fica dedicada a gerenciar as transferências de dados
 - Desvantagem
 - A CPU gasta a maior parte do tempo em um loop de espera (*busy waiting*) até que o dispositivo de entrada esteja pronto para a transferência

Metodologias de Comunicação (3)

Fluxograma de E/S Programada (*polling*) para uma transferência de disco

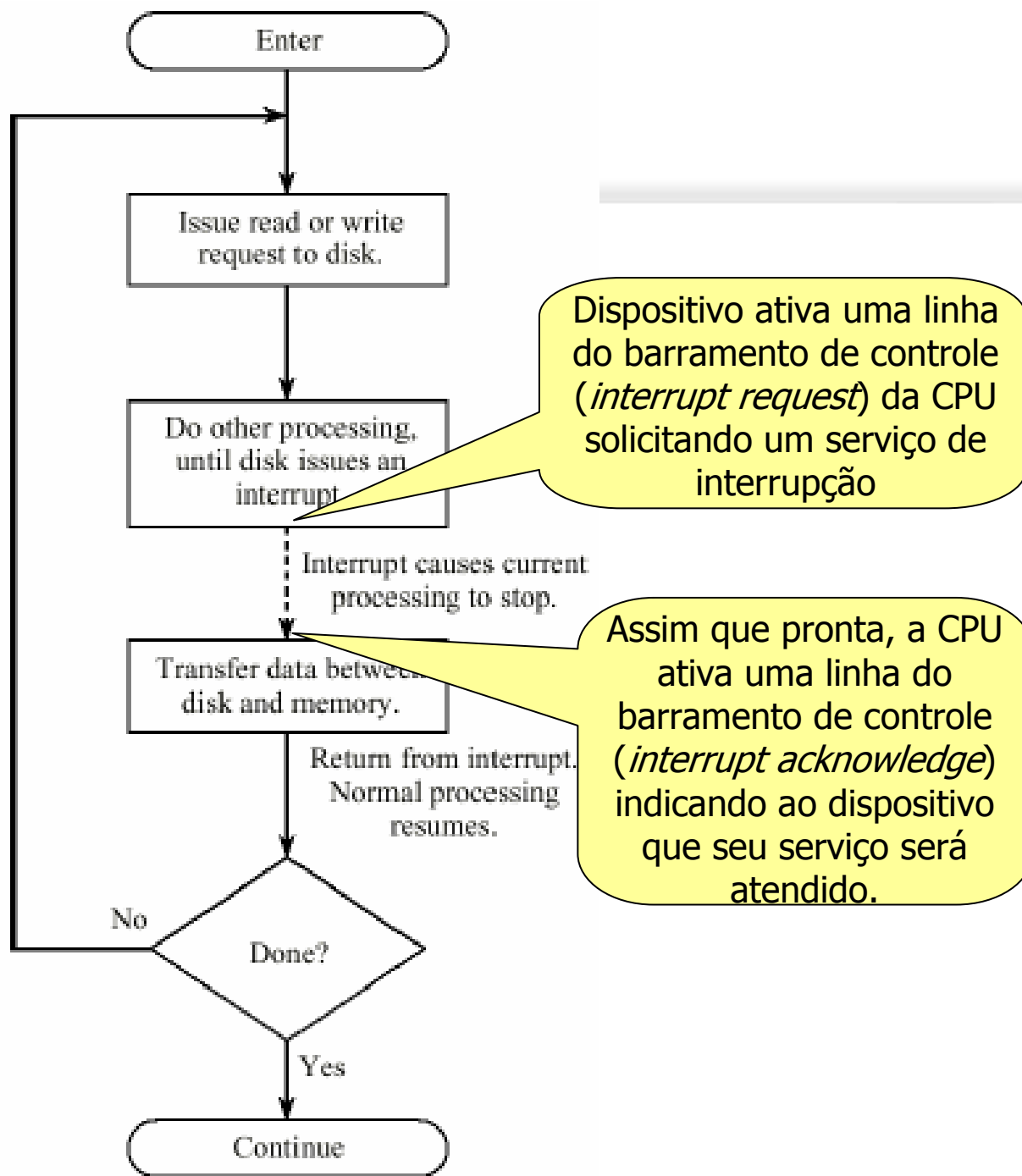


■ Metodologias de Comunicação (4)

■ Entrada e saída por interrupção

- O dispositivo E/S solicita o serviço de transferência apenas quando este estiver pronto para enviar/receber o(s) dado(s)
- A interrupção causa um desvio no fluxo do programa via hardware
 - A interrupção para o programa que está rodando e desvia o fluxo para a **Rotina de Tratamento da Interrupção**
- Rotina de Tratamento da Interrupção
 - Software que realiza o serviço de transferência daquele dispositivo externo que solicitou a interrupção.
 - Verifica a ocorrência de possíveis erros na transferência de dados
- Quando finaliza a rotina de tratamento da interrupção, o processador retoma a execução do programa interrompido

Fluxograma de E/S controlada por interrupção para uma transferência de disco



Metodologias de Comunicação (64)

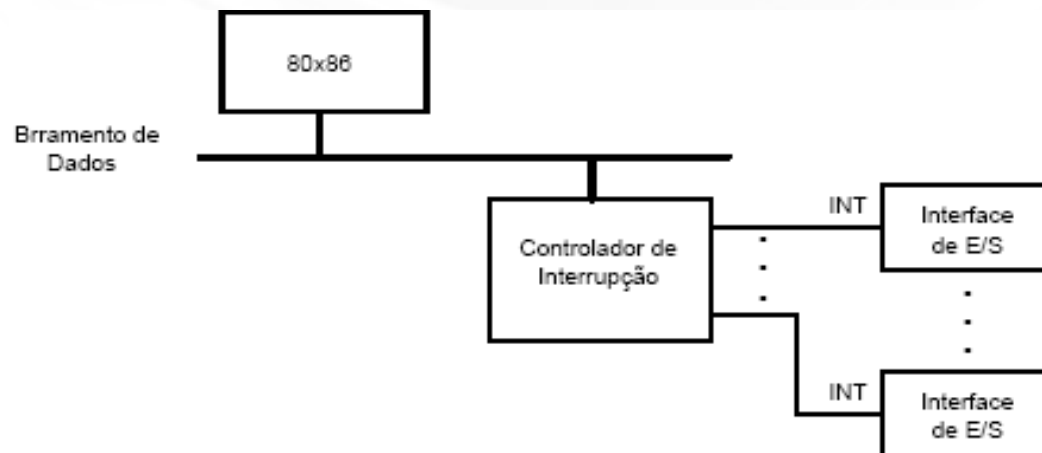
■ **Entrada e saída por interrupção (cont.)**

- É comum existirem várias interfaces diferentes que fazem pedidos de interrupção ao processador
- Cada interface deve ser atendida por uma rotina de serviço de interrupção específica
 - Assim, ao receber um pedido de interrupção, o processador deve determinar qual a rotina de serviço a ser executada
 - Além disso, quando duas ou mais interfaces fazem pedidos de interrupção simultâneos, é necessário decidir qual o pedido de interrupção que será atendido
- Estas duas funções são suportadas por um componente chamado controlador de interrupção (interrupt controller).

Metodologias de Comunicação (8)

■ Entrada e saída por interrupção (cont.)

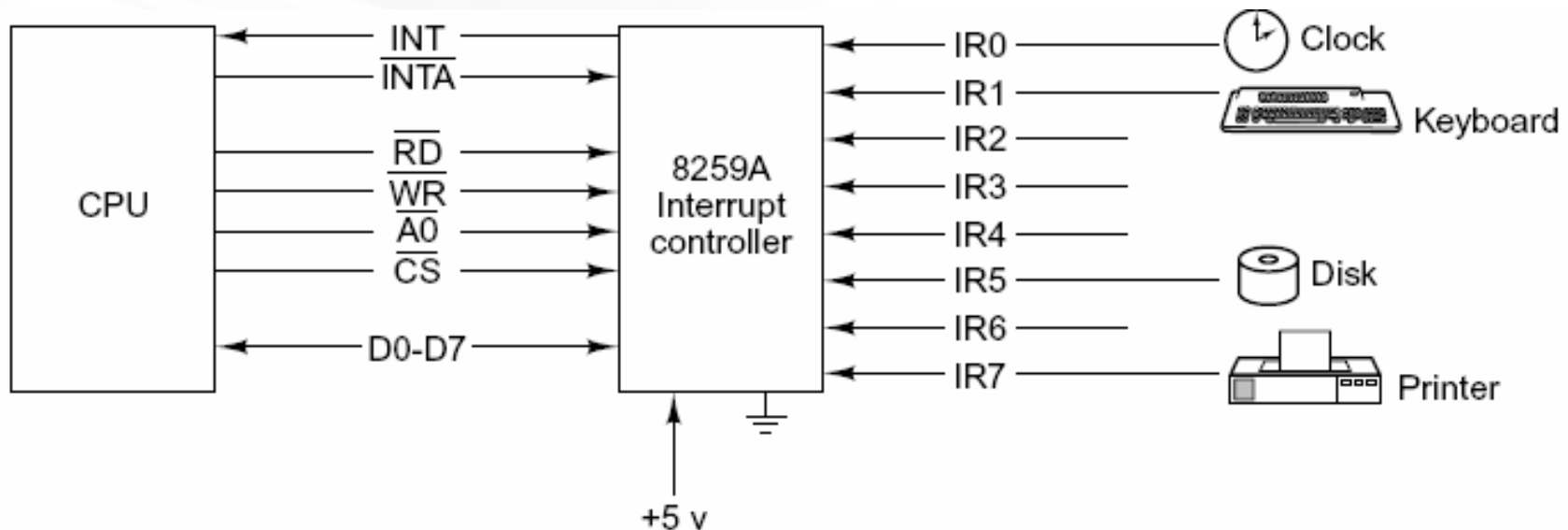
- O sinal de interrupção de cada interface é ligado ao controlador de interrupção.
 - Um número e uma prioridade são atribuídos a cada um destes sinais
- Quando um pedido de interrupção acontece, o controlador de interrupção envia para o processador, através do barramento de dados, o número do pedido.



- No caso de dois ou mais pedidos simultâneos, o controlador decide qual é o pedido com maior prioridade e envia para o processador o número correspondente.

Entrada e Saída (*Input/Output – I/O*) (16)

- Interrupções na família 80x86
 - 8259A permite até 8 interrupções com prioridade (IRQ0 até IRQ7)
 - Se for necessário controlar mais do que 8 dispositivos?
R: Os chips 8259A podem ser ligados em cascata
INT: Pedido de interrupção
 - INTA: Reconhecimento de interrupção



Entrada e Saída (*Input/Output – I/O*) (17)

- Interrupções na família 80x86 (cont.)
 - Quando um dispositivo necessita efetuar alguma operação ele envia um sinal IRx
 - 8259A ativa a linha INT indicando uma interrupção ao processador
 - Quando pronto, o processador envia um sinal (INTA) de volta ao 8259A
 - Ao receber o sinal, o controlador coloca um número inteiro no barramento de dados identificando o tipo de dispositivo
 - A CPU então usa este número para indexar uma tabela de 256 entradas (vetor de interrupções) para encontrar o endereço da rotina de tratamento de interrupção a ser executada
 - Se uma interrupção subsequente ocorrer:
 - De maior prioridade: o 8259A interrompe a CPU pela segunda vez
 - Prioridade menor, ela é suspensa até a primeira terminar
 - Para isto, uma rotina de interrupção deve enviar explicitamente um comando para o 8259A para informar quando ela terminar

Entrada e Saída (*Input/Output – I/O*) (18)

- Uma controladora que lê ou escreve dados da/na memória sem que seja necessária a intervenção do processador utiliza o mecanismo de **Acesso Direto à Memória**
- **DMA (*Direct Memory Access*)**
 - Libera a CPU do intermédio entre dispositivo e memória
 - Uma transferência por DMA essencialmente copia um bloco de memória de um dispositivo para outro
 - A CPU inicia a transferência, mas não executa a transferência
- **Third party DMA**
 - A transferência é realizada pelos controladores DMA que são tipicamente parte do chipset da placa mãe

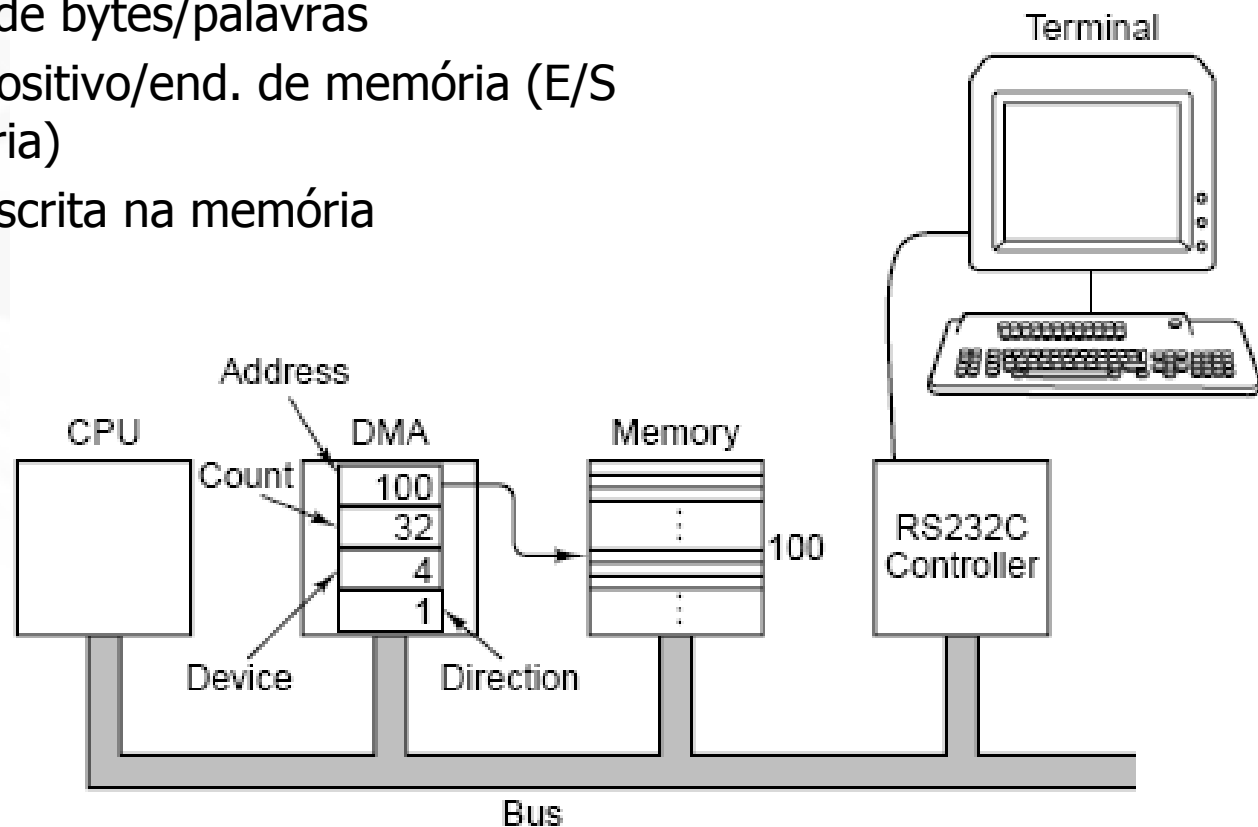
Entrada e Saída (*Input/Output – I/O*) (19)

Address: end. de memória a ser lido/escrito

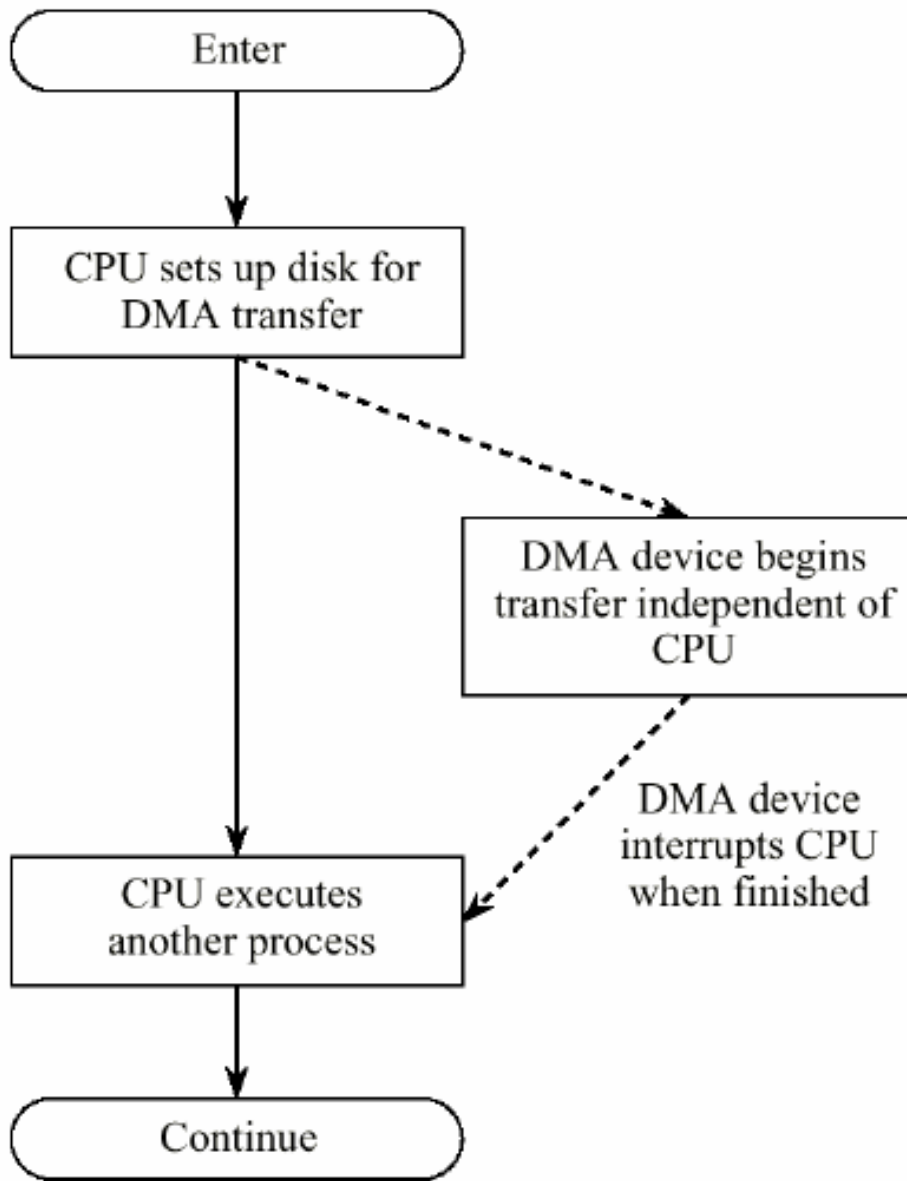
Count: quantidade de bytes/palavras

Device: no. do dispositivo/end. de memória (E/S mapeada em memória)

Direction: leitura/escrita na memória



Entrada e Saída (*Input/Output – I/O*) (20)



Quando termina a transferência dos dados, o controlador DMA força uma **interrupção**, fazendo com que o processador suspenda a execução do programa corrente (**processo**), e inicie a execução da respectiva **rotina de Tratamento da Interrupção**.

Entrada e Saída (*Input/Output – I/O*) (21)

- Em geral, é possível ter vários dispositivos de e/s operando com a técnica de acesso direto à memória
- Para tanto, o controlador de DMA possui várias entradas para pedido de DMA
- O controlador de DMA associa a cada uma destas entradas um conjunto independente de registradores
 - Para armazenar o número de dados a serem transferidos, o endereço inicial e o sentido da transferência
- Um grupo de sinais de controle com seus respectivos registradores formam o chamado **canal de DMA**
- O controlador de DMA se encarrega de arbitrar entre dispositivos que fazem pedidos de DMA simultâneos, usando um esquema de prioridades atribuídas aos canais de DMA.

Referências

- Andrew S. Tanenbaum, ***Organização Estruturada de Computadores***, 5ª edição, Prentice-Hall do Brasil, 2007.
- John L. Hennessy and David A. Patterson, **Arquitetura de Computadores: Uma Abordagem Quantitativa**. 3ª edição. Editora Campus, 2003.
- D. A. Patterson & J. L. Hennessy. **Organização e Projeto de Computadores: a interface hardware/software**. Livros Técnicos e Científicos Editora, Inc. Rio de Janeiro, RJ. Tradução da Segunda Edição americana, 2000.